ACM-ICPC 个人算法模板

算法竞赛常用代码与知识点汇总

整理人: halfpoint 适用场景: ACM-ICPC、CCPC、蓝桥杯、牛客算法竞赛等 代码语言: C++

目录一 · 算法直查表

通用模板

通用模板 6

基础算法

前缀和与差分 7
离散化 8
区间合并 9
MEX 11
归并排序 12
记忆化搜索 13
连续子数组的异或和的和 15
非空子序列的异或和的和 17
灵茶八题 19
常见贪心方法 25
反悔贪心 27
高精度 31

数据结构

树状数组 33
线段树 35
ST 表 39
单调队列 40
单调栈 41

图论

堆优化 Dijkstra 42
SPFA 44
Bellman-Ford 48
DAG 最短路 50
二分图判定 91
匈牙利算法 93
Floyd 53
BFS 55
多源 BFS 56
0-1BFS 59
DFS 60
三色 DFS 61
拓扑排序 64
无向图找环 66
并查集 71
Tarjan 强连通分量 72
割点 75
割边 77
点双连通分量 79
边双连通分量 81
欧拉路径与欧拉回路 85
Prim 87
Kruskal 89
LCA(倍增与树剖) 95
树的遍历(前中后序) 97
树的直径 100
2-SAT 83

字符串

字符串哈希 102
KMP 104
Manacher 105

数论与数学

快速幂 106
费马小定理与逆元 107
矩阵快速幂 108
GCD 与 exgcd 113
欧拉函数 114
线性筛 117
线性筛因数 118
等比数列求和 119
向上取整 120
组合排列 121

动态规划

01 背包 122
背包变式 124
树上背包 126
树形 DP 基础 128
LIS 与 LCS 130

工具速查

GCC 内置函数 132
字符串操作 134
容器操作 136
bitset 139
随机数 141
常用代码片段 143
对拍与打表 144
交互题技巧 148

目录二 · 问题索引树

通用模板

通用模板 6

基础算法

├── 数组预处理
│ ├── 前缀和与差分 7
│ └── 离散化 8
├── 区间合并 9
├── MEX 11
├── 排序
│ └── 归并排序 12
├── 搜索
│ └── 记忆化搜索 13
├── 位运算
│ ├── 连续子数组的异或和的和 15
│ ├── 非空子序列的异或和的和 17
│ └── 灵茶八题 19
├── 贪心
│ ├── 常见贪心方法 25
│ └── 反悔贪心 27
└── 高精度 31

数据结构

├── 单点修改_区间查询
│ └── 树状数组 33
├── 区间修改_区间查询
│ └── 线段树 35
├── 静态区间 RMQ
│ └── ST 表 39
└── 单调性问题
 ├── 单调队列 40
 └── 单调栈 41

图论

├── 最短路
 ├── 单源最短路
 ├── 所有边权都是正数
 └── 堆优化 Dijkstra 42
 ├── 存在负权边
 ├── SPFA 44
 └── Bellman-Ford 48
 └── 有向无环图_正负均可

└── DAG 最短路 50
├── 多源汇最短路
└── Floyd 53
└── 无权或 0-1 边权最短路
├── BFS 55
├── 多源 BFS 56
└── 0-1BFS 59
├── 图的遍历
├── DFS 60
└── BFS 55
├── 环问题
├── 三色 DFS 61
├── 拓扑排序 64
└── 无向图找环 66
├── 为 DP 提供计算顺序
└── 拓扑排序 64
├── 连通性问题
├── 并查集 71
├── Tarjan 强连通分量 72
├── 割点 75
├── 割边 77
├── 点双连通分量 79
├── 边双连通分量 81
└── 2-SAT 83
├── 一笔画问题
└── 欧拉路径与欧拉回路 85
├── 最小生成树
├── Prim 87
└── Kruskal 89
├── 二分图
├── 二分图判定 91
└── 匈牙利算法 93
└── 树上问题
├── LCA(倍增与树剖) 95
├── 树的遍历(前中后序) 97
└── 树的直径 100

字符串

├── 子串是否相等
└── 字符串哈希 102
├── 单模式串匹配
└── KMP 104
└── 最长回文子串

└── Manacher 105

数论与数学

├── 幂运算与逆元
├── 快速幂 106
└── 费马小定理与逆元 107
├── 线性递推加速
└── 矩阵快速幂 108
├── 最大公约数_不定方程
└── GCD 与 exgcd 113
├── 与 n 互质的个数
└── 欧拉函数 114
├── 筛素数
├── 线性筛 117
└── 线性筛因数 118
├── 等比数列求和 119
├── 整除取整技巧
└── 向上取整 120
└── 组合数学
└── 组合排列 121

动态规划

├── 背包
├── 01 背包 122
└── 背包变式 124
├── 树形 DP
├── 树上背包 126
└── 树形 DP 基础 128
└── LIS 与 LCS 130

工具速查

├── GCC 内置函数 132
├── 字符串操作 134
├── 容器操作 136
├── bitset 139
├── 随机数 141
├── 常用代码片段 143
├── 对拍与打表 144
└── 交互题技巧 148

一、通用模板

通用模板

所属分类: 通用 复杂度: -
适用问题

- 所有题目的通用竞赛头部(头文件、宏、快速 IO)

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
const i64 INF = 1e18;
#define endl '\n'
int main() {
  ios::sync_with_stdio(0);
  cin.tie(0);

  return 0;
}
```

二、基础算法

数组预处理

前缀和与差分

所属分类：基础算法 › 数组预处理 复杂度：预处理 $O(n)$ / 区间修改 $O(1)$ / 查询 $O(1)$

适用问题

- 区间批量加减后单点/区间查询
- 区间和快速查询(前缀和)
- 把区间操作转化为两个单点操作

用法

差分数组主要用于解决区间批量修改的问题。它的核心思想是将对区间的操作转化为对两个单点（端点）的操作，从而显著降低时间复杂度。

实现步骤

对于差分数组有两个函数：

1. 初始化差分数组：$d_1 = a_1$，$d_i = a_i - a_{i-1}$（对于 $i > 1$）
2. 区间修改：执行 `d[L] += v; d[R + 1] -= v;`
3. 还原数组：对差分数组 d 求一遍前缀和，即可得到修改后的原数组 a

完整模板

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int n = 5;
    vector<int> a = {0, 1, 2, 3, 4, 5}; // 假设下标从1开始
    vector<int> d(n + 2, 0);

    // 1. 构造差分数组
    for (int i = 1; i <= n; i++) {
        d[i] = a[i] - a[i - 1];
    }

    // 2. 区间修改：在区间 [2, 4] 上加 10
    int L = 2, R = 4, v = 10;
    d[L] += v;
    d[R + 1] -= v;

    // 3. 还原原数组（求前缀和）
    for (int i = 1; i <= n; i++) {
        a[i] = a[i - 1] + d[i];
        cout << a[i] << " ";
    }
    // 输出应为: 1 12 13 14 5

    return 0;
}
```

离散化

所属分类: 基础算法 › 数组预处理 复杂度: $O(n\ log\ n)$

适用问题

- 值域很大、数据稀疏时把原值映射到 $[1, m]$ 连续下标(保持大小关系)
- 为树状数组/线段树等需要下标为键的场景做前置压缩

用法

将值域很大的稀疏数据映射到连续的 $[1, m]$ 下标，保持原大小关系不变。常用于树状数组、线段树等需要以下标为键的场景。

实现步骤

1. 复制原数组，排序后去重，得到有序唯一值列表
2. 遍历原数组，用 `lower_bound` 查出每个值在列表中的排名（1-based），存入映射数组
3. 原值 ↔ 排名通过下标 `i` 对应：`a[i]` 是原值，`rnk[i]` 是排名

完整模板

```
// 1. 排序 + 去重
vector<int> v(a + 1, a + n + 1);
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
int m = v.size(); // 离散化后的值域大小

// 2. 映射原数组为排名 (1-based), 树状数组/线段树可用 rnk[i] 作下标
vector<int> rnk(n + 1);
for (int i = 1; i <= n; i++) {
    //映射离散化后的排名 查询排名也用这种方法
    rnk[i] = lower_bound(v.begin(), v.end(), a[i]) - v.begin() + 1;
}

// 3. 排名和原值的对应关系
//    原值 a[i]        -> 排名 rnk[i]
//    排名 k 对应的原值 -> v[k - 1]
```

区间合并

所属分类: 基础算法 › 区间合并 复杂度: 合并 $O(n\ log\ n)$(输入有序 $O(n)$) / 查询 $O(log\ n)$
适用问题
- 合并重叠(或相邻)区间
- 查询下标 x 属于哪个合并区间及区间内偏移量
- 按排名快速取第 k 个合并区间的左右边界
- 在线顺序扩展可达区间

区间合并常用于查找下标所在区间及相对位置, 在图论、DP、扫描线中都很常见。
1.基础静态合并（离线预处理）
给定 n 个区间 [l, r]，合并所有重叠（或相邻）的区间。 核心思路：按左端点排序，线性扫描维护当前合并块的 [L, R]。 若输入区间保证有序, 时间复杂度 O(n); 否则 O(n log n)。
2.静态区间的高效查询
合并后得到 m 个互不相交的区间，存放在 merged 数组（下标从 0 开始）。
查询下标 x 属于哪个合并区间，以及在该区间内的偏移量（第几位）。
给定排名 k（区间一、区间二...），快速输出该区间的 [l, r]。
因为合并后的左端点严格递增，我们可以提取出所有的 L_i，利用 upper_bound 二分定位。

```
#include <bits/stdc++.h>
#define endl '\n'
using i64 = long long;
using namespace std;

vector<pair<i64, i64>> merge_seg(vector<pair<i64, i64>> &seg) { //合并并且返回合并好的区间数组
  vector<pair<i64, i64>> res;
  if(seg.empty()) {
    return res;
  }
  sort(seg.begin(), seg.end()); // 按 l 排序, l 相同按 r

  i64 L = seg[0].first, R = seg[0].second;
  for(auto &i:seg) {
    auto[l, r] = i;
    // 如果题意是“区间覆盖连续整数”（如 [1,2] 和 [3,4] 算连通),
    // 把 if (l > R) 改成 if (l > R + 1) 即可。
    if(l > R + 1) {
      res.push_back({L, R});
      L = l, R = r;
    } else {
      R = max(r, R);
    }
  }
  res.push_back({L, R});
  return res;
}

pair<i64, i64> get_edge(i64 k, vector<pair<i64, i64>> &fin_seg) {//输入k 返回第k个合并区间的左右边界
  return {fin_seg[k - 1].first, fin_seg[k - 1].second};
}

//查询下标 x属于哪个合并区间 以及在该区间内的偏移量(第几位)
pair<i64, i64> get_pos(i64 x, vector<pair<i64, i64>> &fin_seg) {
  vector<i64> seg_L;//若要多次查询 可以把这个的初始化放在外面 优化时间
  for(auto &i:fin_seg) {
    seg_L.push_back(i.first);
  }
```

```
  // 找到第一个左端点 > x 的位置
  i64 idx = upper_bound(seg_L.begin(), seg_L.end(), x) - seg_L.begin() - 1;
  if(idx < 0 || idx >= fin_seg.size()) return {-1, -1};
  if(x > fin_seg[idx].second) return {-1, -1};
  return {idx, x - fin_seg[idx].first};//区间 相对位置
}

int main() {
  int N;
  cin >> N;
  vector<pair<i64, i64>> seg(N); //原始区间 记录l, r
  for(auto &i:seg) {
    cin >> i.first >> i.second;
  }
  vector<pair<i64, i64>> fin_seg = merge_seg(seg);
  i64 ans = 0;
  for(auto i:fin_seg) {
    ans += i.second - i.first + 1;
  }
  cout << ans;

}
```

在线版(顺序扩展可达区间)

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

// 在线维护：初始位置在 x，顺序读取 m 个区间
void solve() {
    i64 n, x, m;
    cin >> n >> x >> m;

    i64 L = x, R = x; // 初始可达范围只有一个点

    for (int i = 0; i < m; i++) {
        i64 l, r;
        cin >> l >> r;

        // 关键判断：两个闭区间 [L,R] 和 [l,r] 是否有交集
        // 有交集的条件：l <= R && L <= r
        if (l <= R && L <= r) {
            L = min(L, l); // 左边界只能向左扩
            R = max(R, r); // 右边界只能向右扩
        }
    }

    cout << R - L + 1 << '\n'; // 输出最终可达区间长度
}
```

MEX

所属分类*:* 基础算法 › *MEX* 复杂度*:* -

适用问题

- 求一个集合(或多重集合)中最小的没有出现过的非负整数

定义

MEX(Minimum EXcluded)是指一个集合(或多重集合)中最小的没有出现过的非负整数。

例如:

- MEX($\{0, 1, 3\}$) $= 2$, 因为 0、1 出现了, 2 没出现
- MEX($\{1, 2, 3\}$) $= 0$, 因为 0 没出现
- MEX($\emptyset$) $= 0$, 空集没有元素, 最小的非负整数 0 没出现

排序

归并排序

所属分类: 基础算法 › 排序 复杂度: $O(n\ log\ n)$

适用问题

- 稳定排序(相等元素的相对顺序不变)
- 分治模板: 递归拆分 + 合并
- 本模板未含逆序对计数(常见变式)

模板

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
void merge_sort(int l, int r, vector<i64> &a, vector<i64> &tmp) {
  if(l >= r) return; //避免空区间和奇怪的东西所以加
  i64 mid = l + ((r - l) >> 1);

  i64 i = l, j = mid + 1, k = l;

  merge_sort(l, mid, a, tmp);
  merge_sort(mid + 1, r, a, tmp);
  if(a[mid] <= a[mid + 1]) return; //优化

  while (i <= mid && j <= r) {
    if(a[i] <= a[j]) {
      tmp[k++] = a[i++];
    } else {
      tmp[k++] = a[j++];
    }
  }

  while (i <= mid) {
    tmp[k++] = a[i++];
  }

  while (j <= r) {
    tmp[k++] = a[j++];
  }

  for(int p = l; p <= r; p++) {
    a[p] = tmp[p];
  }


}

int main() {
  int n;
  cin >> n;
  vector<i64> a(n);
  vector<i64> tmp(n);
  for (auto &i:a) cin >> i;
  merge_sort(0, a.size() - 1, a, tmp);
  for(auto &i:a) cout << i << ' ';
}
```

搜索

记忆化搜索

所属分类: 基础算法 › 搜索 复杂度: 状态数 × 单状态转移代价 备注: *2026-10-18* 初稿, 待你核对

适用问题

- 有重叠子问题的搜索, 例: P1434 滑雪

模板 1: 二维网格

适用于在地图上向上下左右移动的场景。

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

int n, m;
int dx[4] = {0, 0, 1, -1};
int dy[4] = {1, -1, 0, 0};
int mp[105][105];
int memo[105][105]; // 初始化为 0，表示没算过（如果答案可能是0，则初始化为 -1）

bool in_bound(int x, int y) {
    return x >= 1 && x <= n && y >= 1 && y <= m;
}

// 核心 dfs：返回从 (x, y) 出发的最优答案
int dfs(int x, int y) {
    // 1. 记忆化查询（如果之前算过，直接拿来用）
    if (memo[x][y] != 0) return memo[x][y];

    // 2. 当前状态默认值（通常是 1，因为至少包含当前格子）
    int best = 1;

    // 3. 遍历所有邻居（这里是四个方向）
    for (int i = 0; i < 4; i++) {
        int nx = x + dx[i];
        int ny = y + dy[i];
        if (in_bound(nx, ny) && mp[nx][ny] < mp[x][y]) { // 按题目条件修改
            // 转移：一步代价(通常是 1) + 子问题的答案
            best = max(best, 1 + dfs(nx, ny));
            // 如果是求和：best += dfs(nx, ny);
            // 如果是求方案数：best += dfs(nx, ny);
        }
    }

    // 4. 保存结果到备忘录
    memo[x][y] = best;
    return best;
}
```

模板 2: 有向图(邻接表)

适用于任意有向图。

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

int n, m;
```

```cpp
vector<vector<int>> graph; // 邻接表
vector<int> memo;          // 记忆化数组，初始化为 -1

// 核心 dfs：返回从节点 u 出发的最优答案
int dfs(int u) {
    // 1. 记忆化查询（如果之前算过，直接返回）
    if (memo[u] != -1) return memo[u];

    // 2. 当前状态默认值（默认为0，因为没有邻居时就走0步）
    int best = 0;

    // 3. 遍历所有后继节点 v
    for (int v : graph[u]) {
        // 转移：走一步(1) + v 的结果
        best = max(best, 1 + dfs(v));
        // 同样，如果是求方案数/和，换成相应的运算
    }

    // 4. 保存结果到备忘录
    memo[u] = best;
    return best;
}

int main() {
    // ... 读入 n, m, 建图 ...
    memo.assign(n + 1, -1); // 初始化为 -1 表示未计算

    // 遍历所有节点（图可能不连通）
    for (int i = 1; i <= n; i++) {
        dfs(i); // 只需要调用即可，答案会自动存入 memo
    }
    // 输出 memo[i] 就是每个点的答案
}
```

位运算

连续子数组的异或和的和

所属分类: 基础算法 › 位运算 复杂度: $O(n\ log\ A)$
适用问题
- 求所有连续子数组异或和之和
- 对应题目: U360489 灵茶八题 - 子数组 ^w+

典题
题目描述
给你一个长为 n 的数组 a, 输出它的所有连续子数组的异或和的和。
例如 `a = [3, 5]` 有三个连续子数组 `[3]`、`[5]`、`[3, 5]`, 异或和分别为 3、5、6, 所以答案为 $3 + 5 + 6 = 14$。
输入格式
第一行输入一个整数 n $(1 \le n \le 10^5)$。
第二行输入 n 个非负整数, 表示数组 a 中的元素 $(0 \le a[i] \le 10^6)$。
输出格式
一个整数, 表示 a 的所有连续子数组的异或和的和。
输入输出样例 #1
输入 #1

```
2
3 5
```

输出 #1

```
14
```

输入输出样例 #2
输入 #2

```
1
1
```

输出 #2

```
1
```

这是一道非常典的题目 很有可能能用上
对于子数组以及 ^ | & 的题目 我们通常考虑贡献法及位运算
思路
对于子数组与 ^、|、& 的题目, 通常考虑**贡献法 + 位运算**。
把整数按二进制拆开: X = Σ_b (X 的第 b 位) × 2^b。
所有子数组异或和的总和 = Σ_b 2^b × (异或和第 b 位为 1 的子数组个数)。
对固定的第 b 位: 子数组 [l, r] 的异或和 = `pre[r]` ⊕ `pre[l - 1]`, 其第 b 位为 1 当且仅当 `pre[r]` 与 `pre[l - 1]` 的第 b 位**不同**。
统计 `pre[0..n]` 中第 b 位为 0 的个数 cnt0、为 1 的个数 cnt1 (cnt0 + cnt1 = n + 1), 则这一位的贡献 = cnt0 × cnt1 × 2^b。
代码如下

```
#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
using i64 = long long;

i64 ksm(i64 a,i64 b) {
  i64 res = 1;
  while (b) {
    if(b & 1LL) res *= a;
    a *= a;
    b >>= 1LL;
```

```
  }
  return res;
}

int main() {
  i64 n;
  cin >> n;
  vector<i64> a(n + 1);
  vector<i64> pre(n + 1);
  pre[0] = 0;
  a[0] = 0;
  for(int i = 1; i <= n; i++) {
    cin >> a[i];
    pre[i] = a[i] ^ pre[i - 1];
  }
  vector<i64> cnt0(32, 0), cnt1(32, 0);

  for(int i = 0; i <= n; i++) {
    i64 cnt = 0;
    while (cnt <= 31) {
      if(pre[i] & (1LL << cnt)) cnt1[cnt]++;
      else cnt0[cnt]++;
      cnt++;
    }
  }
  i64 ans = 0;
  for(int i = 0; i <= 31; i++) {
    ans += cnt0[i] * cnt1[i] * ksm(2, i);
  }
  cout << ans << endl;

}
```

非空子序列的异或和的和

所属分类: 基础算法 › 位运算 复杂度: $O(n)$
适用问题
- 求所有非空子序列异或和之和
- 结论: 答案 = 2^(n-1) × 所有数的按位或

这道题比子数组那题简单很多, 关键结论是:
所有非空子序列的异或和的总和 = 2^(n-1) × (所有数的按位或)
下面详细推导。
1. 按二进制位独立考虑
设数组长度为 n。对于二进制的第 b 位:
- 统计数组中第 b 位为 1 的元素个数, 记为 c_1。
- 第 b 位为 0 的元素个数为 $c_0 = n - c_1$。

一个子序列的异或和在第 b 位为 1, 当且仅当子序列中选了**奇数个**第 b 位为 1 的元素。 第 b 位为 0 的元素选或不选, 都不影响这一位的异或结果。
2. 统计第 b 位为 1 的子序列数量
- 从 c_1 个 “1” 元素中选奇数个的方案数:
  - 若 $c_1 = 0$, 方案数为 0。
  - 若 $c_1 > 0$, 选奇数个和选偶数个的方案数相等, 各占一半。总方案数是 $2^{(c_1)}$, 所以奇数个方案数为 $2^{(c_1 - 1)}$。
- 从 c_0 个 “0” 元素中任意选, 方案数为 $2^{(c_0)}$。

因此, 第 b 位为 1 的子序列数量为:
- 若 $c_1 > 0$: $2^{(c_1 - 1)} \times 2^{(c_0)} = 2^{(c_1 + c_0 - 1)} = 2^{(n - 1)}$

若 $c_1 = 0$, 贡献为 0。
3. 总和的表达式
每一位 b 对总和的贡献是:
(该位为 1 的子序列数量) × 2^b
如果数组中至少有一个元素在第 b 位为 1 (即 $c_1 > 0$), 那么贡献就是:
$2^{(n - 1)} \times 2^b$
把所有这样的位加起来, 正好是:
$2^{(n - 1)}$ × (对“存在元素第 b 位为 1”的所有位 b 求 2^b 之和)
而括号里的部分, 正是所有元素的**按位或** (OR) 的值。
所以答案就是:
ans = $2^{(n - 1)}$ × (所有数的按位或) mod $(10^9 + 7)$
4. 验证样例
样例: `a = [1, 2, 3]`
- 按位或: `1 | 2 | 3 = 3`
- $n = 3$, $2^{(n-1)} = 2^2 = 4$
- 答案: $4 \times 3 = 12$, 与题目输出一致。

样例: `a = [3, 5]`
- 按位或: `3 | 5 = 7`
- $n = 2$, $2^{(n-1)} = 2$
- 答案: $2 \times 7 = 14$, 与之前子数组的样例一致 (巧合, 但正确)。

5. 代码实现

```cpp
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1e9 + 7;

long long qpow(long long a, long long b) {
    long long res = 1;
    while (b) {
        if (b & 1) res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
```

```
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    long long or_val = 0;
    for (int i = 0; i < n; i++) {
        long long x;
        cin >> x;
        or_val |= x;
    }

    long long ans = qpow(2, n - 1) * (or_val % MOD) % MOD;
    cout << ans << '\n';

    return 0;
}
```

6. 复杂度分析

- 时间复杂度: $O(n + \log n)$, 主要是读入和快速幂。
- 空间复杂度: $O(1)$, 不需要存储整个数组。

对于 $n \leq 2 \times 10^5$, 完全没问题。

7. 注意事项

- 模数是 $10^9 + 7$。
- $2^{(n-1)}$ 用快速幂计算, 注意 $n = 1$ 时 $2^0 = 1$。
- 按位或的结果可能很大 (最大 10^9 左右), 取模后再乘。
- 输入元素可能为 0, 不影响按位或。

这样就能轻松通过。

灵茶八题(子数组/子序列 × 和/异或和)

所属分类: 基础算法 › 位运算(贡献法) 用途: 八种组合的做法速记(题单: 洛谷《灵茶八题》; 下标从 0, 位从 0) 备注: 2026-10-18 初稿, 待你核对

适用问题

- 子数组/子序列 与 元素和/异或和 的八种组合的求法速查

题	关键结论/做法	复杂度
1. 子数组 +w+	贡献法: `a[i]`(0-based) 出现在 (i+1)(n-i) 个子数组中, 答案 = Σ (i+1)(n-i) · a[i]	O(n)
2. 子数组 ^w^	n 偶 → 0; n 奇 → 偶数下标(0-based)的 `a[i]` 异或和	O(n)
3. 子数组 ^w+	二进制拆位: 第 k 位贡献 cnt0 · cnt1 · 2^k (cnt0/cnt1 为 pre[0..n] 该位 0/1 的个数)	O(n log A)
4. 子数组 +w^	逐位拆解 + 余数区间配对计数: 统计 `(s[j]-s[i]) mod 2^(k+1)` ∈ `[2^k, 2^(k+1))` 的配对个数奇偶(异或 Fenwick)	O(n log n log S)
5. 子序列 +w+	每个 `a[i]` 出现在 2^(n-1) 个非空子序列, 答案 = 2^(n-1) · Σa[i] mod 1e9+7	O(n + log n)
6. 子序列 ^w^	n = 1 → `a[0]`; n ≥ 2 → 0	O(n)
7. 子序列 ^w+	答案 = 2^(n-1) · (所有数的按位或) mod 1e9+7	O(n + log n)
8. 子序列 +w^	子集和奇偶 DP: `dp[j] ^= dp[j-a[i]]`, 答案 = 所有 dp[j] = 1 的 j 的异或和	O(n · 2^16)

子数组 +w+

对应题目: U360300 灵茶八题 - 子数组 +w+。

- 求: 所有连续子数组的元素和之和。
- 做法(贡献法): `a[i]`(0-based) 被 (i+1)(n-i) 个子数组包含, 答案 = Σ (i+1)(n-i) · a[i]。
- 复杂度: O(n)。
- 注意: 答案可能超出 int, 必须用 `long long`; 极端值域下 64 位也可能溢出。

```cpp
#include <bits/stdc++.h>
using i64 = long long;
using namespace std;
int main() {
  int n;
  cin >> n;
  i64 sum = 0;
  i64 cnt = 1;
  vector<i64> a(n + 1);

  for(int i = 1; i <= n; i++) cin >> a[i];

  for(int i = 1; i <= n; i++) {
    sum += (i64)(i) * (n - i + 1) * a[i];
  }
  cout << sum;
}
```

子数组 ^w^

对应题目: U360487 灵茶八题 - 子数组 ^w^。

- 求: 所有连续子数组的异或和的异或和。
- 结论: n 偶 → 0; n 奇 → 偶数下标(0-based)的 `a[i]` 异或和。
- 理由: `a[i]` 出现在 (i+1)(n-i) 个子数组中, 出现次数为偶数时在异或里抵消。n 偶: n-i 与 i 同奇偶, (i+1)(n-i) 恒为偶 → 0; n 奇: n-i 与 i 反奇偶, (i+1)(n-i) 为奇数 ⟺ i 为偶 → 只剩偶数下标。
- 注: 该代码用 1-based 的出现次数 i · (n-i+1) 的奇偶判断, 与 0-based 的 (i+1)(n-i) 是同一个数, 等价。
- 复杂度: O(n)。

- 注意: `i*(n-i+1)` 需在 long long 下计算(n>46340 时 int 会先溢出)。

```cpp
#include <bits/stdc++.h>
using i64 = long long;
using namespace std;
int main() {
  int n;
  cin >> n;
  i64 ans = 0;
  vector<i64> a(n + 1);

  for(int i = 1; i <= n; i++) cin >> a[i];

  for(int i = 1; i <= n; i++) {
    if((i64)((i) * (n - i + 1)) & 1) {
      ans ^= a[i];
    }
  }
  cout << ans;
}
```

子数组 ^w+

对应题目: U360489 灵茶八题 - 子数组 ^w+。

- 求: 所有连续子数组的异或和之和。
- 做法(二进制拆位): 设 `pre[i] = a[0] ^ ... ^ a[i-1]`(`pre[0] = 0`), 子数组 `[l, r-1]` 的异或和 = `pre[r] ^ pre[l]`, 其第 k 位为 1 ⟺ `pre[l]` 与 `pre[r]` 的第 k 位不同。记 `pre[0..n]` 中第 k 位为 1 的个数为 cnt1(0 的个数 cnt0 = n+1-cnt1), 则该位贡献 = cnt0 · cnt1 · 2^k。
- 复杂度: O(n log A)。
- 完整代码与推导见「连续子数组的异或和的和」小节。

子数组 +w^

对应题目: U360500 灵茶八题 - 子数组 +w^。

- 求: 所有连续子数组的元素和的异或和。
- 做法(逐位拆解): 记前缀和 `s[i] = a[0] + ... + a[i-1]`, 子数组 `[i, j-1]` 的和 = `s[j] - s[i]`(i < j)。整数 x 的第 k 位 = 1 ⟺ `(x mod 2^(k+1)) ∈ [2^k, 2^(k+1))`, 所以按位独立: 答案第 k 位 = `(s[j]-s[i]) mod 2^(k+1) ∈ [2^k, 2^(k+1))` 的配对 (i, j)(i < j) 个数的奇偶。
- 实现: 对每个 k 只保留 `r[i] = s[i] mod 2^(k+1)`。顺序扫 j, 用值域上的「异或 Fenwick」(插入 = 翻转一位, 前缀查询 = 前缀个数的奇偶)统计已插入的 i < j 中 r[i] 落在 `[r[j]+1, r[j]+2^k]`(循环区间)的个数奇偶。只关心配对个数的奇偶, 所以计数用异或代替加法。
- 推荐: 先做 ARC092D(同思路: 逐位 + 余数区间计数)。
- 复杂度: O(n log n log S)(S 为前缀和范围)。
- 说明: 用户原文件 U360500 为测试残迹, 本节代码为本次新写; 已与 O(n²) 暴力对拍 833 组(n ≤ 8、小值域)完全一致, 0 mismatch。

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

// 异或版树状数组(只记奇偶): toggle 翻转一位, pref 求前缀个数的奇偶
struct XorFenwick {
  int n;
  vector<int> t;
  XorFenwick(int n) : n(n), t(n + 1, 0) {}
  void toggle(int i) {
    for (; i <= n; i += i & -i) t[i] ^= 1;
  }
  int pref(int i) {
    int res = 0;
```

```
    for (; i > 0; i -= i & -i) res ^= t[i];
    return res;
  }
};

// 所有连续子数组元素和的异或和：对每个二进制位 k，统计 (i<j) 中
// (s[j]-s[i]) mod 2^{k+1} ∈ [2^k, 2^{k+1}) 的配对个数的奇偶
int main() {
  int n;
  cin >> n;
  vector<i64> s(n + 1, 0);
  for (int i = 1; i <= n; i++) {
    i64 x;
    cin >> x;
    s[i] = s[i - 1] + x;
  }

  i64 mn = s[0], mx = s[0];
  for (int i = 1; i <= n; i++) {
    mn = min(mn, s[i]);
    mx = max(mx, s[i]);
  }
  i64 R = mx - mn; // 任一子数组和的绝对值不超过 R

  i64 ans = 0;
  for (int k = 0; k < 62 && (1LL << k) <= R; k++) {
    i64 M = 1LL << (k + 1); // 2^{k+1}
    i64 half = 1LL << k;    // 2^k
    vector<i64> r(n + 1);
    vector<i64> vals;
    vals.reserve(n + 1);
    for (int i = 0; i <= n; i++) {
      r[i] = ((s[i] % M) + M) % M;
      vals.push_back(r[i]);
    }
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    XorFenwick fw((int)vals.size());
    auto pos = [&](i64 v) { // v 在 vals 中的 1-based 下标
      return (int)(lower_bound(vals.begin(), vals.end(), v) - vals.begin()) + 1;
    };
    auto parity = [&](i64 lo, i64 hi) { // 已插入的值落在 [lo, hi] 的个数的奇偶
      if (lo > hi) return 0;
      int l = (int)(lower_bound(vals.begin(), vals.end(), lo) - vals.begin()) + 1;
      int h = (int)(upper_bound(vals.begin(), vals.end(), hi) - vals.begin());
      if (l > h) return 0;
      return fw.pref(h) ^ fw.pref(l - 1);
    };
    int par = 0;
    fw.toggle(pos(r[0]));
    for (int j = 1; j <= n; j++) {
      i64 lo = r[j] + 1, hi = r[j] + half; // 需要 (r[i]-r[j]) mod M ∈ [1, half]
      if (hi < M) {
        par ^= parity(lo, hi);
      } else {
```

```
        par ^= parity(lo, M - 1);
        par ^= parity(0, hi - M);
      }
      fw.toggle(pos(r[j]));
    }
    if (par) ans |= half;
  }
  cout << ans << '\n';
}
```

子序列 +w+

对应题目: U360640(草稿; 题号按题单顺序推断) 灵茶八题 - 子序列 +w+。

- 求: 所有非空子序列的元素和之和 $(\bmod 1e9+7)$。
- 做法: 每个 a[i] 恰出现在 $2^{(n-1)}$ 个非空子序列中, 答案 $= 2^{(n-1)} \cdot \Sigma a[i] \bmod 1e9+7$。
- 复杂度: $O(n + \log n)$。
- 代码由 U360640 残稿补全(补齐读入、求和与输出)。

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'
const i64 MOD = 1e9 + 7;

i64 ksm(i64 a, i64 b) {
  i64 res = 1;
  while (b) {
    if(b & 1LL) res = res * a % MOD;
    a = a * a % MOD;
    b >>= 1;
  }
  return res;
}

int main() {
  int n;
  cin >> n;
  i64 sum = 0;
  for (int i = 0; i < n; i++) {
    i64 tmp;
    cin >> tmp;
    sum = (sum + tmp) % MOD;
  }
  cout << ksm(2, n - 1) * sum % MOD << endl;
}
```

子序列 ^w^

对应题目: U360641(草稿; 题号按题单顺序推断) 灵茶八题 - 子序列 ^w^。

- 求: 所有非空子序列的异或和的异或和。
- 结论: $n = 1 \to a[0]$; $n \geq 2 \to 0$。
- 理由: 对第 k 位, 若没有元素含该位则该位恒为 0; 否则含该位的元素选奇数个的方案数为 $2^{(n-1)}$, $n \geq 2$ 时是偶数 $\to$ 该位贡献 0(空子序列的异或和为 0, 不影响结论)。
- 复杂度: $O(n)$。
- 代码(核心 3 行: 读入、$n == 1$ 判断、输出):

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
int main() {
  int n;
```

```
  cin >> n;
  i64 x = 0;
  for (int i = 0; i < n; i++) cin >> x;
  cout << (n == 1 ? x : 0);
}
```

子序列 ^w+

对应题目: U360642 灵茶八题 - 子序列 ^w+。

- 求: 所有非空子序列的异或和之和 $(\bmod 1e9+7)$。
- 结论: 答案 $= 2^{n-1} \cdot$ (所有数的按位或) $\bmod 1e9+7$: 第 k 位只要按位或为 1, 恰有 2^{n-1} 个子序列在该位为 1。
- 复杂度: $O(n + \log n)$。
- 详解见「非空子序列的异或和的和」小节。

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'
const i64 MOD = 1e9 + 7;

i64 ksm(i64 a, i64 b) {
  i64 res = 1;
  while (b) {
    if(b & 1) res *= a;
    a *= a;
    a %= MOD;
    b >>= 1;
    res %= MOD;
  }
  return res;
}

int main() {
  i64 n;
  cin >> n;
  i64 or_sum = 0;
  for(int i = 0; i < n; i++) {
    i64 tmp;
    cin >> tmp;
    or_sum |= tmp;
  } //不要对按位运算取MOD
  i64 ans = 0;
  for(int i = 0; i < 32; i++) {
    if((or_sum >> i) & 1) ans += ksm(2, n - 1) * ksm(2, i);
    ans %= MOD;
  }
  cout << ans;
}
```

子序列 +w^

对应题目: U360643 灵茶八题 - 子序列 +w^。

- 求: 所有非空子序列的元素和的异或和。
- 做法(子集和奇偶 DP): `dp[j]` = 和为 j 的子集个数的奇偶(`dp[0] = 1`, 空集不影响答案), 每读入 `a[i]` 则倒序 `dp[j] ^= dp[j-a[i]]`。同一个和出现偶数次时在异或里抵消, 所以答案 = 所有 `dp[j] = 1` 的 j 的异或和(即置位下标异或)。
- 注: 2^16 值域为本题条件; 数组按 $(1 << 16) + 1$ 开, 换题需按值域调整。
- 复杂度: $O(n \cdot 2^{16})$。

```cpp
#include <bits/stdc++.h>
using i64 = long long;
using namespace std;
int main() {
  int n;
  cin >> n;
  int MAXN = (1 << 16) + 1;
  vector<int> dp(MAXN + 1, 0);
  dp[0] = 1;
  for(int i = 1; i <= n; i++) {
    int tmp;
    cin >> tmp;
    for(int j = MAXN; j >= 0 ; j--) {
      if(j - tmp < 0) continue;
      else dp[j] = dp[j] ^ dp[j - tmp];
    }
  }
  i64 ans = 0;
  for(int i = 0; i <= MAXN; i++) {
    if(dp[i]) ans ^= i;
  }

  cout << ans;
}
```

贪心

常见贪心方法

所属分类*:* 基础算法 › 贪心 用途*:* 汇总常见贪心套路, 并标注你做过的对应题目 复杂度*:* 依题而定*(*常见 $O(n \log n)$*,* 主要是排序*/*堆的代价*)* 备注*: 2026-10-18* 初稿, 待你核对
本页汇总常见贪心套路, 每节给一句话思想、关键点和对应做过的题(附文件路径, 相对仓库根目录)。反悔贪心单独见「反悔贪心」小节。

邻项交换(微扰)

一句话思想: 只看相邻两项 `x, y`, 交换它们在答案上的差异必须"不让先来的更优"; 解出这个不等式就得到全局排序关键字。

关键点:

- 比较器常形如 `a1 * b2 ≤ a2 * b1`, 乘起来可能爆 int, 统一开 `long long`;
- 排完序往往还要再扫一遍维护前缀(如 P1080 的前缀积, 要配高精度);
- 有时排序只负责定"顺序", 求答案还靠 DP(如 P1417 排序后套 01 背包)。

对应题目:

- P1012 拼数: 比较器 `x + y > y + x`, 拼接结果字典序最大 — `P1012.cpp`
- P1080 国王游戏: 按 `l * r` 升序排大臣, 前缀积除以 r 取最大(高精度) — `P1080.cpp`
- P1248 加工生产调度: Johnson 法则 — $a < b$ 的一组按 a 升序, $a \geq b$ 的一组按 b 降序 — `P1248.cpp`
- P1842 奶牛玩杂技: 按 `w + s` 排序(代码里为降序, 下标 0 视作堆底), 风险 = 上方总重 − s, 求最大风险最小 — `P1842.cpp`
- P1417 烹调方案: 交换比较定顺序(`b_j*c_i` vs `b_i*c_j`), 再套 01 背包 — `P1417.cpp`
- UVA1747: 分段排序贪心($a < b$ 的一段按 a 升序先处理, $a \geq b$ 的一段统一结算), **题意待复核, 存疑** — `UVA1747.cpp`

排序后贪心

一句话思想: 先把元素排好序, 再从头到尾扫, 每个元素只做"当下最优"的决策, 不回头。

关键点:

- 难点全在排序关键字(一般由交换论证或"逆序无益"得到);
- 排序后常配合双指针、堆、DP 使用;
- 带截止时间的计数题, 排序后把截止当背包容量(如 湖工三宝)。

对应题目:

- P1223 排队接水: 按接水时长升序, 平均等待时间最小 — `Luogu/P1223_排队接水.cpp`
- P1803 凌乱的 yyy / 线段覆盖: 按右端点升序, 选尽量多互不重叠的区间 — `Luogu/P1803_凌乱的 yyy_线段覆盖.cpp`
- P1478 陶陶摘苹果(升级版): 够得着的苹果按耗力升序, 能摘就摘 — `P1478.cpp`
- 蚂蚁上树: 对根 1 的每棵子树单独收集叶子深度并排序, 然后按 `cur = max(cur + 1, d)` 贪心分配, 取最大 — `蚂蚁上树.cpp`
- CF1850D Balanced Round: 排序后找相邻差 ≤ k 的最长连续段, 答案 = n − 最长段长 — `CF/Round-1850/1850D.cpp`
- 湖工三宝: 按截止时间排序后做 01 背包, 求最多能选多少门课 — `湖工三宝.cpp`
- CF2065C1: 排序 b 后逐位决策, 每个 `a_i` 取"不小于前一个"的最小可行值(内部二分) — `CF/Round-2065/2065C1.cpp`
- CF2148D: 偶数直接全拿, 奇数排序后减去较小的那一半 — `CF/Round-2148/2148D.cpp`
- CF2148F: 按长度降序排序后, 每轮取字典序最小的序列接进答案 — `2148F.cpp`
- CF2275D Precision Alignment: 按 `a + b + c` 升序, 在预算内贪心扩大可达值, 超了就截断 — `2275D.cpp`
- CF1971G XOUR: 按 `a_i >> 2` 分组, 组内排序后回填 — `1971G.cpp`

双指针

一句话思想: 排序后一左一右夹逼, 能让"最大的尽量配一个最小的"就配成一组。

关键点: 两个指针合计只走 O(n) 步; 每步要么配成一组, 要么丢掉一个端点(丢掉的没法和别人再组)。

对应题目:

- P1094 纪念品分组: 排序后 l/r 双指针, 两件能同箱就配一对 — `P1094.cpp`

排序 + 分数取(部分背包)

一句话思想: 按单位价值(性价比)降序, 尽量整取, 最后一件取一部分。

关键点: 只有"可以切分"的物品能这么做(01 背包不行); 排序关键字 = `v / w`。

对应题目:

- P2240 部分背包: 按 `v / w` 降序依次装, 装不下就按比例取剩余 — `Luogu/P2240_深基 12.例 1_部分背包问题.cpp`

堆贪心

一句话思想: 用堆动态维护当前“最优候选”, 反复取堆顶做局部最优的选择/合并。

关键点:

- `priority_queue` 默认大根堆, 取最小要带 `greater<T>`;
- 合并类问题反复取两个最小的; 多路候选可以多堆 + 惰性删除。

对应题目:

- P1090 合并果子: 每次合并两堆最小的(Huffman 思想)。仓库里的两份实现: 一份用“降序排序 + 手写模拟小根堆”(每次取尾部两个最小, 求和回填后再对尾部做三次交换修正), 另一份用“分层两两合并”; 另外根目录 `P1090.cpp` 是空文件 — `Luogu/P1090_NOIP_2004_提高组_合并果子.cpp`(内含一份被注释掉的旧写法)、`Luogu/P1090_NOIP_2004 提高组_合并果子.cpp`
- P4447 分组: 排序后用小根堆接龙 — `mp[x]` 存“以 x 结尾的每个组的长度”, 新来的数接在最短的那组后面 — `P4447.cpp`

栈贪心

一句话思想: 用栈维护候选答案序列, 新元素比栈顶更优时弹栈(弹一次消耗一次删除次数)。

关键点: 别忘了处理“删不完”: 剩余次数从末尾继续删; 最后还要去前导零。

对应题目:

- P1106 删数问题: 使剩下的数最小 — 栈里比当前位大的都弹掉, 剩余次数从末尾删 — `P1106.cpp`

扫描 + 差分

一句话思想: 从左往右扫一遍, 每步只处理当前位置与相邻位置的约束; 区间式的操作可以压成差分端点。

关键点:

- 相邻约束优先改右边那个(能顺带管到下一对);
- 差分把“一段一起加/减”变成两个端点变化。

对应题目:

- P3817 小 A 的糖果: 相邻两堆和 ≤ x, 超了先削右边那堆(不够再削左边) — `P3817.cpp`
- P5019 铺设道路: 答案 $= \Sigma \max(0, a[i] - a[i-1])$(差分, 下降段不用管) — `P5019.cpp`
- CF1366B Shuffle: 可达区间从 {x} 出发, 能接上就左右扩张 — `1366B.cpp`
- CF1873D 1D Eraser: 从左往右, 遇到未覆盖的 B 就把接下来 k 个全变 W(贪心覆盖) — `CF/Round-1873/1873D.cpp`

按位贪心

一句话思想: 从高位往低位逐位定, 高位能不放 1 就不放; 用“低位全 1 最多能表示多大”来判定。

关键点: 判定要用向上取整防溢出: `need = (rem + n - 1) / n; need <= val - 1` 成立就可以不放这一位。

对应题目:

- 105143B: 把一个数组的和 sum 分给 n 个数, 使按位或最小 — `105143B.cpp`

反悔贪心

一句话思想: 先贪心选上, 之后发现不合法/不优时用堆“退货”或换成更优决策; 关键 = 可反悔量。

关键点: 详见「反悔贪心」小节 — 思想、适用特征、4 类套路、12 题例题和模板骨架。

对应题目(12 题): P2949、CF865D、CF1526C2、P4053、P1484、CF730I、P3545、CF1974G、P16789、AT_agc018_c、3D、U476744。

反悔贪心

所属分类: 基础算法 › 贪心 用途: 反悔贪心思想、分组例题与骨架 复杂度: 一般为 $O(n \log n)$(每步堆操作) 备注: *2026-10-18* 初稿, 待你核对

本页整理反悔贪心的思想、适用特征、四类套路、已做的 12 道例题和一个模板骨架(文件路径相对仓库根目录)。

思想

反悔贪心 = 普通贪心 + "退货"。先按贪心把决策选上, 当后面的选择让当前方案越界(超截止/超预算/超容量)或不再最优时, 用堆把之前选过的一项撤销、或替换成更优决策, 让贪心具备全局视野。

核心是找到**可反悔量**: 撤销/替换对答案的增量要能算出来、能维护:

- 退货型: 已选项都进堆, 越界时退掉堆顶"最差"的一项 — 见 P2949、P4053;
- 换手型: 把"换成另一个选择"的净收益放进堆 — 见 CF865D(差价)、CF730I(转组 `b - a`);
- 合并型: 把相邻项并成一个"组合点", 再选它就等价于反悔上一手、改选两边 — 见 P1484;
- 翻转型: 先全按一种填法填, 不合法时挑代价最小的位置翻过来 — 见 3D。

适用特征

- 决策按时间/顺序逐个到来, 后面的信息可以推翻前面的选择;
- 约束是数量、预算、容量、合法性这类"全局约束", 局部贪心容易越界;
- 需要"前 i 个里最多选 i 个""最多花 x 元"这类信息, 且约束单调;
- 单步反悔的代价可以算出来(如 P1484 的 `nv = val[L] + val[R] - val[id]`、CF865D 的差价 `p - pq.top()`)。

常见套路

①按截止/预算排序 + 堆扔最差

逐项先选上; 一旦越界(超过截止时间、预算、库存), 就退掉"最差"的一项, 保住数量/收益最多。

对应: P2949、P4053、CF1526C2、P3545、CF1974G、P16789。

②交换/替换反悔(换手)

把"换成另一个选择"的净收益放进堆; 每次要么直接加一个新人, 要么让老人转组、再补个新人, 两者取优。

对应: CF865D、CF730I、AT_agc018_c、U476744。

③合并反悔(P1484 种树式)

选中 id 后, 把 id 和左右邻居合并成一个新点, 新值 `nv = val[L] + val[R] - val[id]`; 以后再选到这个点, 就等价于"退掉 id、改选 L 和 R"。相邻关系用链表维护。

对应: P1484。

④翻转反悔(括号型)

先全部按同一种方式填; 扫描中只要不合法, 就挑代价最小的位置翻成另一种。

对应: 3D(问号先全填 `')'`, 前缀失衡时把 `a - b` 最小的问号翻成 `'('`)。

已做例题

1. P2949 [USACO09OPEN] Work Scheduling G

- 题意: N 个任务, 每个有截止时间 D 和收益 P, 每天做一件, 求最大总收益。
- 反悔点: 先全部接下; 已选数量超过当前截止 D 时, 退掉收益最小的那个。

```
sort(task.begin(), task.end());        // 按截止时间 D 排序
auto[D, P] = task[i];                  // D = 截止, P = 收益
ans += P; pq.push(P);                  // 先贪心选上
if(pq.size() > D) {                    // 已选数超过截止 D
  ans -= pq.top(); pq.pop();          // 退掉收益最小的
}
```

- 文件: `P2949.cpp`

2. P4053 [JSOI2007] 建筑抢修

- 题意: N 栋建筑, 修第 i 栋耗时 T1、必须在 T2 前修完, 求最多能修几栋。
- 反悔点: 按截止时间排序; 修不过来就退掉已修里最耗时的那栋(保数量)。

```
sort(build.begin(), build.end(), [](auto& x, auto& y){ return x.second < y.second; });
T += T1; pq.push(T1);                  // 先修上
if(T > T2) {                           // 修不完
  T -= pq.top(); pq.pop();            // 退掉最耗时的一栋
}
```

- 文件: `P4053.cpp`

3. CF1526C2 Potions (Hard Version)

- 题意: n 瓶药水, 初始生命 0, 喝第 i 瓶生命 += a_i(a_i 可负), 任何时候生命必须 ≥ 0, 求最多喝几瓶。
- 反悔点: 生命为负时, 吐掉堆里“毒性最大”(即最小)的一瓶 — 改成不喝它。

```
pq.push(a);
hp += a;
if(hp < 0) {                          // 生命为负
  hp -= pq.top(); pq.pop();          // 吐掉毒性最大(最小)的一瓶
}
```

- 文件: CF1526C2.cpp

4. P3545 [POI 2012] HUR-Warehouse Store

- 题意: 共 n 天, 第 i 天进货 a_i, 同时有要买 b_i 的客户; 库存不能为负, 求最多能满足几位客户(输出数量与客户编号)。
- 反悔点: 每天先接客户; 库存为负时退掉堆里需求最大的一位。

```
sum += a[i];                          // 进货 a_i
custom.push({b[i], i});               // 先接客户(需求 b_i)
sum -= b[i];
if(sum < 0) {                         // 库存不够: 退掉需求最大的客户
  sum += custom.top().first; custom.pop();
}
```

- 文件: P3545.cpp

5. CF1974G Money Buys Less Happiness Now

- 题意: 共 m 个月, 每月发工资 x; 第 i 月可花 c_i 买一份快乐(或不买), 余额不能为负, 求最多买几份。
- 反悔点: 有钱就买; 余额为负时退掉已买里最贵的一份。

```
money -= c;                           // 先买
pq.push(c);
if(money < 0) {                       // 余额为负
  money += pq.top(); pq.pop();       // 退掉最贵的一份
}
money += x;                           // 月末发工资
```

- 文件: CF1974G.cpp

6. P16789(题名待复核)

- 题意(按代码推断, 待复核): N 个任务, 每个给出 t、w、is_key; 按 t 排序后保证“已选数不超过当前 t”, 超了就淘汰; 输出保留下来的关键任务数与总权重。关键任务只有在没有非关键任务可淘汰时才被淘汰, 同类里也先淘汰 w 最小的。
- 反悔点: 比较器定义“谁先被扔”: 非关键任务优先被扔; 同类里 w 小的先扔(保住关键任务和高权重)。

```
bool operator()(const task &x, const task &y) {
  if(x.is_key == y.is_key) return x.w > y.w;   // 同类: w 小的先被扔
  return x.is_key > y.is_key;                  // 非关键任务先被扔
}
```

- 文件: P16789.cpp

7. CF865D Buy Low Sell High

- 题意: N 天股价, 每天可买一股或卖出一股(至多持一股), 求最大利润。
- 反悔点: 堆顶价 < 当前价就“卖掉”它; 同时把当前价再 push 一次 — 若将来更贵, 相当于反悔这笔卖出、改成当天买入。

```
pq.push(p);                           // 当前价入堆(可当买点)
if (pq.top() < p) {                   // 堆顶更低 → 卖掉它
  ans += p - pq.top(); pq.pop();
  pq.push(p);                        // 当前价再入堆: 这笔卖出可反悔
}
```

- 文件: CF865D.cpp

8. CF730I Olympiad in Programming and Sports

- 题意: n 个人, 挑 p 个进编程队、s 个进运动队, 第 i 人在编程队贡献 a_i、运动队贡献 b_i, 求最大总贡献。
- 反悔点: 每加一个运动队名额, 取两种方案更优者 —①直接挑剩余里 b 最大的人;②让已进编程队的某人转去运动队(b - a), 再补一个 a 最大的人进编程队。

```
if(!p1.empty() && !p3.empty()) n1 = p1.top().first + p3.top().first;   // 方案②: a→s 再补新 a
if(!p2.empty()) n2 = p2.top().first;                                   // 方案①: 直接选 b 最大
if(n1 > n2) {
  ans += n1;
  type[p1.top().second] = 1;
  type[p3.top().second] = 2;
```

- 文件: CF730I.cpp

9. AT_agc018_c Coins

- 题意: x+y+z 个人, 每人拿金/银/铜币之一(分别为 x、y、z 枚); 第 i 人拿金贡献 a_i、银 b_i、铜 c_i, 求最大总贡献。
- 反悔点: 先全按铜币计, 再上 x 枚金币(取 a - c 最大的); 之后每上一个银币, 取"直接把铜换银(b - c)"与"把一个金币改银 + 补一个新金币((a - c) + (b - a))"的较优者。

```
if(val1 > val2 + val3) {                    // 直接把 c 改 b 更优
  ctob.pop(); ans += val1; type[id1] = 2;
} else if(val1 <= val2 + val3){             // 换手: c→a 再 a→b, 补个新 a
  ctoa.pop(); atob.pop(); ans += (val2 + val3);
  atob.push({b[id2].first - a[id2].first, id2});
  type[id2] = 1; type[id3] = 2; }
```

- 文件: AT_agc018_c.cpp

10. U476744 加油站(题名待复核)

- 题意: 从 0 开车到 D, 油箱容量 G、初始油量 B; 沿途 n 个加油站(位置 x、油价 y), 求到终点的最小油费(到不了输出 −1)。
- 反悔点: 候选油池 — 行驶消耗时取最便宜的候选油; 进站先把"最多 G 单位、当前油价"加入候选, 若候选总量超容量就丢掉最贵的(贵油被后面的便宜油反悔掉)。

```
while(t > G) {                          // 候选油总量超过油箱容量 G
  ll d = pq_mx.top();                  // 丢掉最贵的候选油
  ll k = mp[d]; ll re = min(k, t - G);
  t -= re; mp[d] -= re;
  if(mp[d] == 0) pq_mx.pop();
}
```

- 文件: U476744.cpp

11. P1484 种树

- 题意: 路上一排 n 个坑, 至多选 k 个互不相邻的坑种树, 使总收益最大。
- 反悔点: 选 id 后把它与左右合并成一个新候选 nv = val[L] + val[R] - val[id]; 之后再选中这个合并点, 就等于反悔(退掉 id、改选 L 和 R)。

```
i64 nv = val[L] + val[R] - val[id];    // 合并点: 选它 = 退掉 id 改选 L, R
val[id] = nv;
used[L] = 1, used[R] = 1;
nxt[L1] = id; pre[R1] = id;            // 链表接上合并点
pq.push({nv, id});                     // 新候选继续参与竞争
```

- 文件: P1484.cpp

12. CF 3D Least Cost Bracket Sequence

- 题意: 括号串里含 ?; 第 j 个 ? 改成 '(' 花费 a_j、改成 ')' 花费 b_j; 求补成合法括号串的最小花费。
- 反悔点: ? 先一律填 ')'(同时把 a - b 入堆); 扫描到前缀失衡时, 把堆里最便宜(a - b 最小)的 ? 翻成 '('。

```
if(blc < 0) {                          // 前缀失衡
  while (!pq.empty() && blc < 0) {
    auto [val, pos] = pq.top(); pq.pop();
```

```
    ans += val; blc += 2; s1[pos] = '(';   // 翻成 '('
  }
}
```

- 文件: `3D.cpp`

模板骨架

退货型骨架: `priority_queue` 维护已选项, 越界就退堆顶“最差”的一项。

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

// 反悔贪心骨架(退货型): 先贪心选上, 越界就把堆顶"最差"的一项退掉。
// 变体 1: 带截止时间的题 → 先按截止排序, 判断条件换成 "已选数量 > 截止"(见 P2949 / P4053)。
// 变体 2: 要最大收益而不是最大数量 → 用小根堆退"收益最小"的一项(见 CF1526C2)。
// 变体 3: 换手型 → 退掉堆顶时再 push 一个新值进去(见 CF865D / CF730I)。
int main() {
  int n;
  i64 limit;
  cin >> n >> limit;      // n 项, 总代价上限 limit
  vector<i64> cost(n);
  for (auto &c : cost) cin >> c;

  priority_queue<i64> pq; // 已选项; 大根堆 → 堆顶 = 已选里代价最大(最差)的
  i64 used = 0;           // 已选代价之和
  for (i64 c : cost) {
    pq.push(c); used += c;          // ① 先贪心选上
    if (used > limit) {             // ② 越界就反悔
      used -= pq.top(); pq.pop(); //    退掉堆顶
    }
  }
  cout << pq.size() << '\n';          // ③ 堆里剩下的就是答案
  return 0;
}
```

待做

- CF335F Buy One Get One Free(难度 *3000): 反悔贪心进阶难题; 目前 `CF335F.cpp` 只有空 main, 待补。
- P1792 [国家集训队] 种树: 环形版本的种树(与 P1484 同型, 链表 + 堆反悔); 目前 `P1792.cpp` 是空文件。

高精度

所属分类: 基础算法 › 高精度 复杂度: 加/减/高×低精/高÷低精 均为 $O(位数)$; 高×高精 $O(位数^2)$ 备注: 2026-10-18 初稿, 待你核对

适用问题

- 运算数超出 long long 范围时的整数四则运算(大整数运算)
- 典型场景: 阶乘、递推大数、大数加减乘除
- 对应题目: P1601 A+B Problem(高精)、P2142 高精度减法

低位在前表示

- 用 `vector<int>` 存十进制数: `a[0]` 是个位, 下标越大位越高; 每个元素是一位数字($0 \sim 9$)
- 约定无前导零(最高位不为 0, 数 0 存为 `{0}`); 以下片段均省略头文件(默认已 `#include <bits/stdc++.h>` 且 `using namespace std;`)

```
vector<int> read() {
    string s;
    cin >> s;
    vector<int> a;
    for (int i = (int)s.size() - 1; i >= 0; i--)
        a.push_back(s[i] - '0');                    // 低位在前
    while (a.size() > 1 && a.back() == 0) a.pop_back();  // 去前导零
    return a;
}

void print(const vector<int> &a) {
    for (int i = (int)a.size() - 1; i >= 0; i--)
        cout << a[i];
    cout << '\n';
}
```

加

逐位相加、进位。

```
vector<int> add(const vector<int> &a, const vector<int> &b) {
    vector<int> c;
    int carry = 0;
    for (size_t i = 0; i < max(a.size(), b.size()) || carry; i++) {
        int sum = carry;
        if (i < a.size()) sum += a[i];
        if (i < b.size()) sum += b[i];
        c.push_back(sum % 10);
        carry = sum / 10;
    }
    return c;
}
```

减(大减小)

要求 `a >= b`(否则结果不对; 通用减法先按「长度不同则长的更大、长度相同从最高位逐位比」判断大小, 负号自行处理)。

```
vector<int> sub(const vector<int> &a, const vector<int> &b) {  // 要求 a >= b
    vector<int> c = a;
    int borrow = 0;
    for (size_t i = 0; i < c.size(); i++) {
        int cur = c[i] - borrow - (i < b.size() ? b[i] : 0);
        if (cur < 0) { cur += 10; borrow = 1; }
        else borrow = 0;
        c[i] = cur;
    }
```

```
    while (c.size() > 1 && c.back() == 0) c.pop_back();  // 去前导零
    return c;
}
```

高×低精
低精数 b 直接乘进每一位, 中间量用 long long, 防止超 int。

```
vector<int> mul(const vector<int> &a, int b) {
    vector<int> c;
    long long carry = 0;
    for (size_t i = 0; i < a.size() || carry; i++) {
        long long cur = carry + (i < a.size() ? 1LL * a[i] * b : 0);
        c.push_back(cur % 10);
        carry = cur / 10;
    }
    while (c.size() > 1 && c.back() == 0) c.pop_back();  // 去前导零(b=0 时)
    return c;
}
```

高×高精
两两相乘落位到 i + j, 再就地进位。

```
vector<int> mul(const vector<int> &a, const vector<int> &b) {
    vector<int> c(a.size() + b.size(), 0);
    for (size_t i = 0; i < a.size(); i++)
        for (size_t j = 0; j < b.size(); j++) {
            c[i + j] += a[i] * b[j];
            c[i + j + 1] += c[i + j] / 10;       // 进位
            c[i + j] %= 10;
        }
    while (c.size() > 1 && c.back() == 0) c.pop_back();  // 去前导零
    return c;
}
```

高÷低精(商+余)
从最高位往下带余除法, 商从函数返回、余数由引用参数 rem 带回(要求低精数非 0)。

```
vector<int> div(const vector<int> &a, int b, int &rem) {  // 返回商, rem 带回余数
    vector<int> c(a.size());
    long long cur = 0;
    for (int i = (int)a.size() - 1; i >= 0; i--) {
        cur = cur * 10 + a[i];
        c[i] = (int)(cur / b);
        cur %= b;
    }
    rem = (int)cur;
    while (c.size() > 1 && c.back() == 0) c.pop_back();  // 去前导零
    return c;
}
```

三、数据结构

单点修改_区间查询

树状数组

所属分类: 数据结构 › 单点修改_区间查询 复杂度: 修改 $O(log\ n)$ / 查询 $O(log\ n)$ / 初始化 $O(n\ log\ n)$(可优化到 $O(n)$)

适用问题

- 单点修改 + 区间和查询
- 一切可差分的信息(总和/总乘等)都能用树状数组维护
- 区间修改 + 单点查询(树状数组维护差分数组)

在基础的算法中 如果我们想要查找一段区间和 我们常用前缀和 pre[n] - pre[m] 这样的时间查询复杂度为 O(1)
但是当需要修改一点的情况下 全部都得重新计算 在长数组中 这是不划算的
那么我们就需要一种数据结构 需要满足 1.查询时间不长 2.修改快
树状数组就刚好满足这种需要 其查询修改的时间复杂度均为 logn
想要完全理解树状数组 就要从以下入手
众所周知 一个数字 如 13(1101) 可以被拆成 8 + 4 + 1 的形式 那么对于 一段区间长为 13 运用分块的思想 假设我们知道前 8 中间 4 最后 1 的和 就可以在 logn 的时间内得到前 13 的前缀和
但是问题来了 对于一个真实的数组 如总长度 13 的数组 我们需要随时能够查询任意的前缀和 就必须存储每个数字需要的区间 如 11(1011) 需要 8 + 2 + 1 7(0111) 4 + 2 + 1
那么怎么样才能存储每个前缀和需要的长度区间呢?
我们首先发现 对于每个前缀和 如同 11(1101) 我们只需要 lowbit 拆解二进制 成 长度为 1 对应数组下标 1101 长度为 100 对应下标 1100 长度为 1000 对应下标 1000
这样子能保证每个数组下标间 只需要-lowbit 就能找到天然接壤的位置
同时对于数组从 1 开始到 n 每次 不断添加新数下标 维护新范围 能够保证复用性和倍增 那么设计好了这样的数据结构 如何使用他呢?
如果需要修改的话 就只需要取解决会用到当前下标的

用法

一切可差分的信息 都能用树状数组来维护 例如 总和 总乘
树状数组能维护

树状数组的思想

在设计之初 数组一定要从 1 开始 树状数组的下标代表的是该下标维护的原数组右边界 下标的 lowbit 是维护的数组的长度 通过这两个信息就能知道该下标的所有信息
在查询时候 通过两个前缀和就能得到需要的差分信息 反之 在修改的时候 一直加 lowbit 就是一直在扩展自己的范围且不会遗漏

复杂度

时间 查询 修改均为 logn 初始化 一般为 nlogn 如果有所有信息 通过只修改自己的上一个父数组可以优化到 n

函数

树状数组一般需要四个函数
1. `lowbit` 函数 获取 lowbit 值
2. `update` 函数 用来更新树状数组
3. `init` 函数 初始化树状数组
4. `query` 函数 用来查找

(以下板子均为和 查询也是区间和)

完整模板

```
// 1.lowbit函数如下
i64 lowbit(int x) {
  return (x & -x);
}

// 2.update函数如下
void update(int x, int k) {
```

```
  for(int i = x; i <= n; i += lowbit(i)) {
    c[i] += k; //c[i]为树状数组
  }
}

// 3.init函数如下 logn版本
  for(int i = 1; i <= n; i++) {
    cin >> arr[i];
    update(i, arr[i]);
  }

// 3.init函数如下 n版本
// 1. 预装填：把 tree 变成 a 的副本（下标从 1 开始）
for (int i = 1; i <= n; i++) {
    tree[i] = a[i];
}

// 2. 线性合并
for (int i = 1; i <= n; i++) {
    int parent = i + lowbit(i);
    if (parent <= n) {
        // 将当前节点的【最终总和】直接累加给【直接父亲】
        // 这样 parent 遍历到的时候，它就已经拿到了所有下属的值
        tree[parent] += tree[i];
    }
}

// 4.query函数如下
int query(int a) {
  int sum = 0;
  for(int i = a; i >= 1; i -= lowbit(i)) {
    sum += c[i];
  }
  return sum;
}
```

区间修改_区间查询

线段树

所属分类: 数据结构 › 区间修改_区间查询 复杂度: 修改/查询 $O(log\ n)$
适用问题
- 区间修改 + 区间查询(父范围信息能从子范围合并: 最大值/最小值/和/乘积等)

维护的信息类型
父范围的信息能从子范围得到 满足的如区间的 最大值 最小值 和 乘积等 不满足的如 范围上出现次数最多的数字
功能
范围查询 修改 时间复杂度 $\log n$
如何判断范围修改能否在 logn 时间内完成
就看在修改后能否在 $O(1)$时间内得到修改后的信息
线段树的思想
首先 线段树是一个维护信息的完全二叉树 如图
线段树的构建过程是一个递归的过程 在这个过程中 存在函数 $f(l, r, i)$ 依次代表 l, r 维护的区间的长度 i 存储这个区间信息的树的数组下标 递归 $f(l, m, 2\ \mathbf{i})\ \mathbf{f(m + 1, r, 2}\ i + 1)$就能维护线段树的左右区间并且以此放在左孩子和右孩子上
空间需要一般为 $4n$ 需要极限压缩可以为不大于 n 的最大 2 的幂次（即 $2^{\lfloor \log_2 n \rfloor}$）
代码结构
线段树的代码一般包含三个部分
1. `bulid` 函数 建立一开始的线段树 用递归来完成
2. `query` 函数 查询线段树上的信息 用递归来完成 每次查询三种可能 要查询的元素在不在当前区间上，在则认为命中了 直接返回 如果部分在区间上 分为在左边 在右边的情况 依次判断 继续递归
3. `update` 函数 更新区间的信息 其中还包含 up 和 down 数组

完整模板

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// ============================================================
// 1. 业务逻辑区：打比赛时只需修改下面这三个部分
// ============================================================

struct Tag { //懒标记
    long long add = 0; // 例子：区间加法

    Tag(long long v = 0) : add(v) {}

    // 动作叠加：旧标记如何吸收新标记 t
    void apply(const Tag &t) {
        add += t.add;
    }
};

struct Info { //信息
    long long sum = 0;
    int len = 0; // 维护区间长度，方便计算区间加

    Info() {}
    Info(long long v, int l) : sum(v), len(l) {}
```

```
    // 动作作用于结果：标记 t 如何改变当前信息
    void apply(const Tag &t) {
        sum += (long long)len * t.add;
    }
};

// 合并：两个子区间如何合并成父区间
Info operator+(const Info &a, const Info &b) {
    return {a.sum + b.sum, a.len + b.len};
}

// ============================================================
// 2. 框架区：通用的带懒标记线段树（左闭右开）
// ============================================================

template<class Info, class Tag> //模版 传入信息和懒标记
struct LazySegmentTree {
    int n;
    vector<Info> info;
    vector<Tag> tag; //初始化变量和数组

    LazySegmentTree(int n) : n(n) {
        info.assign(4 << __lg(n), Info());
        tag.assign(4 << __lg(n), Tag());
    }//初始化懒标记数组和信息数组长度

    LazySegmentTree(vector<Info> init_v) : n(init_v.size() - 1) {
        info.assign(4 << __lg(n), Info());
        tag.assign(4 << __lg(n), Tag());

        auto build = [&](auto &self, int p, int l, int r) -> void {
            if (r - l == 1) {
                info[p] = init_v[l];
                return;
            }
            int mid = (l + r) >> 1;
            self(self, p << 1, l, mid);
            self(self, p << 1 | 1, mid, r);
            pull(p);
        };
        build(build, 1, 1, n + 1);
    } //构造函数 传入最初的数组

    void pull(int p) {
        info[p] = info[p << 1] + info[p << 1 | 1];
    } //向上传递信息

    void apply(int p, const Tag &v) {
        info[p].apply(v);
        tag[p].apply(v);

    } //修改当前点的信息和懒标记信息

    void push(int p) {
```

```
        apply(p << 1, tag[p]);
        apply(p << 1 | 1, tag[p]);
        tag[p] = Tag();
    } // 向下传递信息

    void modify(int p, int l, int r, int x, const Info &v) {
        if (r - l == 1) {
            info[p] = v;
            return;
        }
        int mid = (l + r) >> 1;
        push(p);
        if (x < mid) modify(p << 1, l, mid, x, v);
        else modify(p << 1 | 1, mid, r, x, v);
        pull(p);
    }

    void rangeApply(int p, int l, int r, int x, int y, const Tag &v) {
        if (l >= y || r <= x) return;
        if (l >= x && r <= y) {
            apply(p, v);
            return;
        }
        int mid = (l + r) >> 1;
        push(p);
        rangeApply(p << 1, l, mid, x, y, v);
        rangeApply(p << 1 | 1, mid, r, x, y, v);
        pull(p);
    }

    Info rangeQuery(int p, int l, int r, int x, int y) {
        if (l >= y || r <= x) return Info();
        if (l >= x && r <= y) return info[p];
        int mid = (l + r) >> 1;
        push(p);
        // 性能优化：如果查询范围只在某一侧，不进行跨侧合并
        if (y <= mid) return rangeQuery(p << 1, l, mid, x, y);
        if (x >= mid) return rangeQuery(p << 1 | 1, mid, r, x, y);
        return rangeQuery(p << 1, l, mid, x, y) + rangeQuery(p << 1 | 1, mid, r, x, y);
    }

    // 外部调用接口（1-based）
    void modify(int p, const Info &v) { modify(1, 1, n + 1, p, v); }
    void rangeApply(int l, int r, const Tag &v) { rangeApply(1, 1, n + 1, l, r, v); }
    Info rangeQuery(int l, int r) { return rangeQuery(1, 1, n + 1, l, r); }
};

// ============================================================
// 使用示例
// ============================================================

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n = 5;
```

```
    // 准备初始数据，假设数组为 [1, 2, 3, 4, 5]
    vector<Info> a(n + 1);
    for (int i = 1; i <= n; i++) {
        a[i] = Info(i, 1); // 初始值 i，长度 1
    }

    LazySegmentTree<Info, Tag> st(a);

    // 区间 [1, 3] 加上 10 -> 注意左闭右开 rangeApply(1, 4, ...)
    st.rangeApply(1, 4, Tag(10));

    // 查询区间 [2, 4] 的和 -> rangeQuery(2, 5)
    cout << st.rangeQuery(2, 5).sum << endl;

    return 0;
}
```

静态区间 RMQ

ST 表(倍增)

所属分类: 数据结构 › 静态区间 *RMQ* 复杂度: 预处理 *O(n log n)* / 查询 *O(1)*
适用问题
- 静态可重复贡献 RMQ: MAX、MIN、GCD、LCM、按位或、按位与

核心思想
ST 表运用了倍增的思想: 先维护每个位置从 2^0 到 2^n(小于最后位置)长度中的 RMQ。 例子: 数组 1 2 3 4 5 6 7 8, 查询 2-6 只需 2-4 和 4-6 合并。

```
#include <bits/stdc++.h>
using i64 = long long;
using namespace std;
i64 n;
vector<i64> logN;
vector<i64> arr;
vector<vector<i64>> st; //修复: 原为 st(100, vector<i64>(100)) 硬编码

void pre() {
  logN[2] = 1;
  for(i64 i = 3; i <= n; i++) { //修复: 原为 i < n, logN[n] 未初始化
    logN[i] = logN[i / 2] + 1;
  }
}

void init() {
  st.assign(n, vector<i64>(logN[n] + 1)); //修复: 动态分配
  for(i64 i = 0; i < n; i++) st[i][0] = arr[i];
  for(i64 i = 1; i <= logN[n]; i++) {
    for(i64 j = 0; j + (1 << i) <= n; j++) {
      st[j][i] = max(st[j][i - 1], st[j + (1 << (i - 1))][i - 1]);
    }
  }
}

i64 query(i64 l, i64 r) { //0-based, l <= r
  i64 k = logN[r - l + 1]; //修复: 原为 l - r + 1, 下标为负越界
  return max(st[l][k], st[r - (1 << k) + 1][k]);
}

int main() { //修复: 原 main 只读入 n 未做任何事, 补全
  cin >> n;
  arr.resize(n + 1);
  logN.resize(n + 1);
  for(i64 i = 0; i < n; i++) cin >> arr[i];
  pre();
  init();
  i64 l, r;
  cin >> l >> r;
  cout << query(l, r) << endl;
}
```

注意
- 静态数据结构, 不支持修改;需要修改用线段树

单调性问题

单调队列

所属分类：数据结构 › 单调性问题 复杂度：$O(n)$
适用问题
- 滑动窗口最值(最大值/最小值)

用法
单调队列可用以解决滑动窗口最值问题
实现步骤
单调队列的本质可以分为 6 步
1. 从左到右遍历整个队列
2. 每次遍历到新元素的时候分别考虑
3. 队头是否滑出范围 是则删除到不是为止
4. 队尾是否比新来的更小或者更大 是则删除到不是为止
5. 入队新元素
6. 此时队头就是窗口的最小值

完整模板

```
deque<int> dq;
for (int i = 0; i < n; i++) {
    if (!dq.empty() && dq.front() < i - k + 1) dq.pop_front();
    while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
    dq.push_back(i);
    if (i >= k - 1) cout << a[dq.front()] << " ";
}
```

单调栈

所属分类: 数据结构 › 单调性问题 复杂度: $O(n)$

适用问题

- 下一个更大/更小元素
- 找左右边界、柱状图最大矩形、接雨水、子数组贡献、嵌套括号

用法

找左右边界、下一个大小元素、矩形 / 接雨水、子数组贡献、嵌套括号

实现步骤

单调栈的本质可以分为 7 步

1. 从左到右遍历整个队列
2. 每次遍历到新元素的时候考虑
3. 新元素是不是比当前的栈头更大或者更小或其他
4. 如新元素是 则弹出头到不是为止
5. 入栈新元素
6. 此时栈头就是窗口的最小值
7. 答案数组记录下此时的位置即可

完整模板

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
int main() {
    i64 n;
    cin >> n;
    vector<i64> arr(n);
    vector<i64> ans(n, 0);
    for(auto &i:arr) {
        cin >> i;
    }
    stack<i64> S;
    for(i64 i = 0; i < n; i++) {
        while (!S.empty() && arr[S.top()] < arr[i]) {
            ans[S.top()] = i + 1;
            S.pop();
        }
        S.push(i);
    }
    for(auto &i:ans) {
        cout << i << ' ';
    }
}
```

四、图论

最短路

单源最短路
所有边权都是正数

堆优化 Dijkstra

所属分类: 图论 › 最短路 › 单源最短路 › 所有边权都是正数 复杂度: $O(m\ log\ n)$, 适合稀疏图
适用问题

- 非负权图的单源最短路
- 多源最短路(超级源点技巧, 见例题)

完整模板
用法
求解非负权图单源最短路，时间复杂度 $\mathcal{O}(m \log n)$。
实现步骤
1. `dist[s] = 0`，其余为 INF
2. 小顶堆存 (距离，结点)，起点入堆
3. 每次取出堆顶，跳过已确定的点，否则标记并松弛邻接点
4. 堆空即得起点到各点的最短路

完整模板

```cpp
#include <bits/stdc++.h>
using namespace std;
int n, m, s;
vector<bool> vis;
int main() {
  cin >> n >> m >> s;
  vis.resize(n + 1, 0);
  vector<vector<pair<int, int>>> E(n + 1);
  for(int i = 1; i <= m; i++) {
    int u, v, w;
    cin >> u >> v >> w;
    E[u].push_back({w, v});
  }
  priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
  vector<int> dis(n + 1, 1e9 + 5);
  dis[s] = 0;
  pq.push({0,s});

  while (!pq.empty()) {
    auto [d, u] = pq.top();
    pq.pop();
    if(vis[u]) continue;
    vis[u] = 1;
    for(auto [w, v]:E[u]) {
      if(dis[v] > dis[u] + w) {
        dis[v] = dis[u] + w;
        pq.push({dis[v], v});
      }
    }
  }
  for(int i = 1; i <= n; i++) {
    cout << dis[i] << ' ';
  }
```

```
}
```

例题(超级源点技巧)

值得注意的是, 可以将点权转换为一个点, 每个点都指向这个点 0, 路径权就为点权。(作者原话)

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
const i64 INF = 1e18;
#define endl '\n'
int main() {
  i64 n, m;
  cin >> n >> m;
  vector<vector<pair<i64, i64>>> Node(n + 1);
  for(int i = 1; i <= m; i++) {
    i64 u, v, w;
    cin >> u >> v >> w;
    Node[u].push_back({2 * w, v});
    Node[v].push_back({2 * w, u});
  }
  for(int i = 1; i <= n; i++) {
    i64 w; cin >> w;
    Node[0].push_back({w, i});
    Node[i].push_back({w, 0});
  }

  priority_queue<pair<i64, i64>, vector<pair<i64, i64>>, greater<>> pq;
  pq.push({0, 0});
  vector<bool> vis(n + 1, 0);
  vector<i64> dis(n + 1, INF);
  dis[0] = 0;
  while (!pq.empty()) {
    auto[w, cur] = pq.top();
    pq.pop(); //修复: 原代码漏此行 -> 死循环
    if(vis[cur]) {continue;}
    vis[cur] = 1;
    for(auto [len, u]:Node[cur]) {
      if(dis[cur] + len < dis[u]) dis[u] = dis[cur] + len; //修复: 原用旧距离w
      pq.push({dis[u], u});
    }
  }
  for(int i = 1; i <= n; i++) {
    cout << dis[i] << ' ';
  }
  return 0;
}
```

注意

- 稠密图用朴素 Dijkstra O(n^2);有负权边不能用 Dijkstra

SPFA

所属分类: 图论 › 最短路 › 单源最短路 › 存在负权边 复杂度: 平均 $O(kE)$, 最坏 $O(VE)$

适用问题

- 含负权边的单源最短路
- 判断负环(单源版/全源版/边数法)
- 最短路径回溯

SPFA 是 Bellman-Ford 的进阶版: 对于一条边, 如果它的前驱节点根本没更新, 那这个节点连接的所有结点根本就没必要再看, 这样可以节省时间复杂度。
具体上, 维护一个队列: 对于更新了的结点, 入队它的相邻结点即可。
但竞赛中经常会被卡, 被卡特征(满足越多越危险):

- 图是网格状
- 边权差异大且方向性强
- 存在长链 + 大量旁支
- V 很大且 E 接近 O(V)

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'
const i64 INF = (1LL << 31) - 1; // 题目要求的上限 依据题目更改

// ==================== SPFA 单源版（入队次数判负环） ====================
// 返回 1 表示存在负环（源点可达），0 表示正常
bool spfa(vector<vector<pair<i64, i64>>> &Node, vector<i64> &dis,
          vector<i64> &pre_node, vector<i64> &cnt, int n, int start) {

    vector<bool> inq(n + 1, false);
    queue<i64> q;

    dis[start] = 0;
    q.push(start);
    inq[start] = true;
    cnt[start] = 1;

    while (!q.empty()) {
        i64 u = q.front();
        q.pop();
        inq[u] = false;

        if (dis[u] == INF) continue;

        for (auto &[v, w] : Node[u]) {
            if (dis[u] + w < dis[v]) {
                dis[v] = dis[u] + w;
                pre_node[v] = u;

                if (!inq[v]) {
                    inq[v] = true;
                    q.push(v);
                    cnt[v]++;
                    if (cnt[v] >= n) {
                        return 1; // 存在负环
                    }
                }
            }
        }
    }
```

```
    }
    return 0;
}

// ==================== SPFA 全源版（检测任意位置的负环） ====================
// 返回 1 表示存在负环，0 表示正常
bool spfa_all(vector<vector<pair<i64, i64>>> &Node, vector<i64> &dis,
              vector<i64> &pre_node, vector<i64> &cnt, int n) {

    vector<bool> inq(n + 1, true);
    queue<i64> q;

    for (int i = 1; i <= n; i++) {
        dis[i] = 0;   // 超级源点：所有点距离初始为 0
        cnt[i] = 1;
        q.push(i);
    }

    while (!q.empty()) {
        i64 u = q.front();
        q.pop();
        inq[u] = false;

        for (auto &[v, w] : Node[u]) {
            if (dis[u] + w < dis[v]) {
                dis[v] = dis[u] + w;
                pre_node[v] = u;

                if (!inq[v]) {
                    inq[v] = true;
                    q.push(v);
                    cnt[v]++;
                    if (cnt[v] > n) { // 全源版阈值是 > n
                        return 1;
                    }
                }
            }
        }
    }
    return 0;
}

// ==================== SPFA 边数法版（更稳健，推荐） ====================
// 返回 1 表示存在负环，0 表示正常
bool spfa_len(vector<vector<pair<i64, i64>>> &Node, vector<i64> &dis,
              vector<i64> &pre_node, vector<i64> &len, int n, int start) {

    vector<bool> inq(n + 1, false);
    queue<i64> q;

    dis[start] = 0;
    len[start] = 0;
    q.push(start);
    inq[start] = true;
```

```
    while (!q.empty()) {
        i64 u = q.front();
        q.pop();
        inq[u] = false;

        if (dis[u] == INF) continue;

        for (auto &[v, w] : Node[u]) {
            if (dis[u] + w < dis[v]) {
                dis[v] = dis[u] + w;
                pre_node[v] = u;
                len[v] = len[u] + 1;

                if (len[v] >= n) {
                    return 1; // 路径边数达到 n，必有负环
                }

                if (!inq[v]) {
                    inq[v] = true;
                    q.push(v);
                }
            }
        }
    }
    return 0;
}

// ==================== 路径回溯 ====================
vector<i64> get_path(vector<i64> &pre_node, i64 start, i64 end) {
    vector<i64> res;
    if (end < 1 || end >= (i64)pre_node.size()) return res;

    i64 cur = end;
    while (cur != -1) {
        res.push_back(cur);
        if (cur == start) { // 关键：遇到 start 就停
            reverse(res.begin(), res.end());
            return res;
        }
        cur = pre_node[cur];
    }
    cout << "不可达" << endl;
    return {};
}

// ==================== 主函数 ====================
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        i64 n, m;
        cin >> n >> m;
```

```
        vector<vector<pair<i64, i64>>> Node(n + 1);
        vector<i64> pre_node(n + 1, -1);
        vector<i64> dis(n + 1, INF);
        vector<i64> cnt(n + 1, 0);   // 入队次数法用
        // vector<i64> len(n + 1, 0); // 边数法用

        for (int i = 1; i <= m; i++) {
            i64 u, v, w;
            cin >> u >> v >> w;
            Node[u].push_back({v, w});
            // 无向图额外加：Node[v].push_back({u, w});
        }

        // ===== 选择一种调用方式 =====

        // 【1】单源 SPFA（从 1 号点出发），入队次数判负环
        if (spfa(Node, dis, pre_node, cnt, n, 1)) {
            cout << "YES" << endl; // 有负环
        } else {
            cout << "NO" << endl;
            // 最短路输出示例：
            // for (int i = 1; i <= n; i++) {
            //     if (dis[i] == INF) cout << "INF ";
            //     else cout << dis[i] << " ";
            // }
            // cout << endl;
        }

        // 【2】全源 SPFA（检测任意位置负环）
        // if (spfa_all(Node, dis, pre_node, cnt, n)) {
        //     cout << "YES" << endl;
        // } else {
        //     cout << "NO" << endl;
        // }

        // 【3】边数法 SPFA（更稳健）
        // if (spfa_len(Node, dis, pre_node, len, n, 1)) {
        //     cout << "YES" << endl;
        // } else {
        //     cout << "NO" << endl;
        // }
    }
    return 0;
}
```

Bellman-Ford

所属分类: 图论 › 最短路 › 单源最短路 › 存在负权边 复杂度: $O(VE)$

适用问题

- 含负权边的单源最短路
- 对经过边数有限制的最短路
- 判断负环(末轮仍在松弛)

有向无环图不管正负权, 都可以用拓扑序 + DP 求最短路。 但对于有环图, 我们无法确认当前的拓扑序, 就只能用最笨的方法: 对每条边遍历 n - 1 轮。 原因是第 k 轮后, 可以确定所有"最多 k 条边"的最短路径。
比如图 a -> b -> c -> d, 最坏情况按 c-d、b-c、a-b 的顺序遍历, 每轮只能多确定一条边的松弛, 所以最坏需要 n - 1 轮。 Bellman-Ford 本质就是一个不确定拓扑序顺序的暴力遍历每条边, 和拓扑序 + DP 求最短路类似。
Bellman-Ford 记录两个数组: 距离数组(起点距离为 0, 其他为正无穷)和前驱节点数组(用来回溯路径)。
每一轮都要遍历全部 E 条边, 因此总操作次数为 $(V - 1) \times E$, 即 $O(V \cdot E)$。
补充说明: 稠密图 E 接近 V^2, 复杂度约为 $O(V^3)$; 稀疏图 E 接近 V, 复杂度约为 $O(V^2)$。
对于 Bellman-Ford, 存在以下几种情况:
1. 提前发现距离数组不再变化, 就直接停止。
2. 最后一轮后还在变, 就再跑一次: 不再变化即有解; 还在变小则存在负权环(在环上能一直变小), 视为无解。

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'
const i64 INF = (1LL << 31) - 1;//题目要求的上限 依据题目更改

bool bellman_ford(vector<vector<pair<i64, i64>>> &Node, vector<i64> &dis,
                  vector<i64> &pre_node, int n) {
  bool stop = 0;
  for(int i = 0; i < n - 1; i++) {
    if(stop) return 0;
    stop = 1;
    for(int j = 1; j <= n; j++) {
      if (dis[j] == INF) continue;
      for(auto &i:Node[j]) {
        auto [u, w] = i;
        if(dis[j] + w < dis[u]) {
          dis[u] = dis[j] + w;
          pre_node[u] = j;
          stop = 0;
        }
      }
    }
  }

  //再进行一次测试 看看最后还变不变 还变就代表有负权环
  stop = 1;
  for(int j = 1; j <= n; j++) {
    if (dis[j] == INF) continue;
    for(auto &i:Node[j]) {
      auto [u, w] = i;
      if(dis[j] + w < dis[u]) {
        stop = 0;
      }
    }
  }
  if(!stop) {
```

```
    return 1;
  }
  return 0;

}

vector<i64> get_path(vector<i64> &pre_node, i64 start, i64 end) {
    vector<i64> res;
    if (end < 1 || end >= (i64)pre_node.size()) return res;

    i64 cur = end;
    while (cur != -1) {
        res.push_back(cur);
        if (cur == start) {                 // 关键：遇到 start 就停
            reverse(res.begin(), res.end());
            return res;
        }
        cur = pre_node[cur];
    }
    // 走到 -1 还没遇到 start，说明不可达
    cout << "不可达" << endl;
    return {};
}

int main() {
  int T;
  cin >> T;
  while (T--) {

    i64 n, m;
    cin >> n >> m;
    vector<vector<pair<i64, i64>>> Node(n + 1);
    vector<i64> pre_node(n + 1, -1);
    vector<i64> dis(n + 1, INF);
    dis[1] = 0;
    for(int i = 1; i <= m; i++) {
      i64 u, v, w;
      cin >> u >> v >> w;
      if(w >= 0) Node[v].push_back({u, w});
      Node[u].push_back({v, w});

    }

    if(bellman_ford(Node, dis, pre_node ,n)) {
      cout << "YES" << endl;
    } else cout << "NO" << endl;


  }


}
```

DAG 最短路

所属分类: 图论 › 最短路 › 单源最短路 › 有向无环图_正负均可 复杂度: $O(V + E)$

适用问题

- 有向无环图上正负权均可的最短路 / 最长路 / 路径计数(拓扑序 + DP)

对于有向无环图, 不管正负权都可以用拓扑序 + DP 求最短路, 时间复杂度为 O(V + E)。

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'

struct Edge {
    int to, w;
};

// 返回拓扑序，如果图有环则返回空 vector
vector<int> topo_sort(int n, vector<vector<Edge>> &adj, vector<int> &in_deg) {
    vector<int> res;
    queue<int> q;
    for (int i = 1; i <= n; i++) {
        if (in_deg[i] == 0) q.push(i);
    }
    while (!q.empty()) {
        int u = q.front(); q.pop();
        res.push_back(u);
        for (auto &e : adj[u]) {
            if (--in_deg[e.to] == 0) {
                q.push(e.to);
            }
        }
    }
    if ((int)res.size() != n) return {}; // 有环
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<Edge>> adj(n + 1);
    vector<int> in_deg(n + 1, 0);

    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;  // 如果无权，读 u,v 后 w=1
        adj[u].push_back({v, w});
        in_deg[v]++;
    }

    vector<int> topo = topo_sort(n, adj, in_deg);
    if (topo.empty()) {
        cout << "存在环，无法拓扑排序" << endl;
```

```
        return 0;
    }

    // 根据题目修改 DP 初始化
    vector<i64> dp(n + 1, 0);  // 示例：最长路，初始为 0，源点可设为 0
    // 如果需要每个点至少为 1，改成 vector<i64> dp(n + 1, 1);

    for (int u : topo) {
        for (auto &e : adj[u]) {
            int v = e.to, w = e.w;
            // 修改转移方程：
            // 最长路：dp[v] = max(dp[v], dp[u] + w);
            // 最短路：dp[v] = min(dp[v], dp[u] + w);
            // 路径计数：dp[v] += dp[u];
            dp[v] = max(dp[v], dp[u] + w);
        }
    }

    // 按题目要求输出
    for (int i = 1; i <= n; i++) {
        cout << dp[i] << endl;
    }

    return 0;
}
```

例题(DAG 最长链)
(注: 仅示意拓扑序 + DP, 未输出答案)

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'

vector<int> topo_sort(vector<vector<int>> &Node, vector<int>& in_deg) {
  vector<int> res;
  queue<int> q;
  for(int i = 1; i < Node.size(); i++) {
    if(in_deg[i] == 0) q.push(i);
  }
  while (!q.empty()) {
    int cur = q.front();
    q.pop();
    res.push_back(cur);
    for(auto &i:Node[cur]) {
      if(--in_deg[i] == 0) {
        q.push(i);
      }
    }
  }
  return res;
}

int main() {
  int n, m;
  cin >> n >> m;
```

```
  vector<vector<int>> Node(n + 1);
  vector<int> in_deg(n + 1, 0);
  for(int i = 1; i <= m; i++) {
    int u, v;
    cin >> u >> v;
    Node[u].push_back(v);
    in_deg[v]++;
  }
  vector<int> topo = topo_sort(Node, in_deg);
  vector<int> dis(n + 1, 1);
  for(auto &i:topo) {
    for(auto &j:Node[i]) {
      dis[j] = max(dis[j], dis[i] + 1);
    }
  }


}
```

多源汇最短路

Floyd

所属分类: 图论 › 最短路 › 多源汇最短路 复杂度: $O(n^3)$

适用问题

- 任意两点间的最短路径
- 回溯任意两点的具体路径(前驱矩阵)

核心思想

对于一个图(任何的) 我们想知道任意两点之间的最短路径 有两种方法

1. 对每个点 djs 这样的时间复杂度是 u^2*log(e) 但是有负数或者边很多的时候很亏
2. 对每个点 floyd 这样时间复杂度是 u ^ 3

floyd 的思想

1. 对每个点创建一个邻接矩阵 其中是他们边的距离 如果是自身 边距就是 0
2. 建立一个相同大小的矩阵 前驱节点矩阵 记录当前 a -> b 上 离 b 最近的节点
3. 对每个节点 x 进行遍历 尝试若 a -> b 中间走 x 即 a -> x -> b 会不会更短 若会更短 就更新走这条路 更新前驱节点矩阵和邻接矩阵
4. 对每个节点 a -> b 进行回溯 回溯到前驱节点为 a 为止即可

完整模板

```
#include <bits/stdc++.h>
#define endl '\n'
using i64 = long long;
const i64 INF = 1e9;
using namespace std;
void init(const i64 &n, const i64 &m, vector<vector<i64>> &G, vector<vector<i64>> &Pre) {
  G.resize(n + 1, vector<i64>(n + 1, INF));
  Pre.resize(n + 1, vector<i64>(n + 1, -1));
  for(int i = 0; i < m; i++) {
    i64 u, v, w;
    cin >> u >> v >> w;
    G[u][v] = min(w, G[u][v]);
    G[v][u] = min(w, G[v][u]);
    Pre[u][v] = u;
    Pre[v][u] = v;
  }
  for(int i = 1; i <= n; i++) {
    G[i][i] = 0;
  }
}

void Floyd(i64 n, i64 m) { //n个点m条边
  vector<vector<i64>> G;
  vector<vector<i64>> Pre;
  init(n, m, G, Pre);
  for(int k = 1; k <= n; k++) { //k为当前尝试的前驱点
    for(int i = 1; i <= n; i++) {
      if(i == k) continue; //如果含当前点的临接矩阵 不会变化
      for(int j = 1; j <= n; j++) {
        if(j == k) continue;
        if(G[i][j] > G[i][k] + G[k][j]) {
          G[i][j] = G[i][k] + G[k][j];
          Pre[i][j] = Pre[k][j];
        }
      }
```

```
    }
  }
  for(int i = 1; i <= n; i++) {
    for(int j = 1; j <= n; j++) {
      cout << G[i][j] << ' ';
    }
    cout << endl;
  }
}

vector<int> getPath(int s, int t, const vector<vector<i64>>& Pre) {
    vector<int> path;
    if (Pre[s][t] == -1) { // 不连通
        return path;
    }

    int cur = t;
    while (cur != s) {
        path.push_back(cur);          // 把当前节点加入路径
        cur = Pre[s][cur];
        if (cur == -1) return {};     // 安全保护
    }
    path.push_back(s);                // 最后加入起点
    reverse(path.begin(), path.end()); // 反转得到正向路径
    return path;
}

int main() {
  int n, m;
  cin >> n >> m;
  Floyd(n, m);

}
```

无权或 0-1 边权最短路

BFS

所属分类: 图论 › 最短路 › 无权或 *0-1* 边权最短路 / 图论 › 图的遍历 复杂度: $O(n + m)$

适用问题

- 无权图单源最短路(首次到达即最短路)
- 层序遍历、连通块
- 多源版本见 多源 BFS

用法

用于无权图最短路、拓扑排序、层序遍历，按层次逐层扩展，首次到达即为最短路。

完整模板

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> G;
vector<bool> vis;
vector<int> dis;

void bfs(int s) {
    queue<int> q;
    q.push(s);
    vis[s] = true;
    dis[s] = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : G[u]) {
            if (!vis[v]) {
                vis[v] = true;
                dis[v] = dis[u] + 1;
                q.push(v);
            }
        }
    }
}
```

多源 BFS

所属分类: 图论 › 最短路 › 无权或 0-1 边权最短路 复杂度: $O(n + m)$

适用问题

- 无权图中每个点到最近源点的距离(多个起点)
- 多源传染/燃烧蔓延的最短时间(每层 BFS = 一个时间单位)
- 多起点到目标的最短距离(反向多源)

核心思想

多源 BFS 本质: 你可以想象在棋盘外有一个虚拟的“超级源点”，用权值为 0 的边连接所有真实源点。多源 BFS 就是从该超级源点出发的单源 BFS，保证了所有源点处于同一“起跑线”上，公平竞争最近领地。

适用问题

1. 最近距离 / 最近服务点（静态图）

这是最经典的应用。给定若干个“服务点”（源点）和若干个“居民点”，求每个居民点离它最近的服务点有多远。

典型题目：LeetCode 542. 01 矩阵（求每个格子到最近 0 的距离）、LeetCode 1162. 地图分析（求离陆地最远的海洋，本质先求最近陆地距离）。

关键逻辑：所有源点初始距离为 0，同时入队，扩散时取 $\text{dist}[\text{新}] = \text{dist}[\text{旧}] + 1$。

2. 多源传染 / 燃烧蔓延（动态模拟）

模拟多个感染源、火源或病毒同时向四周扩散的过程，求整个图被“占领”所需的最短时间，或每个节点被“感染”的时间戳。

典型题目：LeetCode 994. 腐烂的橘子（求所有橘子腐烂的最短分钟数）。

关键逻辑：每一层 BFS 代表一个时间单位，队列中所有节点同步向外扩展一层，记录层数即为时间。

3. 多起点最优路径（简化状态）

当求解从“多个可能的起点”出发，到达某个终点的最短路径时，如果终点唯一，反向思维即可：把终点当作单源，反向 BFS；但若起点是一大片区域（而非离散点），多源 BFS 能极快求出起点集合到目标的最短距离。

4. 解决“被多个障碍物影响”的问题

在地图路径规划中，如果多个障碍物（或敌人）会周期性扩张领土，多源 BFS 可以预先计算出每个格子被扩张势力覆盖的“危险时间”，从而帮助主角找到一条避开所有危险源的安全路径（常用于游戏 AI 寻路）。

多源 BFS 的板子很简单 只需要在正常的 BFS 初始队列中 一次性加入所有的源头结点即可

完整模板

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
const i64 INF = 1e18;
#define endl '\n'

// 多源BFS: 无权图，求每个点到最近源点的距离
// sources: 所有源点; dis[i] == INF 表示不可达
void multi_source_bfs(i64 n, const vector<vector<i64>> &Node, const vector<i64> &sources,
                      vector<i64> &dis) {
  queue<i64> q;
  dis.assign(n + 1, INF);
  for(auto &s : sources) {
    dis[s] = 0;
    q.push(s);
  }
  while (!q.empty()) {
    i64 cur = q.front();
    q.pop();
    for(auto &i : Node[cur]) {
      if(dis[i] == INF) {
        dis[i] = dis[cur] + 1;
        q.push(i);
      }
```

```
    }
  }
}
```

例题(CF 986A Fair, 原题解仅修 3 处 bug)
n 个点每个点一种颜色(1..k), 求每个点离它最近的 s 种颜色的距离之和。

```
#include <bits/stdc++.h>
#include <iostream>
#include <queue>

using i64 = long long;
const i64 INF = 1e18;
#define endl '\n';
using namespace std;

void h_BFS(i64 n, vector<vector<i64>> &Node, vector<vector<i64>> &Good, deque<i64> &q,
           i64 good_now) {
  vector<bool> vis(n + 1, 0);
  vector<i64> dis(n + 1, INF);
  for(auto &i:q) {
    vis[i] = 1; //修复: 原为 0, 源点应标记已访问
    dis[i] = 0;
  }

  while (!q.empty()) {
    i64 cur = q.front();
    q.pop_front(); //修复: 原为 pop_back(), 与 front() 不一致
    for(auto &i:Node[cur]) {
      if(vis[i]) continue;
      vis[i] = 1;
      dis[i] = dis[cur] + 1;
      q.push_back(i); //修复: 原代码从不入队, BFS 不扩散
    }
  }

  for(int i = 1; i <= n; i++) {
    Good[i].push_back(dis[i]);
  }

}

int main() {
  i64 n, m, k, s;
  cin >> n >> m >> k >> s;
  vector<i64> a(n + 1);
  vector<vector<i64>> Node(n + 1);
  vector<vector<i64>> Good(n + 1);
  vector<deque<i64>> q(k + 1);

  for(int i = 1; i <= n; i++){
    cin >> a[i];
    q[a[i]].push_back(i);
  }

  for(int i =1; i <= m; i++) {
    i64 u, v;
```

```
    cin >> u >> v;
    Node[u].push_back(v);
    Node[v].push_back(u);
  }

  for(int i = 1; i <= k; i++) {
    h_BFS(n ,Node, Good, q[i], i);
  }

  for(int i = 1; i <= n; i++) {
    sort(Good[i].begin(), Good[i].end());
    i64 ans = 0 ;
    for(int j = 0; j < s; j++) {
      ans += Good[i][j];
    }
    cout << ans << ' ';
  }
}
```

注意

- 边权全为 1 才能用 BFS;0-1 边权用 0-1BFS;正权用 Dijkstra
- 多源 BFS 和多源 djs 几乎写法一致, 思想也一致(作者原话)

0-1 BFS

所属分类: 图论 › 最短路 › 无权或 *0-1* 边权最短路 复杂度: $O(n + m)$

适用问题

- 边权值可能有也可能没有(一般为 0 或 1)的最短路问题

适用范围

边权值为可能有，也可能没有（由于 BFS 适用于权值为 1 的图，所以一般权值是 0 或 1），或者能够转化为这种边权值的最短路问题.

例如在走迷宫问题中，你可以花 1 个金币走 5 步，也可以不花金币走 1 步，这就可以用 0-1 BFS 解决.

实现

一般情况下，我们把没有权值的边扩展到的点放到队首，有权值的边扩展到的点放到队尾．这样即可保证像普通 BFS 一样整个队列队首到队尾权值单调不下降.

完整模板

```
void bfs01(int s) {
  deque<int> dq;
  vector<int> dis(n + 1, INF);
  dis[s] = 0;
  dq.push_front(s);
  while (!dq.empty()) {
    int u = dq.front();
    dq.pop_front();
    for (auto [v, w] : E[u]) { // w 为 0 或 1
      if (dis[u] + w < dis[v]) {
        dis[v] = dis[u] + w;
        if (w == 0) dq.push_front(v); // 无权边放队首
        else dq.push_back(v);         // 有权边放队尾
      }
    }
  }
}
```

图的遍历

BFS 见「无权或 *0-1* 边权最短路」。

DFS

所属分类*:* 图论 › 图的遍历 复杂度*:* $O(n + m)$

适用问题

- 连通性判断、连通块
- 回溯搜索(排列/组合枚举)
- 有向图判环参见 三色 DFS;拓扑序参见 拓扑排序

完整模板

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> G;
vector<bool> vis;

void dfs(int u) {
    vis[u] = true;
    for (int v : G[u]) {
        if (!vis[v]) {
            dfs(v);
        }
    }
}
```

环问题

三色 DFS

所属分类: 图论 › 环问题 复杂度: $O(n + m)$
适用问题
1. 快速的判断图上是否有环
2. 给出最近找到的环的路径
3. 对每个点进行判断 判断其路径是否是安全的 即是否怎么走都不会走到环中

核心思想
三色 dfs 本质是 tarjan 的一个简写方式 对于有环的有向图 能快速的解决相关的简单问题 时间复杂度为 O(n)
逻辑如下 在 dfs 的递归中 定义如下三种状态
0.该节点从未被 dfs 访问过
1.该节点被访问过并且在栈中 意味着该结点的子结点还在被访问中 不能下定论
2.该节点被访问且已经不在栈中 意味该结点的子结点已经全部访问完了 并且没有环的存在
如果想要记录环的路径 只需要定义一个 path 数组 记录当前的路径 当遇到环时 回溯 path 到上一个相同的结点即可 如 path 1->2->3->2 从 2 回溯到上一个 2 就行了
需要判断是否安全 需要保证遍历了所有的结点 不要提前结束就行了

1.判断有没有环

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'
bool _3c_dfs(vector<vector<i64>> &Node, vector<i64> &color, i64 cur) {
  if(color[cur] == 0) color[cur] = 1;
  for(auto &i:Node[cur]) {
    i64 jud = 0;
    if(color[i] == 1) return 1;
    else if(color[i] == 0) {
      if(_3c_dfs(Node, color, i)) return 1;
    }
  }
  color[cur] = 2;
  return 0;
}

int main() {
  ios::sync_with_stdio(0);
  cin.tie(0);
  i64 n, m;
  cin >> n >> m;
  vector<vector<i64>> Node(n + 1);
  vector<i64> color(n + 1, 0); //0未访问 1在栈中 2已遍历完全
  for(int i = 1; i <= m; i++) {
    i64 u, v;
    cin >> u >> v;
    Node[u].push_back(v);
  }
  bool jud = 0;
  for(int i = 1; i <= n; i++) {
    if(color[i] == 0) jud |= _3c_dfs(Node, color, i);
  }
  cout << jud << endl;

}
```

2.判断环的同时给出路径

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'
vector<i64> get_path(vector<vector<i64>> &Node, vector<i64> &color, vector<i64> &Path, i64 node) {
  vector<i64> ans;
  for(int i = Path.size() - 1; i >= 0; i--) {
    ans.push_back(Path[i]);
    if(Path[i] == node) break;
  }
  return ans;
}

bool _3c_dfs(vector<vector<i64>> &Node, vector<i64> &color, vector<i64> &Path, i64 cur,
             vector<i64> &ans) {
  if(color[cur] == 0) color[cur] = 1;
  Path.push_back(cur);
  for(auto &i:Node[cur]) {
    i64 jud = 0;
    if(color[i] == 1) {
      ans = get_path(Node, color, Path, i);
      return 1;
    }
    else if(color[i] == 0) {
      if(_3c_dfs(Node, color, Path, i, ans)) {
        return 1;
      };
    }
  }
  color[cur] = 2;
  Path.pop_back();
  return 0;
}

int main() {
  ios::sync_with_stdio(0);
  cin.tie(0);
  i64 n, m;
  cin >> n >> m;
  vector<vector<i64>> Node(n + 1);
  vector<i64> color(n + 1, 0); //0未访问 1在栈中 2已遍历完全
  vector<i64> Path;
  vector<i64> ans;
  for(int i = 1; i <= m; i++) {
    i64 u, v;
    cin >> u >> v;
    Node[u].push_back(v);
  }
  for(int i = 1; i <= n; i++) {
    if(color[i] == 0 && _3c_dfs(Node, color, Path, i, ans)) {
      for(auto &x:ans) cout << x << ' ';
      return 0;
    }
  }
}
```

3.判断安全 不提前结束的写法

```
bool _3c_dfs(vector<vector<i64>> &Node, vector<i64> &color, vector<i64> &Path, i64 cur,
             vector<i64> &ans) {
  if(color[cur] == 0) color[cur] = 1;
  bool jud = 0;
  for(auto &i:Node[cur]) {
    if(color[i] == 2) continue;
    if(color[i] == 1) {
      jud = 1;
      continue;
    }
    if(color[i] == 0) {
      if(_3c_dfs(Node, color, Path, i, ans)) jud = 1;
    }
  }
  if(jud == 1) return 1;
  color[cur] = 2;
  return 0;
}
```

拓扑排序

所属分类: 图论 › 环问题 / 图论 › 为 DP 提供计算顺序 复杂度: $O(n + m)$

适用问题

- 有向无环图(DAG): 得到点的先后顺序, 供线性 DP/树形 DP 等需要前置顺序的算法使用
- 有环图: 拓扑排序后剩下的不在答案数组中的就是环上的节点
- 判环: 答案长度 $<$ n 说明有环

说明

拓扑排序

1. 对于有向无环图 得到点的先后顺序 得到先后顺序后 可以在 dp 之类需要前置顺序的算法中使用
2. 对于有环图 进行拓扑排序后 剩下的不在答案数组中的就是 环上的节点

时间复杂度为 $O(n)$

完整模板

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'

vector<i64> topo_sort(vector<i64> &In_Deg, vector<vector<i64>>  &Node, queue<i64> &q) {
  i64 n = Node.size() - 1;
  vector<i64> ans;
  for(int i = 1; i <= n; i++) {
    if(In_Deg[i] == 0) q.push(i);
  }
  while (!q.empty()) {
    i64 cur = q.front();
    q.pop();
    ans.push_back(cur);
    for(auto &i:Node[cur]) {
      In_Deg[i]--;
      if(In_Deg[i] == 0) {
        q.push(i);
      }
    }
  }
  return ans;
}

int main() {
  int n;
  cin >> n;
  vector<i64> ans;
  vector<i64> In_Deg(n + 1); //记录每个节点的入度
  vector<vector<i64>> Node(n + 1); //记录每个节点的出度
  queue<i64> q; //当前入度为0的节点
  for(int i = 1; i <= n; i++) {
    while (1) {
      int cur;
      cin >> cur;
      if(cur == 0) break;
      Node[i].push_back(cur);
      In_Deg[cur]++;
    }
  }
  ans = topo_sort(In_Deg, Node, q);
```

```
    for(auto &i:ans) cout << i << ' ';

}
```

无向图找环

所属分类: 图论 › 环问题 复杂度: $O(V+E)$(并查集版 $O(E\,\alpha(V))$)

适用问题

- 判断无向图是否有环
- 输出环上的节点路径(DFS 版)
- 只需判断时用并查集最快(不能输出路径)

下面三个板子都能返回是否存在结点数 ≥ 3 的环, 并尽可能给出环上的节点。

一、DFS 找环（推荐，能直接输出环路径）

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'

vector<vector<int>> Node;
vector<int> parent_;
vector<int> cycle;
vector<int> vis;
bool found = false;

void dfs(int u, int p) {
    vis[u] = 1;
    parent_[u] = p;
    for(auto &v : Node[u]) {
        if(found) return;
        if(v == p) continue;
        if(vis[v]) {
            // 找到环, 从 u 回溯到 v
            found = true;
            cycle.push_back(v);
            int cur = u;
            while(cur != v) {
                cycle.push_back(cur);
                cur = parent_[cur];
            }
            cycle.push_back(v);
            return;
        }
        if(!vis[v]) {
            dfs(v, u);
            if(found) return;
        }
    }
}

int main() {
    int n, m;
    cin >> n >> m;
    Node.assign(n + 1, {});
    vis.assign(n + 1, 0);
    parent_.assign(n + 1, -1);
    for(int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        if(u == v) continue;   // 忽略自环
```

```
        Node[u].push_back(v);
        Node[v].push_back(u);
    }
    for(int i = 1; i <= n; i++) {
        if(!vis[i]) {
            dfs(i, -1);
            if(found) break;
        }
    }
    if(!found) {
        cout << "无环" << endl;
    } else {
        cout << "有环: ";
        for(auto &x : cycle) cout << x << " ";
        cout << endl;
    }
}
```

说明：
- parent_[u] 记录 DFS 树上的父节点；
- 遇到 vis[v] && v != p 就是回边，说明有环；
- 从 u 沿着 parent_ 回溯到 v，路径就是环。

复杂度：O(V + E)

二、BFS 找环（能判断，路径略麻烦）

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'

vector<vector<int>> Node;
vector<int> parent_;
vector<int> vis;

bool bfs_check(int start) {
    queue<int> q;
    q.push(start);
    vis[start] = 1;
    parent_[start] = -1;
    while(!q.empty()) {
        int u = q.front();
        q.pop();
        for(auto &v : Node[u]) {
            if(!vis[v]) {
                vis[v] = 1;
                parent_[v] = u;
                q.push(v);
            } else if(v != parent_[u]) {
                return true;   // 有环
            }
        }
    }
    return false;
}

int main() {
    int n, m;
```

```
    cin >> n >> m;
    Node.assign(n + 1, {});
    vis.assign(n + 1, 0);
    parent_.assign(n + 1, -1);
    for(int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        if(u == v) continue;   // 忽略自环
        Node[u].push_back(v);
        Node[v].push_back(u);
    }
    bool found = false;
    for(int i = 1; i <= n; i++) {
        if(!vis[i]) {
            if(bfs_check(i)) {
                found = true;
                break;
            }
        }
    }
    if(found) cout << "有环" << endl;
    else cout << "无环" << endl;
}
```

说明：
- BFS 在无向简单图中遇到 `vis[v] && v != parent_[u]` 就说明有环；
- 想输出环路径的话，需要额外从 `u` 和 `v` 同时向上回溯到 LCA，比 DFS 麻烦，一般检测用 BFS，输出环用 DFS。

复杂度：`O(V + E)`

三、并查集找环（最简单，但不能输出环路径）

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'

vector<int> fa;

int find(int x) {
    return fa[x] == x ? x : fa[x] = find(fa[x]);
}

bool merge(int u, int v) {
    int ru = find(u), rv = find(v);
    if(ru == rv) return false;  // 已经连通，再加边就有环
    fa[ru] = rv;
    return true;
}

int main() {
    int n, m;
    cin >> n >> m;
    fa.resize(n + 1);
    for(int i = 1; i <= n; i++) fa[i] = i;

    bool found = false;
```

```
    for(int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        if(u == v) continue;   // 忽略自环
        if(!merge(u, v)) {
            found = true;
            break;
        }
    }
    if(found) cout << "有环" << endl;
    else cout << "无环" << endl;
}
```

说明：
- 遍历每条边，如果两端已经属于同一集合，说明加入这条边会形成环；
- 简单无向图中，这样找到的环长度一定 ≥ 3（自环已忽略，重边需额外去重）；
- 不能给出环上的具体节点，只能判断有没有环。

复杂度：$O(E \cdot \alpha(V))$，近似 $O(E)$

三个板子的对比

方法	能否判断	能否输出环路径	复杂度	适用场景
DFS	能	能，回溯即可	$O(V+E)$	需要具体环，最常用
BFS	能	麻烦，需要找 LCA	$O(V+E)$	只需要判断，或者求最短环
并查集	能	不能	$O(E\ \alpha(V))$	只需要判断，代码最短

注意事项

1. **自环**：`u == v` 是长度 1 的环，题目要求 ≥ 3，所以读入时直接跳过。
2. **重边**：`u - v` 有两条边会形成长度 2 的环，也不满足 ≥ 3，需要去重。 如果题目可能有重边，可以在读入时用 `set<pair<int,int>>` 去重，或者只保留一条。
3. **非连通图**：需要遍历所有未访问的点，确保每个连通分量都检查过。
4. **多个环**：以上代码找到第一个环就返回。如果需要找所有环，要用 Tarjan 求点双 / 边双，或者 DFS 枚举。

选择建议：

“**要判断有没有环，用并查集最快最简洁；要输出环上的节点，用 DFS；BFS 适合只需要判断或求最短环的场景。**”

为 DP 提供计算顺序

拓扑排序见「环问题」。

连通性问题

并查集

所属分类: 图论 › 连通性问题 复杂度: 路径压缩后近似 $O(1)$

适用问题

- 动态维护/查询集合的合并与连通性
- 无向图连通分量、Kruskal 判环

用法

用于动态维护和查询集合的合并与连通性，支持 Find（查找根结点）和 Union（合并集合）操作，路径压缩可优化查询效率至近似 O(1)。

实现步骤

1. 存储结构：
   - 使用数组，下标表示结点，数组值表示该结点的父结点，根结点的父结点是自己
2. Find 操作：
   - 普通写法：递归向上找根结点，不破坏原树结构
   - 路径压缩：递归时让路径上的每个结点直接指向根，优化后续查询
3. Union 操作：找到两个元素的根结点，若不同则让一个根指向另一个根

完整模板

```
vector<int> P;

int Find(int x) {
    if(P[x] == x) {
        return x;
    }
    return Find(P[x]);
}

int Find(int x) {
    if(P[x] == x) {
        return x;
    }
    return P[x] = Find(P[x]);
}

void Union(int x, int y) {
    int rootx = Find(x);
    int rooty = Find(y);
    if(rootx != rooty) {
        P[rootx] = rooty;
    }
}
```

Tarjan 强连通分量

所属分类: 图论 › 连通性问题 复杂度: $O(V + E)$

适用问题

- 求有向图的强连通分量
- 缩点(把每个 SCC 缩成一个点)
- 缩点后在 DAG 上做 DP(如点权最长路)

原理

tarjan 算法 能找出图中的最大联通量 从而帮助缩点 时间复杂度为 $O(n + m)$

对于一个图 是否联通 其实就是看这个结点是否能从一个结点到另一个结点 对于一个点 如果他的下一个结点能到达之前的结点 那它自身也可以到达 如果通过回溯边他能到达之前的结点 那它以及能到达他的边也能到达他能到达的结点 这个思想非常递归 所以只需要维护 dfn 和 low 即自己的 dfs 序 以及自己能到达的 dfn 序最小的前结点

对于每次 dfs 遍历每个结点 如果当前的结点是不在 dfs 栈中的 就认为是一个新的结点 要先对当前结点递归 因为他有可能是能找到更优路径的 更新 $low[u] = min(low[u], low[v])$

若当前节点已经在栈中了 那么此时遍历到的节点一定是前面的节点 此时更新 $low[u] = min(low[u], dfn[v])$ 这里是为了避免割点的时候错误 让点错误拿到子树的信息 一定要用 dfn

在无向割点边中 在递归结束后 如果发现

1. $low[u] >= dfn[v]$ 那就是说 不经过点 v 点 u 没有办法到达之前的节点 v 是一个割点
2. $low[u] > dfn[v]$ 也就是说 不经过边 uv 点 u 没办法到达之前节点 uv 是一个割边

```
#include <bits/stdc++.h>
using i64 = long long;
const i64 INF = 1e18;
#define endl '\n'
using namespace std;
i64 n, m;
i64 tim = 0;
i64 cnt = 0;
vector<i64> a, dfn, low, root, scc, scc_w;
vector<bool> instk;
vector<vector<i64>> Node;
stack<i64> S;
void tarjan(i64 u) {
  dfn[u] = low[u] = ++tim;//对每个刚进入的结点初始化并且入栈
  S.push(u);
  instk[u] = 1;
  for(auto &v:Node[u]) {
    if(dfn[v] == 0) {//如果dfs到的结点没有入栈 入栈
      tarjan(v);
      low[u] = min(low[u], low[v]);//自己的子结点是否有回溯路径更短
    } else if(instk[v]){
      //如果instk == 1 此时的路径是回溯结点 比对下是否更短
      //low的定义是自己的一条回溯结点 所以不要用low
      low[u] = min(low[u], dfn[v]);
    }
  }

  if(dfn[u] == low[u]) {
    cnt++;
    i64 sum = 0;
      while (true) {
          i64 v = S.top(); S.pop();
      scc[v] = cnt;
      sum += a[v];
```

```
            instk[v] = false;
            root[v] = u;
            if (v == u) break;
        }
      scc_w.push_back(sum);

    }//出栈 给每个结点缩点绑定根 并且统计总点权 标号从1开始
}

int main() {

    cin >> n >> m;
    a.assign(n + 1, 0);
    scc.assign(n + 1, 0);
    Node.resize(n + 1);
    dfn.assign(n + 1, 0);
    low.assign(n + 1, 0);
    root.assign(n + 1, 0);
    instk.assign(n + 1, 0);
    for(i64 i = 1; i <= n; i++) cin >> a[i];
    for(i64 i = 1; i <= m; i++) {
      i64 u, v;
      cin >> u >> v;
      Node[u].push_back(v);
    }
    scc_w.push_back(0); //入一个空结点 从而从1开始
    // 关键：对每个未访问节点运行 Tarjan 避免森林的存在
      for(i64 i = 1; i <= n; i++) {
          if (dfn[i] == 0) {
              tarjan(i);
          }
      }
    vector<vector<i64>> dag(cnt + 1);
    for (i64 u = 1; u <= n; u++) {
        for (auto v : Node[u]) {
            if (scc[u] != scc[v]) {
                dag[scc[u]].push_back(scc[v]);
            }
        }
    }


    vector<i64> dp(cnt + 1, 0);
      i64 ans = 0;

      for (i64 i = cnt; i >= 1; i--) {
          if (dp[i] == 0) dp[i] = scc_w[i];
          ans = max(ans, dp[i]);
          for (auto v : dag[i]) {
              dp[v] = max(dp[v], dp[i] + scc_w[v]);
          }
      }

      cout << ans << endl;
      return 0;
```

```
}
```

割点

所属分类: 图论 › 连通性问题 复杂度: $O(n + m)$

适用问题

- 求无向图的割点(删去后连通分量增加的节点)
- 同一份 tarjan 代码同时可求割边(见 割边.md)

```
#include <bits/stdc++.h>
using i64 = long long;
using namespace std;
#define endl '\n'
i64 n, m, ans, timer = 0;
vector<i64> dfn;
vector<vector<pair<i64, i64>>> Node;
vector<i64> low;
vector<bool> is_cut, is_bridge;

void tarjan(i64 u, i64 edge_id) {
  dfn[u] = low[u] = ++timer;
  i64 child = 0;
  for(auto [v, id]:Node[u]) {
    if(id == edge_id) continue;
    if(!dfn[v]) {
      child++;
      tarjan(v, id);
      low[u] = min(low[u], low[v]);
      if(edge_id != -1 && low[v] > dfn[u]) {
        is_bridge[id] = 1; // 【割边】low[v] > dfn[u]: 不经过边 uv 回不到更早 -> uv 是割边
      }
      if(edge_id != -1 && low[v] >= dfn[u]) {
        is_cut[u] = 1; // 【割点】low[v] >= dfn[u]: 子树不经过 u 回不到更早 -> u 是割点
      }
    } else {
      low[u] = min(low[u], dfn[v]);
    }
  }
  if (edge_id == -1 && child > 1) {
        is_cut[u] = true; // 【割点】根节点孩子数 > 1 才是割点
    }
}

int main() {

  cin >> n >> m;
  Node.resize(n + 1); //节点， 边编号
  dfn.assign(n + 1, 0);
  low.assign(n + 1, 0);
  is_cut.assign(n + 1, 0);
  is_bridge.assign(m + 1, 0);
  for(i64 i = 1; i <= m; i++) {
    i64 u, v;
    cin >> u >> v;
    Node[u].push_back({v, i});
    Node[v].push_back({u, i});
  }
  for(i64 i = 1; i <= n; i++) {
```

```
    if(!dfn[i]) tarjan(i, -1);
  }

  // ===== 【割点】输出部分 =====
  for(int i = 1; i <= n; i++) {
    if(is_cut[i]) ans++;
  }
  cout << ans << endl;
  for(int i = 1; i <= n; i++) {
    if(is_cut[i]) cout << i << ' ';
  }
  // 若要输出割边：遍历 is_bridge[1..m] 为 1 的边编号即可
}
```

割边

所属分类: 图论 › 连通性问题 复杂度: $O(n + m)$

适用问题

- 求无向图的割边(桥: 删去后连通分量增加的边)
- 同一份 tarjan 代码同时可求割点(见 割点.md)

```
#include <bits/stdc++.h>
using i64 = long long;
using namespace std;
#define endl '\n'
i64 n, m, ans, timer = 0;
vector<i64> dfn;
vector<vector<pair<i64, i64>>> Node;
vector<i64> low;
vector<bool> is_cut, is_bridge;

void tarjan(i64 u, i64 edge_id) {
  dfn[u] = low[u] = ++timer;
  i64 child = 0;
  for(auto [v, id]:Node[u]) {
    if(id == edge_id) continue;
    if(!dfn[v]) {
      child++;
      tarjan(v, id);
      low[u] = min(low[u], low[v]);
      if(edge_id != -1 && low[v] > dfn[u]) {
        is_bridge[id] = 1; // 【割边】low[v] > dfn[u]: 不经过边 uv 回不到更早 -> uv 是割边
      }
      if(edge_id != -1 && low[v] >= dfn[u]) {
        is_cut[u] = 1; // 【割点】low[v] >= dfn[u]: 子树不经过 u 回不到更早 -> u 是割点
      }
    } else {
      low[u] = min(low[u], dfn[v]);
    }
  }
  if (edge_id == -1 && child > 1) {
        is_cut[u] = true; // 【割点】根节点孩子数 > 1 才是割点
    }
}

int main() {

  cin >> n >> m;
  Node.resize(n + 1); //节点， 边编号
  dfn.assign(n + 1, 0);
  low.assign(n + 1, 0);
  is_cut.assign(n + 1, 0);
  is_bridge.assign(m + 1, 0);
  for(i64 i = 1; i <= m; i++) {
    i64 u, v;
    cin >> u >> v;
    Node[u].push_back({v, i});
    Node[v].push_back({u, i});
  }
  for(i64 i = 1; i <= n; i++) {
```

```
    if(!dfn[i]) tarjan(i, -1);
  }

  // ===== 【割点】输出部分(割边输出见下) =====
  for(int i = 1; i <= n; i++) {
    if(is_cut[i]) ans++;
  }
  cout << ans << endl;
  for(int i = 1; i <= n; i++) {
    if(is_cut[i]) cout << i << ' ';
  }
  // 【割边】输出: for(int i = 1; i <= m; i++) if(is_bridge[i]) cout << i << ' ';
}
```

点双连通分量

所属分类: 图论 › 连通性问题 复杂度: $O(n + m)$
适用问题

- 求无向图的点双连通分量(v-BCC, 块)
- 每个点双内部无割点; 割点可属于多个点双

点联通分量 v-BCC 也称作块 即对当前点分量及其边构成的图来说 去掉任意一个点 任意两点依然保持联通的极大图 也就是说这个图中没有割点
对于该问题 我们使用的解决方法依旧是 tarjan
我们知道 求割点的方式是判断 low[v] >= dfn[u] 但是单一的割点判断会使得一个割点同时在多个联通分量时出 bug 所以点双是直接入边 然后出栈的 最后对每个分量集合去重
板子如下

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'
i64 n, m, tim = 0, x = 0;
vector<i64> dfn, low;
stack<array<i64, 3>> S;
vector<vector<pair<i64, i64>>> Node;
vector<vector<i64>> ans;

void tarjan(i64 u, i64 p_edge) {
  dfn[u] = low[u] = ++tim;
  for(auto [v, id] : Node[u]) {
    if(id == p_edge) continue;
    if(!dfn[v]) {
      S.push({u, v, id});
      tarjan(v, id);
      low[u] = min(low[u], low[v]);
      if(low[v] >= dfn[u]) {
        x++;
        vector<i64> tmp;
        while (1) {
          auto[a, b, eid] = S.top();
          S.pop();
          tmp.push_back(a);
          tmp.push_back(b);
          if(eid == id) break;
        }
        sort(tmp.begin(), tmp.end());
        tmp.erase(unique(tmp.begin(), tmp.end()), tmp.end() );
        ans.push_back(tmp);
      }
    } else if(dfn[v] < dfn[u]) {//回边
      S.push({u, v, id});
      low[u] = min(low[u], dfn[v]);
    }
  }

}

int main() {
  ios::sync_with_stdio(0);
  cin.tie(0);
```

```
  cin >> n >> m;
  Node.resize(n + 1);
  dfn.resize(n + 1, 0);
  low.resize(n + 1, 0);

  for(int i = 1; i <= m; i++) {
    i64 u, v;
    cin >> u >> v;
    if (u == v) continue; // 忽略自环，最后孤立点逻辑会处理只有自环的点 按照题目要求决定放不放
    Node[u].push_back({v, i});
    Node[v].push_back({u, i});
  }

  for(int i = 1; i <= n; i++) {
    if(!dfn[i]) {
      if(Node[i].empty()) {
        ans.push_back({i});
      } else tarjan(i, -1);
    }
  }

  cout << ans.size() << endl;
  for(auto &i:ans) {
    cout << i.size() << ' ';
    for(auto &j:i) {
      cout << j << ' ';
    }
    cout << endl;
  }

}
```

边双连通分量

所属分类: 图论 › 连通性问题 复杂度: $O(n + m)$

适用问题

- 求无向图的边双连通分量(e-BCC)
- 每条边双内部去任意一条边仍连通; 缩点后是一棵树

边联通分量 e-BCC 即对当前点分量及其边构成的图来说 去掉任意一个边 任意两点依然保持联通的极大图 对于该问题 我们使用的解决方法依旧是 tarjan tarjan 的本质其实就是判断 一个点除了原先的来边 有没有别的办法能回到更早的结点

有向图中 对于一个点 如果他能回到比自己 dfs 序 更早的结点 那么理所当然的 这个结点和早结点间都有两条路径可以到达 所以这必然是一个边联通分量 那么对于 $\text{low[u]} == \text{dfn[u]}$ 的结点 这个结点就必然是联通的终点了

所以当我们遇到 $\text{low[u]} == \text{dfn[u]}$时候 只需要在 tarjan 的基础上进行弹栈 就可以了 这就是一个联通集

```cpp
#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
using i64 = long long;

i64 n, m, timer;
vector<vector<pair<i64, i64>>> Node;
vector<i64> low, dfn;
stack<i64> S;
vector<vector<i64>> ans;


void tarjan(i64 u, i64 p_id) {
  dfn[u] = low[u] = ++timer;
  S.push(u);
  for(auto [v, id]:Node[u]) {
    if(id == p_id) continue;
    if(dfn[v] == 0) {
      tarjan(v, id);
      low[u] = min(low[u], low[v]);
    } else if(dfn[v] < dfn[u]) {
      low[u] = min(low[u], dfn[v]);
    }
  }

  if(dfn[u] == low[u]) {
    vector<i64> tmp;
    while (1) {
      i64 cur = S.top();
      S.pop();
      tmp.push_back(cur);
      if(cur == u) break;
    }
    ans.push_back(tmp);
  }

}


int main() {
  cin >> n >> m;
  Node.resize(n + 1);
```

```
  low.assign(n + 1, 0);
  dfn.assign(n + 1, 0);
  for(int i = 1; i <= m; i++) {
    i64 u, v;

    cin >> u >> v;
    if(u == v) continue;
    Node[u].push_back({v, i});
    Node[v].push_back({u, i});
  }

  for(int i = 1; i <= n; i++) {
    if(!dfn[i]) tarjan(i, -1);
  }

  cout << ans.size() << endl;
  for(auto &i:ans) {
    cout << i.size() << ' ';
    for(auto &j:i) {
      cout << j << ' ';
    }
    cout << endl;
  }

}
```

2-SAT

所属分类: 图论 › 连通性问题(*Tarjan* 应用) 复杂度: $O(n+m)$ 备注: *2026-10-18* 初稿, 待你核对

适用问题

- 布尔变量约束的可行解判定: 条件形如 $(x_i = a) \vee (x_j = b)$(a,b $\in \{0,1\}$)
- 对应题目: P4782 【模板】2-SAT 问题、CF1971H ±1(已做)

建图

- 每个变量 x_i 拆成两个点(本项目约定): 点 `i` $= x_i$ 取 0, 点 `i+n` $= x_i$ 取 1
- 条件 $(x_i = a) \vee (x_j = b)$ 转成两条蕴含边: $\neg(x_i = a) \to (x_j = b)$、$\neg(x_j = b) \to (x_i = a)$(「至少一个成立」等价于「其中一个不成立则另一个必须成立」)
- 代码读入 (i,a,j,b) 后: `x` $= x_i$ 取 a 的点(a=1 取 `i+n`, 否则取 `i`), `nx` $=$ `x` 的反点(`i` 与 `i+n` 互换); `y`/`ny` 同理; 加边 `nx` $\to$ `y`、`ny` $\to$ `x`
- 建好图后跑一遍 Tarjan 求 SCC(模板见下)

模板(Tarjan)

```cpp
#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
using i64 = long long;
i64 n, m;
vector<vector<i64>> Node;
vector<i64> dfn, low, root, scc;
vector<bool> instk;
stack<i64> S;
i64 tim = 0, cnt = 0;

void tarjan(i64 u) {
  dfn[u] = low[u] = ++ tim;
  S.push(u);
  instk[u] = 1;
  for(auto &v:Node[u]) {
    if(dfn[v] == 0) {
      tarjan(v);
      low[u] = min(low[u], low[v]);
    } else if(instk[v]) {
      low[u] = min(low[u], dfn[v]);
    }
  }

  if(dfn[u] == low[u]) {
    cnt++;
    while (1) {
      i64 v = S.top(); S.pop();
      scc[v] = cnt;
      instk[v] = 0;
      root[v] = u;
      if(v == u) break;
    }
  }

}

void _2SAT() {
  cin >> n >> m;
  Node.resize(2 * n + 1); // 0 到 n - 1 是0 n到2n是1
  scc.assign(2 * n + 1, 0);
```

```
  dfn.assign(2 * n + 1, 0);
  low.assign(2 * n + 1, 0);
  root.assign(2 * n + 1, 0);
  instk.assign(2 * n + 1, 0);

  for(int k = 0; k < m; k++) {
    i64 i, a, j, b;
    cin >> i >> a >> j >> b;
    i64 x = i, y = j, nx = x + n, ny = y + n;
    if(a == 1) {x = i + n; nx = i;}
    if(b == 1) {y = j + n; ny = j;}
    Node[nx].push_back(y);
    Node[ny].push_back(x);
  }

  for(int i = 0; i <= 2 * n; i++) {
    if(dfn[i] == 0) {
      tarjan(i);
    }
  }

  for(int i = 1; i <= n; i++) {
    if(scc[i] == scc[i + n]) {cout << "IMPOSSIBLE"; return;}
  }
  cout << "POSSIBLE" << endl;
  for(int i = 1; i <= n; i++) {
    if(scc[i] > scc[i + n]) cout << 1 << ' ';
    else cout << 0 << ' ';
  }

}

int main() {
  ios::sync_with_stdio(0);
  cin.tie(0);

  _2SAT();


}
```

注: 模板内 `Node.resize(2 * n + 1); // 0 到 n - 1 是 0 n 到 2n 是 1` 这行注释按 0-based 描述, 实际本板子按 1-based 使用$(1..n \ / \ n+1..2n)$。

判定与取值

- 若存在 `i` 使 `scc[i] == scc[i+n]`(x_i 的两种取值点在同一 SCC, 矛盾): 无解, 输出 IMPOSSIBLE
- 否则有解: `scc[i] > scc[i+n]` $\rightarrow x_i = 1$, 否则 $x_i = 0$(与 Tarjan 出栈顺序有关, 直接用本板子即可)

一笔画问题

欧拉路径与欧拉回路

所属分类: 图论 › 一笔画问题 复杂度: $O(n + m)$

适用问题

- 判断图是否存在欧拉路径/欧拉回路(一笔画)
- 输出一条欧拉路径/欧拉回路(Hierholzer 算法)

8.1 判定条件

用法

判断一个图是否存在欧拉路径或欧拉回路（一笔画问题）：能否从一点出发，每条边恰好经过一次（欧拉回路要求最终回到起点）。

判定条件（图需连通/弱连通）

- 无向图：
  - 欧拉回路：所有点的度数均为偶数
  - 欧拉路径：恰有 0 或 2 个奇度点（恰有 2 个奇度点时，它们分别是起点和终点）
- 有向图：
  - 欧拉回路：所有点 入度 $=$ 出度
  - 欧拉路径：恰有一个点 出度 - 入度 $= 1$（起点），恰有一个点 入度 - 出度 $= 1$（终点），其余点 入度 $=$ 出度

8.2 Hierholzer 算法

用法

在 $\mathcal{O}(n+m)$ 时间内求出一条欧拉路径/欧拉回路。

实现步骤

1. 选起点：欧拉回路任选一个有边的点；欧拉路径选出度 - 入度 $= 1$ 的点（无向图选奇度点）
2. 从起点 DFS，每走过一条边就删掉这条边（无向图还要删掉反向边）
3. 走到无路可走时，将当前点压入答案
4. 答案逆序输出即为欧拉路径/回路

完整模板（有向图）

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

vector<vector<int>> G; // 邻接表
vector<int> indeg, outdeg;
vector<int> ans;
int n, m;

bool check() { // 有向图判定：是否存在欧拉路径/回路
    int start_cnt = 0, end_cnt = 0;
    for (int i = 1; i <= n; i++) {
        if (outdeg[i] - indeg[i] == 1) start_cnt++;
        else if (indeg[i] - outdeg[i] == 1) end_cnt++;
        else if (indeg[i] != outdeg[i]) return false;
    }
    return (start_cnt == 0 && end_cnt == 0) || (start_cnt == 1 && end_cnt == 1);
}

void hierholzer(int u) {
    while (!G[u].empty()) {
        int v = G[u].back();
        G[u].pop_back();
        hierholzer(v);
    }
    ans.push_back(u); // 逆序输出 ans 即欧拉路径
```

```
}

int main() {
    cin >> n >> m;
    G.resize(n + 1);
    indeg.assign(n + 1, 0);
    outdeg.assign(n + 1, 0);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        G[u].push_back(v);
        outdeg[u]++;
        indeg[v]++;
    }
    if (!check()) {
        cout << "No" << endl;
        return 0;
    }
    int start = 1;
    for (int i = 1; i <= n; i++) {
        if (outdeg[i] - indeg[i] == 1) start = i;
    }
    hierholzer(start);
    reverse(ans.begin(), ans.end());
    for (auto &i : ans) {
        cout << i << ' ';
    }
    return 0;
}
```

完整模板（无向图）

```
vector<vector<pair<int, int>>> G; // (邻接点，边编号)
vector<bool> used; // 边是否被使用过
vector<int> ans;

void dfs(int u) {
    while (!G[u].empty()) {
        auto [v, eid] = G[u].back();
        G[u].pop_back();
        if (used[eid]) continue;
        used[eid] = true;
        dfs(v);
    }
    ans.push_back(u);
}
```

最小生成树

Prim

所属分类: 图论 › 最小生成树 复杂度: $O(m\ log\ n)$, 适合稠密图

适用问题

- 稠密图最小生成树(MST)
- 判断图是否连通(cnt == n 才成树)

用法

在**稠密图**中求解最小生成树，从任意结点出发，通过维护当前连通块到未连通结点的最小边权，逐步扩展连通块。时间复杂度 $\mathcal{O}(m \log n)$（优先队列实现）。

实现步骤

1. 初始化距离数组 `dist[]` 为无穷大，选择任意起点（如结点 1），将其距离设为 0
2. 使用优先队列（小顶堆）存储（距离，结点），将起点入队
3. 用 `vis[]` 数组维护已加入最小生成树的结点
4. 每次取出队首距离最小的未访问结点，标记为已访问，累加其距离到总权值
5. 遍历该结点的所有邻接点，若邻接点未访问且当前边权小于其 `dist`，更新 `dist` 并入队

完整模板

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
const int INF = 0x3f3f3f3f;

vector<vector<pair<int, int>>> G;
vector<int> dist;
vector<bool> vis;

i64 prim(int n, int start = 1) {
    dist.assign(n + 1, INF);
    vis.assign(n + 1, false);
    dist[start] = 0;
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
    pq.emplace(0, start);
    i64 total = 0;
    int cnt = 0;
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (vis[u]) continue;
        vis[u] = true;
        total += d;
        cnt++;
        for (auto [v, w] : G[u]) {
            if (!vis[v] && w < dist[v]) {
                dist[v] = w;
                pq.emplace(dist[v], v);
            }
        }
    }
    return cnt == n ? total : -1;
}

int main() {
    int n, m;
```

```
    cin >> n >> m;
    G.resize(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        G[u].emplace_back(v, w);
        G[v].emplace_back(u, w);
    }
    i64 ans = prim(n);
    if (ans == -1) cout << "orz" << endl;
    else cout << ans << endl;
    return 0;
}
```

Kruskal

所属分类: 图论 › 最小生成树 复杂度: $O(m\ log\ m)$, 适合稀疏图

适用问题

- 稀疏图最小生成树(MST)
- 判断图是否连通

用法

在**稀疏图**中更优，通过将所有边按权值排序，结合并查集逐步合并连通分量，每次选择连接两个不同连通分量的最小边。时间复杂度 $\mathcal{O}(m \log m)$（主要为边排序的时间）。

实现步骤

1. 定义边的结构体，包含起点、终点、权值
2. 将所有边按权值从小到大排序
3. 初始化并查集，每个结点独立成一个集合
4. 遍历排序后的边，若边的两个端点不在同一连通分量，则合并它们，并累加该边权值到总权值
5. 当合并次数达到 `n-1` 时（已形成生成树），提前退出

完整模板

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

struct Edge {
    int u, v, w;
    Edge(int u, int v, int w) : u(u), v(v), w(w) {}
    bool operator<(const Edge& other) const {
        return w < other.w;
    }
};

vector<Edge> edges;
vector<int> P;

int Find(int x) {
    if (P[x] != x) P[x] = Find(P[x]);
    return P[x];
}

i64 kruskal(int n) {
    sort(edges.begin(), edges.end());
    P.resize(n + 1);
    for (int i = 1; i <= n; i++) P[i] = i;
    i64 total = 0;
    int cnt = 0;
    for (auto& e : edges) {
        int rootu = Find(e.u);
        int rootv = Find(e.v);
        if (rootu != rootv) {
            P[rootu] = rootv;
            total += e.w;
            cnt++;
            if (cnt == n - 1) break;
        }
    }
    return cnt == n - 1 ? total : -1;
}
```

```cpp
int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        edges.emplace_back(u, v, w);
    }
    i64 ans = kruskal(n);
    if (ans == -1) cout << "orz" << endl;
    else cout << ans << endl;
    return 0;
}
```

二分图

二分图判定

所属分类: 图论 › 二分图 复杂度: $O(n + m)$
适用问题
- 判断一个图是否为二分图(染色法: 相邻节点颜色不同)
- 输出二分图的两部分点集

根据匈牙利算法 我们已经发现了 如果一个图是二分图 那么这个图上的每个相邻节点 假设有黑白两种颜色 相邻节点的颜色都不相同
所以我们可以对图进行一次 bfs 或者是 dfs 的遍历 对每个结点进行染色 然后对他的相邻结点染色相反色 当 bfs 完成 没有发现冲突 这个图就是一个二分图 如果发现存在冲突的 则退出 不是染色图

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<int>> g(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }

    vector<int> color(n + 1, -1);
    queue<int> q;

    for (int i = 1; i <= n; i++) {
        if (color[i] != -1) continue;
        color[i] = 0;
        q.push(i);

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : g[u]) {
                if (color[v] == -1) {
                    color[v] = color[u] ^ 1;
                    q.push(v);
                } else if (color[v] == color[u]) {
                    cout << -1 << '\n';
                    return 0;
                }
            }
        }
    }

    vector<int> A, B;
    for (int i = 1; i <= n; i++) {
```

```
        if (color[i] == 1) A.push_back(i);
        else B.push_back(i);  // color[i] == 0
    }

    // 输出 A
    cout << A.size() << '\n';
    for (int x : A) cout << x << ' ';
    cout << '\n';

    // 输出 B
    cout << B.size() << '\n';
    for (int x : B) cout << x << ' ';
    cout << '\n';

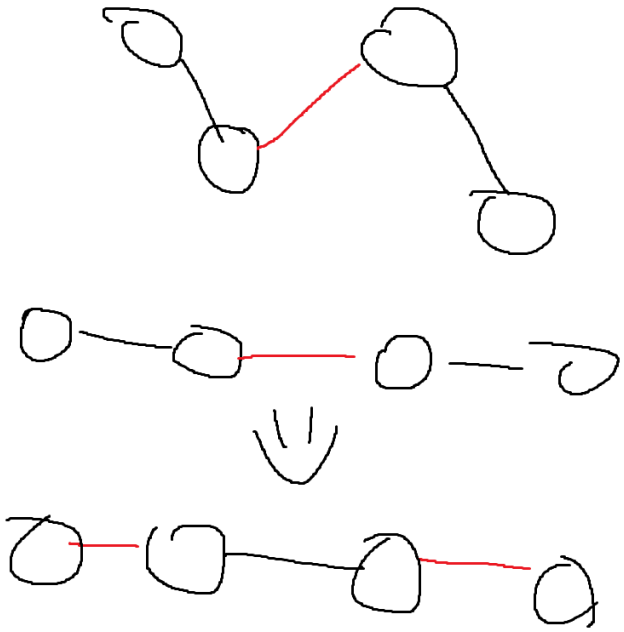
    return 0;
}
```

匈牙利算法

所属分类: 图论 › 二分图 复杂度: $O(V \cdot E)$
适用问题
- 二分图最大匹配
- 基于增广路的匹配算法(dfs 版)

匈牙利算法 本质是基于二分图增广路的算法
增广路在二分图中的定义是
1. 首尾边是非匹配边
2. 匹配边和首尾边交替出现
3. 此时交换匹配边和首尾边 匹配边就会+1

如下图



所以要求二分图最大匹配 只需要对每个点不断的求增广路即可 单次的时间复杂度为 $O(n + m)$ 找的次数为 $\min$(两边点数) 假设点分布均匀 则最差为 $O(n(n + m))$
寻找的过程可以用 dfs 来不断寻找递归 只需要维护
1. g[u] 左节点能连接的节点
2. match_r, 当前右节点匹配的左节点
3. timer 计算当前轮次
4. vis 数组 在循环中作为当前轮次的判断

板子

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'

i64 n, m, e, timer = 0;
vector<vector<i64>> g;
vector<i64> match_r;
vector<i64> vis;

bool dfs(i64 u) {
  for(auto &v:g[u]) {
    if(vis[v] == timer) continue;
```

```
    vis[v] = timer;
    if(match_r[v] == -1 || dfs(match_r[v])) {
      match_r[v] = u;
      return 1;
    }
  }
  return 0;
}

int main() {

  cin >> n >> m >> e;
  g.resize(n + 1);
  match_r.assign(m + 1, -1);
  vis.assign(m + 1, 0);

  for(int i = 1; i <= e; i++) {
    i64 u, v;
    cin >> u >> v;
    g[u].push_back(v);
  }

  i64 ans = 0;
  for(i64 i = 1; i <= n; i++) {
    timer++;
    if(dfs(i)) ans++;
  }
  cout << ans << endl;
}
```

树上问题

LCA(最近公共祖先)

所属分类: 图论 › 树上问题 复杂度: 树链剖分 预处理 $O(n)$/查询 $O(log\ n)$;倍增 预处理 $O(n\ log\ n)$/查询 $O(log\ n)$

适用问题

- 树上两点的最近公共祖先
- 树上两点距离: dist(u,v) = dep[u] + dep[v] - 2*dep[lca]

2.5.1 树链剖分

用法

预处理 O(n)，查询 O(logn)，通过重链剖分将树划分为若干重链，快速跳转到 LCA。

实现步骤

1. 第一个 DFS（dfs1）：处理父结点 father、深度 deepth、子树大小 size、重子结点 son
2. 第二个 DFS（dfs2）：创建链顶 top，重子结点的链顶为父结点的链顶，轻子结点的链顶为自己
3. 寻找 LCA：
   - 若二者不在同一个重链上，让链顶深度大的点跳转到他链顶的父结点
   - 直到二者在同一个重链上，深度更小的点为 LCA

完整模板

```
vector<vector<int>> G;
vector<int> fa, dep, siz, son, top;

void dfs1(int u, int f) {
  fa[u] = f, dep[u] = dep[f] + 1, siz[u] = 1;
  for (auto v : G[u]) {
    if (v == f) continue;
    dfs1(v, u);
    siz[u] += siz[v];
    if (siz[v] > siz[son[u]]) son[u] = v;
  }
}

void dfs2(int u, int f) {
    if (son[f] == u)
        top[u] = top[f];
    else
        top[u] = u;
    for (int v : G[u]) {
        if (v != f)
            dfs2(v, u);
    }
}

int LCA(int a, int b) {
    while(top[a] != top[b]) {
        if (dep[top[a]] > dep[top[b]]) a = fa[top[a]];
        else b = fa[top[b]];
    }
    return dep[a] <= dep[b] ? a : b;
}
```

2.5.2 倍增

用法

预处理 O(nlogn)，查询 O(logn)，利用二进制思想，通过倍增表快速跳转到祖先结点。

实现步骤

1. DFS：处理父亲函数 fa[0][]（2^0 = 1）和深度 dep

2. 倍增表初始化：第 2^k 祖先是第 2^(k-1)祖先的第 2^(k-1)祖先
3. LCA 函数：
   - 先将二者跳到同一深度
   - 若相同则直接返回
   - 否则从 2^k 到 2^0 尝试跳转，直到找到 LCA

完整模板

```
vector<vector<int>> Edge;
vector<vector<int>> fa;
vector<int> dep;
int n;

void dfs(int u, int f) {
    fa[0][u] = f;
    dep[u] = dep[f] + 1;
    for(auto &i:Edge[u]) {
        if(i != f) dfs(i, u);
    }
}

void init() {
    for(int i = 1; i <= 20; i++) {
        for(int j = 1; j <= n; j++) {
            fa[i][j] = fa[i - 1][fa[i - 1][j]];
        }
    }
}

int LCA(int a, int b) {
    if(dep[a] < dep[b]) swap(a, b);
    for(int i = 20; i >= 0; i--) {
        if(dep[fa[i][a]] >= dep[b]) a = fa[i][a];
    }
    if(a == b) return a;
    for(int i = 20; i >= 0; i--) {
        if(fa[i][a] != fa[i][b]) {
            a = fa[i][a];
            b = fa[i][b];
        }
    }
    return fa[0][a];
}
```

树的遍历(前中后序)

所属分类: 图论 › 树上问题 复杂度: $O(n)$

适用问题

- 按前序(根-左-右)/中序(左-根-右)/后序(左-右-根)访问二叉树所有结点
- 用栈模拟递归(避免递归深度过大)

2.2 前中后序遍历

用法

按照不同顺序访问树的所有结点，分为前序（根-左-右）、中序（左-根-右）、后序（左-右-根），可通过递归或栈实现。

实现步骤

- 前序：中左右，递归序 $= 1$
- 中序：左中右，递归序 $= 2$
- 后序：左右中，递归序 $= 3$

递归写法：

```
void 递归() {
  if(head == NULL) {
    return; //修复: 原为 break, void 函数编译错误
  }
  //输出head 前序
  递归(head.left)
  //输出head 中序
  递归(head.right)
  //输出head 后序

}
```

完整模板（栈写法）

```
#include <iostream>
#include <vector>
#include <stack>
#include <algorithm>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

vector<int> preorderTraversal(TreeNode* root) {
    vector<int> result;
    if (!root) return result;
    stack<TreeNode*> stk;
    stk.push(root);
    while (!stk.empty()) {
        TreeNode* node = stk.top();
        stk.pop();
        result.push_back(node->val);
        if (node->right) stk.push(node->right);
        if (node->left)  stk.push(node->left);
    }
    return result;
}
```

```cpp
vector<int> inorderTraversal(TreeNode* root) {
    vector<int> result;
    stack<TreeNode*> stk;
    TreeNode* cur = root;
    while (cur || !stk.empty()) {
        while (cur) {
            stk.push(cur);
            cur = cur->left;
        }
        cur = stk.top();
        stk.pop();
        result.push_back(cur->val);
        cur = cur->right;
    }
    return result;
}

vector<int> postorderTraversal(TreeNode* root) {
    vector<int> result;
    if (!root) return result;
    stack<TreeNode*> stk;
    TreeNode* cur = root;
    TreeNode* prev = nullptr;
    while (cur || !stk.empty()) {
        while (cur) {
            stk.push(cur);
            cur = cur->left;
        }
        cur = stk.top();
        if (!cur->right || cur->right == prev) {
            result.push_back(cur->val);
            stk.pop();
            prev = cur;
            cur = nullptr;
        } else {
            cur = cur->right;
        }
    }
    return result;
}

vector<int> postorderTraversal_reverse(TreeNode* root) {
    vector<int> result;
    if (!root) return result;
    stack<TreeNode*> stk;
    stk.push(root);
    while (!stk.empty()) {
        TreeNode* node = stk.top();
        stk.pop();
        result.push_back(node->val);
        if (node->left)  stk.push(node->left);
        if (node->right) stk.push(node->right);
    }
    reverse(result.begin(), result.end());
    return result;
```

```
}

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    cout << "前序遍历: ";
    for (int x : preorderTraversal(root)) cout << x << " ";
    cout << "\n中序遍历: ";
    for (int x : inorderTraversal(root)) cout << x << " ";
    cout << "\n后序遍历: ";
    for (int x : postorderTraversal(root)) cout << x << " ";
    return 0;
}
```

树的直径

所属分类: 图论 › 树上问题 复杂度: $O(n)$

适用问题

- 求非负权树的直径(最远两点的距离)
- 两次 BFS / 两次 DFS 求直径

```cpp
// 树的直径有很多种类 对于不同的种类也有不同的求法

// 1.直接的 求树的直径 并且树的直径是非负权边(>=0)
// 两次BFS/DFS 时间复杂度为O(n)

// 先对树上的任意一点u进行BFS/DFS 找到离这个点最远的v 在树的概念上 就是以一个点为根节点 找他的最深的叶节点
// 然后对找到的u/v再进行一次BFS/DFS 找到离v最远的点 这个距离就是树的直径
// 这非常的直觉 不需要怎么证明也能知道

#include <bits/stdc++.h>
using i64 = long long;
using namespace std;
#define endl '\n'

int n;
vector<vector<int>> Node;

pair<int, int> bfs(int x) { //返回最远点和距离
  vector<int> dis(n + 1, -1);
  deque<int> dq;
  int u = x;
  dq.push_back(x);
  dis[x] = 0;
  while (!dq.empty()) {
    int cur = dq.front();
    dq.pop_front();
    if (dis[cur] > dis[u]) u = cur;
    for (auto &i : Node[cur]) {
      if (dis[i] != -1) continue;
      dis[i] = dis[cur] + 1;
      dq.push_back(i);
    }
  }
  return {u, dis[u]};
}

int main() {
  ios::sync_with_stdio(0);
  cin.tie(0);

  cin >> n;
  Node.assign(n + 1, {});
  for (int i = 1; i < n; i++) {
    int u, v;
    cin >> u >> v;
    Node[u].push_back(v);
    Node[v].push_back(u);
  }
```

```
  if (n == 1) { cout << 0 << endl; return 0; }

  auto [u, _] = bfs(1);
  auto [v, dis] = bfs(u);
  cout << dis << endl;
}
```

```
#include <bits/stdc++.h>
using i64 = long long;
using namespace std;
#define endl '\n'

int n;
vector<vector<int>> Node;
vector<int> dis;
int far;
int MAX;

void dfs(int cur) {
  if(dis[cur] > MAX) {
    MAX = dis[cur];
    far = cur;
  }
  for(auto &i : Node[cur]) {
    if(dis[i] != -1) continue;
    dis[i] = dis[cur] + 1;
    dfs(i);
  }
}

void run(int start) {
  dis.assign(n + 1, -1);
  dis[start] = 0;
  MAX = -1;
  dfs(start);
}

int main() {
  ios::sync_with_stdio(0);
  cin.tie(0);

  cin >> n;
  Node.assign(n + 1, {});
  for (int i = 1; i < n; i++) {
    int a, b;
    cin >> a >> b;
    Node[a].push_back(b);
    Node[b].push_back(a);
  }

  run(1);
  int p = far;
  run(p);
  cout << MAX << endl;
}
```

五、字符串

子串是否相等

字符串哈希

所属分类: 字符串 › 子串是否相等 复杂度: 预处理 $O(n)$ / 查询 $O(1)$

适用问题

- 快速比较区间子串是否相等
- 判断回文(正反哈希比较)

用法

维护字符串区间哈希值，快速比较子串是否相等、判断回文等。

- 预处理时间：$\mathcal{O}(n)$
- 查询时间：$\mathcal{O}(1)$

实现步骤

1. 生成随机基数 `base`（用 `mt19937_64` 生成，防卡哈希）
2. 预处理 `base` 的幂次数组 `power`，其中 `power[i] = base^i`
3. 预处理前缀哈希数组 `hash_pre`，其中 `hash_pre[i]` 表示前 `i` 个字符的哈希值
4. 查询区间 `[l, r]`（1-based）的哈希值：`hash_pre[r] - hash_pre[l-1] * power[r-l+1]`

完整模板

```cpp
#include <bits/stdc++.h>
#include <random>
#include <chrono>
using namespace std;
using i64 = long long;
using u64 = unsigned long long;

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

const int MAXN = 1e5 + 5;
u64 base;
u64 power[MAXN];
u64 hash_pre[MAXN];

void init_hash() {
    base = rng();
    power[0] = 1;
    for (int i = 1; i < MAXN; i++) {
        power[i] = power[i-1] * base;
    }
}

void build_hash(const string &s) {
    int n = s.size();
    for (int i = 0; i < n; i++) {
        hash_pre[i+1] = hash_pre[i] * base + s[i];
    }
}

u64 get_hash(int l, int r) {
    return hash_pre[r] - hash_pre[l-1] * power[r - l + 1];
}

int main() {
    init_hash();
```

```
    string s;
    cin >> s;
    build_hash(s);
    int l = 2, r = 4;
    cout << get_hash(l, r) << endl;
    return 0;
}
```

单模式串匹配

KMP

所属分类: 字符串 › 单模式串匹配 复杂度: $O(n + m)$
适用问题
- 在主串中查找模式串的所有出现位置(单模式匹配)

用法
用于单模式串匹配，可在主串中快速查找模式串的所有出现位置，避免暴力匹配的回溯，时间复杂度 $\mathcal{O}(n + m)$。
实现步骤
对于一串字符串 如 匹配串: abcdabxa... 模式串: abcdabcy
可以发现 abcdab 是匹配的 且 abcdab 的最长前后缀存在为 ab 因为前面都匹配 只有从 c-x 不匹配 所以 x 前的肯定是 ab 那就意味着可以从 c-x 开始比较 而不用从头开始遍历
完整模板

```
#include <bits/stdc++.h>
using namespace std;
int main() {
  string s1;
  string s2;
  cin >> s1 >> s2;
  int s2_len = s2.size();
  int s1_len = s1.size();
  vector<int> next(s2_len, 0);
  int i = 0, j = 1;
  while(i < s2_len && j < s2_len) {
    while (i > 0 && s2[i] != s2[j]) {
      i = next[i - 1];
    }
    if(s2[i] == s2[j]) i++;
    next[j] = i;
    j++;
  }
  i = 0;
  j = 0;
  while (j < s1_len) {
    while(i > 0 && s2[i] != s1[j]) {
      i = next[i - 1];
    }
    if(s2[i] == s1[j]) {
      i++;
    }
    if(i == s2_len) {
      cout << j - i + 2 << endl;
      i = next[i - 1]; //修复: 回退以继续找重叠匹配
    }
    j++;
  }
  for(auto &i:next) {
    cout << i << ' ';
  }
}
```

最长回文子串

Manacher(马拉车)

所属分类: 字符串 › 最长回文子串 复杂度: $O(n)$

适用问题

- 线性时间求最长回文子串长度
- 回文半径(以每个位置为中心的最长回文)

用法

用于在线性时间 $\mathcal{O}(n)$ 内求解字符串的最长回文子串长度。

实现步骤

1. 预处理字符串：插入 # 统一奇偶回文
2. 维护 maxr、mid、ans
3. 遍历中心，利用对称性扩展回文半径

完整模板

```cpp
#include <bits/stdc++.h>
using namespace std;
string change(string &s) {
  string temp = "!#";
  for(auto &c:s) {
    temp += c;
    temp += '#';
  }
  temp += "^";
  return temp;
}

int manacher(string &s) {
  int maxr = 0, mid = 0, ans = 0;
  vector<int> p(s.size(), 0);
  for(int i = 0; i < s.size(); i++) {
    if(maxr >= i) {
      p[i] = min(p[2 * mid - i], maxr - i);
    }
    while(s[i - p[i] - 1] == s[i + p[i] + 1])
    p[i]++;
    if(i + p[i] > maxr) {
      mid = i;
      maxr = i + p[i];
    }
    ans = max(ans, p[i]);
  }
  return ans;
}

int main() {
  string s;
  cin >> s;
  s = change(s);
  cout << manacher(s);
}
```

六、数论与数学

幂运算与逆元

快速幂

所属分类: 数论与数学 › 幂运算与逆元 复杂度: $O(log\ b)$

适用问题

- 求 a^b mod p(指数极大)
- 矩阵快速幂/模逆元的基础

3. 快速幂模板

```cpp
#include <iostream>
using namespace std;
using i64 = long long;

i64 fast_pow(i64 base, i64 exponent, i64 mod) {
    i64 result = 1;
    base %= mod;
    while (exponent > 0) {
        if (exponent % 2 == 1) {
            result = (result * base) % mod;
        }
        base = (base * base) % mod;
        exponent /= 2;
    }
    return result;
}

i64 fermat_inverse(i64 a, i64 p) {
    return fast_pow(a, p - 2, p);
}

int main() {
    i64 a, p;
    cin >> a >> p;
    cout << fermat_inverse(a, p) << endl;
    return 0;
}
```

费马小定理与逆元

所属分类: 数论与数学 › 幂运算与逆元 复杂度: $O(log\ p)$

适用问题

- 质数模下的模逆元: a^{-1} = a^{p-2} (mod p), p 为质数且 gcd(a,p)=1

6.1 费马小定理笔记

1. 定义与基本形式

定义：若 p 是质数，且 $\gcd(a,p)=1$，则：

$$a^{p-1} \equiv 1 (\mathrm{mod}\, p)$$

一般形式：

$$a^{p} \equiv a (\mathrm{mod}\, p)$$

2. 核心应用：模逆元

$$a^{-1} \equiv a^{p-2} (\mathrm{mod}\, p)$$

3. 快速幂模板

```cpp
#include <iostream>
using namespace std;
using i64 = long long;

i64 fast_pow(i64 base, i64 exponent, i64 mod) {
    i64 result = 1;
    base %= mod;
    while (exponent > 0) {
        if (exponent % 2 == 1) {
            result = (result * base) % mod;
        }
        base = (base * base) % mod;
        exponent /= 2;
    }
    return result;
}

i64 fermat_inverse(i64 a, i64 p) {
    return fast_pow(a, p - 2, p);
}

int main() {
    i64 a, p;
    cin >> a >> p;
    cout << fermat_inverse(a, p) << endl;
    return 0;
}
```

线性递推加速

矩阵快速幂

所属分类: 数论与数学 › 线性递推加速 复杂度: $O(m^3 \log k)$, m 为矩阵大小
适用问题
- 线性递推求第 n 项(如斐波那契)
- 图上长度为 k 的路径计数、DP 转移矩阵快速幂

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
const i64 MOD = 1e9 + 7;
#define endl '\n'
vector<vector<i64>> cacl_mat(vector<vector<i64>> &M1, vector<vector<i64>> &M2) {
  i64 m = M1.size(), n = M2[0].size(), o = M1[0].size();
  vector<vector<i64>> res(m, vector<i64>(n, 0));
  for(int i = 0; i < m; i++) {
    for(int j = 0; j < n; j++) {
      for(int k = 0; k < o; k++) {
        res[i][j] += (M1[i][k] * M2[k][j]) % MOD;
        res[i][j] %= MOD;
      }
    }
  }
  return res;
} //矩阵乘法通用模板

vector<vector<i64>> mat_ksm(vector<vector<i64>> M, i64 k) { // M转换矩阵 k指数
  i64 n = M.size(); //矩阵大小
  vector<vector<i64>> res(n, vector<i64>(n));
  for(int i = 0; i < n; i++) {
    res[i][i] = 1;
  } //初始化单位矩阵
  while (k) {
    if(k & 1) {
      res = cacl_mat(res, M);
    }
    M = cacl_mat(M, M);
    k >>= 1;
  }
  return res;
}

int main() {
  i64 n; cin >> n;
  vector<vector<i64>> M = {{1, 1}, {1, 0}};
  vector<vector<i64>> ans = mat_ksm(M, n - 1);
  cout << ans[0][0];
}
```

典题

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
const i64 MOD = 998244353;
```

```cpp
#define endl '\n'

vector<vector<vector<i64>>> Begi;

vector<vector<i64>> cacl_mat(const vector<vector<i64>> &M1, const vector<vector<i64>> &M2) {
    i64 m = M1.size(), n = M2[0].size(), o = M1[0].size();
    vector<vector<i64>> res(m, vector<i64>(n, 0));
    for(int i = 0; i < m; i++) {
        for(int k = 0; k < o ; k++) {
            if(M1[i][k] == 0) continue;
            for(int j = 0; j < n; j++) {
                res[i][j] += (M1[i][k] * M2[k][j]) % MOD;
                res[i][j] %= MOD;
            }
        }
    }
    return res;
} //矩阵乘法通用模板

vector<vector<i64>> mat_ksm(vector<vector<i64>> M, i64 k) { // M转换矩阵 k指数
    i64 n = M.size(); //矩阵大小
    vector<vector<i64>> res(n, vector<i64>(n));
    for(int i = 0; i < n; i++) {
        res[i][i] = 1;
    } //初始化单位矩阵
    while (k) {
        if(k & 1) {
            res = cacl_mat(res, M);
        }
        M = cacl_mat(M, M);
        k >>= 1;
    }
    return res;
}

i64 ksm(i64 a, i64 b) {
    i64 res = 1;
    i64 base = a;
    while (b) {
        if(b & 1) {
            res *= base;
        }
        base *= base;
        b >>= 1;
    }
    return res;
}

// 新增：颜色标准化函数，将序列按第一次出现顺序映射为 0,1,2,...
vector<int> canonical(const vector<int>& a) {
    vector<int> res(a.size());
    vector<int> mp(3, -1);  // 只有三种颜色
    int cur = 0;
    for (int i = 0; i < (int)a.size(); ++i) {
        if (mp[a[i]] == -1) mp[a[i]] = cur++;
```

```
        res[i] = mp[a[i]];
    }
    return res;
}

void get(i64 n) {
    i64 MAXN = ksm(3, n);
    vector<vector<i64>> cur(1, vector<i64>(n, 0));
    for(i64 i = 0; i < MAXN; i++) {
        i64 NUM = i;
        i64 pre = 0;
        bool jud = 1;
        for(int j = 0; j < n; j++) {
            i64 dig_now = (NUM % 3);
            if(j != 0) {
                if(pre == dig_now) {
                    jud = 0;
                    break;
                }
            }
            cur[0][j] = dig_now;
            NUM /= 3;
            pre = dig_now;
        }

        if(jud) {
            Begi.push_back(cur);
        }
    }
}

i64 get_mat(i64 n, i64 m) {
    int s = Begi.size();  // 原始合法状态数

    // 提取颜色序列
    vector<vector<int>> states(s, vector<int>(n));
    for (int i = 0; i < s; ++i) {
        for (int k = 0; k < n; ++k) {
            states[i][k] = Begi[i][0][k];
        }
    }

    // 构建等价类
    map<vector<int>, int> id;
    int idx = 0;
    vector<int> sz;                 // 每个等价类的大小
    vector<vector<int>> repr;       // 每个等价类的代表状态

    for (int i = 0; i < s; ++i) {
        vector<int> canon = canonical(states[i]);
        if (!id.count(canon)) {
            id[canon] = idx++;
            sz.push_back(0);
            repr.push_back(states[i]);   // 第一个遇到的作为代表
        }
```

```
        sz[id[canon]]++;
    }

    int K = idx;  // 等价类数量

    // 构建带权转移矩阵
    vector<vector<i64>> trans(K, vector<i64>(K, 0));
    for (int i = 0; i < K; ++i) {
        const auto& a = repr[i];
        for (int j = 0; j < s; ++j) {
            const auto& b = states[j];
            bool ok = true;
            for (int k = 0; k < n; ++k) {
                if (a[k] == b[k]) {
                    ok = false;
                    break;
                }
            }
            if (ok) {
                int nxt = id[canonical(b)];
                trans[i][nxt] = (trans[i][nxt] + 1) % MOD;
            }
        }
    }

    if (m == 1) {
        return s % MOD;   // 第一列可以取所有原始合法状态
    }

    // 矩阵快速幂
    vector<vector<i64>> T = mat_ksm(trans, m - 1);

    // 初始向量：各等价类的大小
    vector<i64> ans(K, 0);
    for (int i = 0; i < K; ++i) {
        for (int j = 0; j < K; ++j) {
            ans[j] = (ans[j] + sz[i] * T[i][j]) % MOD;
        }
    }

    i64 res = 0;
    for (int j = 0; j < K; ++j) {
        res = (res + ans[j]) % MOD;
    }
    return res;
}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n, m;
    cin >> n >> m;
    get(n);
    cout << get_mat(n, m);
    return 0;
}
```

例题中的 $\mathrm{ksm}(3,\ \mathrm{n})$ 未取模, n 较大时会溢出(仅服务于该题的合法状态枚举)

最大公约数_不定方程

GCD 与扩展欧几里得

所属分类: 数论与数学 › 最大公约数_不定方程 复杂度: $O(log\ n)$

适用问题

- 求最大公约数 gcd / 最小公倍数 lcm
- 求解不定方程 $\mathrm{ax} + \mathrm{by} = \mathrm{gcd(a,\ b)}$ 的一组特解

6.3 GCD（最大公约数）

用法

求解两个整数的最大公约数，辗转相除递归即可。扩展欧几里得（exgcd）可求解不定方程 $ax + by = \gcd(a, b)$ 的一组特解。

完整模板

```
int gcd(int a, int b) {
    return b ? gcd(b, a % b) : a;
}

int gcd(int a, int b) {
    if(b == 0) return a;
    else return gcd(b, a % b);
}

int exgcd(int a, int b, int &x, int &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    int g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}
```

与 n 互质的个数

欧拉函数

所属分类: 数论与数学 › 与 n 互质的个数 复杂度: 线性筛 $O(n)$
适用问题
- 求 $\varphi(n) = [1,n]$ 中与 n 互质的整数个数
- 筛出 1 n 的所有 φ 值

这份笔记涵盖了欧拉函数的定义、通式推导，以及如何将其嵌套进线性筛（欧拉筛）的完整逻辑。

📖 **欧拉函数 $\phi(n)$ 笔记**

1. 定义与通式

定义：$\phi(n)$ 表示 $[1,n]$ 中与 n 互质的整数个数。

通式：若 $n = p_1^{k_1} p_2^{k_2} ... p_m^{k_m}$（质因数分解），则：

$$\phi(n) = n \prod_{i=1}^{m} \left(1 - \frac{1}{p_i}\right)$$

---

2. 核心性质推导（用于线性筛）

线性筛的核心是通过 $n = i \times p$（p 为最小质因子）来递推。我们需要根据 i 是否包含 p 分两种情况：

情况 A：$p \mid i$（p 整除 i）
- **前提**：i 已经包含了质因子 p。
- **推导**：根据通式，$\phi(i)$ 已经包含了 $\left(1 - \frac{1}{p}\right)$ 这个项。当计算 $\phi(i \times p)$ 时，由于**质因子种类没有增加**，通式右边的乘积项 $\prod\left(1 - \frac{1}{p_i}\right)$ 完全不变。唯一的改变是前面的系数从 i 变成了 $i \times p$。
- **结论**：

$$\phi(i \times p) = \phi(i) \times p$$

情况 B：$p \nmid i$（p 不整除 i）
- **前提**：p 是一个全新的质因子，且 p 是质数，故 $\gcd(i,p) = 1$。
- **推导**：根据积性函数性质：$\phi(i \times p) = \phi(i) \times \phi(p)$。由于 p 是质数，$\phi(p) = p - 1$。
- **结论**：

$$\phi(i \times p) = \phi(i) \times (p - 1)$$

---

3. 线性筛欧拉函数标准模板（C++）

```cpp
#include <iostream>
#include <vector>

using namespace std;

typedef long long i64;
const int MAXN = 20000005; // 根据题目要求修改

int phi[MAXN];
int primes[1500000]; // 预估素数数量
bool is_not_prime[MAXN];
int cnt = 0;

void precompute_phi(int n) {
    phi[1] = 1; // 1的欧拉函数定义为1
    for (int i = 2; i <= n; ++i) {
        if (!is_not_prime[i]) {
            primes[cnt++] = i;
            phi[i] = i - 1; // 性质：素数 p 的 phi(p) = p - 1
        }
        for (int j = 0; j < cnt; ++j) {
            i64 temp = (i64)i * primes[j];
            if (temp > n) break;

            is_not_prime[temp] = true;
```

```
            if (i % primes[j] == 0) {
                // 情况 A: primes[j] 是 i 的老面孔 (p | i)
                // 质因子种类没变，只增加数值倍数
                phi[temp] = phi[i] * primes[j];
                break; // 线性筛精髓：找到最小质因子后停止
            } else {
                // 情况 B: primes[j] 是新质因子 (p 不整除 i)
                // 利用积性函数性质 phi(i*p) = phi(i) * (p-1)
                phi[temp] = phi[i] * (primes[j] - 1);
            }
        }
    }
}

int main() {
    // 快速 I/O
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    precompute_phi(20000000);

    int T, x;
    if (!(cin >> T)) return 0;
    while (T--) {
        cin >> x;
        cout << phi[x] << "\n";
    }
    return 0;
}
```

---

4. 记忆要点总结

1. **初始化**：`phi[1] = 1`。
2. **遇到素数**：`phi[i] = i - 1`。
3. **内层标记**：
   - **能整除**：说明质因子已经在地盘里了，直接**乘** p。
   - **不能整除**：说明来了个新质因子，要**乘** $(p-1)$。

这两个公式的推导，本质上是观察**“质因子的种类”**有没有发生变化。我们再次祭出欧拉函数的通式：

$$\phi(n) = n \cdot \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

这个公式告诉我们：$\phi(n)$ 的值由 n 本身 和 n 包含的所有不同质因子 共同决定。公式一：当 $i \pmod p = 0$ 时这种情况意味着 p 已经是 i 的质因子了。分析 $\phi(i)$：

$$\phi(i) = i \cdot \left(1 - \frac{1}{p_1}\right) \cdot \left(1 - \frac{1}{p_2}\right) \dots \cdot \left(1 - \frac{1}{\boldsymbol{p}}\right)$$

分析 $\phi(i \cdot p)$：当我们把 i 乘以 p 得到新数时，新数的质因子集合与 i 的完全一样（因为 p 本来就在里面）。所以，公式右边的连乘部分（那些括号）一个都不会变。

$$\phi(i \cdot p) = (i \cdot p) \cdot \underbrace{\left(1 - \frac{1}{p_1}\right) \dots \cdot \left(1 - \frac{1}{p}\right)}_{\text{这部分就是 } \frac{\phi(i)}{i}}$$

得出结论：

$$\phi(i \cdot p) = (i \cdot p) \cdot \frac{\phi(i)}{i} = \boldsymbol{\phi(i) \cdot p}$$

直观理解：质因子种类没变，“打折比例”就没变，数值翻了 p 倍，结果自然也翻 p 倍。公式二：当 $i \pmod p \neq 0$ 时这种情况意味着 p 是一个全新的质因子，它不在 i 原有的质因子列表里。分析 $\phi(i)$：

$$\phi(i) = i \cdot \prod_{p_{\text{old}} | i} \left(1 - \frac{1}{p_{\text{old}}}\right)$$

分析 $\phi(i \cdot p)$：因为 p 是新来的，根据通式，我们不仅要把前面的系数乘以 p，还要在后面多乘一个新括号 $\left(1 - \frac{1}{p}\right)$。

$$\phi(i \cdot p) = (i \cdot p) \cdot \left[\prod_{p_{\text{old}} | i} \left(1 - \frac{1}{p_{\text{old}}}\right)\right] \cdot \left(\mathbf{1} - \frac{\mathbf{1}}{\boldsymbol{p}}\right)$$

化简：中间那一团就是 $\phi(i)$，所以：

$$\phi(i \cdot p) = \phi(i) \cdot p \cdot \left(1 - \frac{1}{p}\right)$$

把 p 乘进括号里：$p \times \left(1 - \frac{1}{p}\right) = p - 1$。

$$\boldsymbol{\phi(i \cdot p) = \phi(i) \cdot (p - 1)}$$

筛素数

欧拉筛(线性筛)

所属分类: 数论与数学 › 筛素数 复杂度: $O(n)$

适用问题

- 求 [1,n] 内所有素数(每个合数只被最小质因子筛掉一次)

7.1 欧拉筛（线性筛）

用法

欧拉筛（线性筛）用于在 $O(n)$ 时间内求出 $[1,n]$ 范围内的所有素数，每个合数只被其最小质因子筛掉一次，避免了埃氏筛的重复标记。

实现步骤

1. 维护 `is_not_prime[]` 标记合数，`primes` 列表存储素数
2. 遍历 $i = 2 \ldots n$：
   - 若 i 未被标记，则为素数，加入 `primes`
   - 遍历 `primes`，将 $i \times$ `primes`$[j]$ 标记为合数
   - 若 $i \bmod$ `primes`$[j] = 0$，则 **break**
3. 最终 `primes` 列表即为 $[1,n]$ 内所有素数

完整模板

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
const i64 MAXN = 1e7 + 5; //需要的素数的范围 自行修改
vector<i64> primes; //求得的素数数组
bool is_not_prime[MAXN]; //不是素数的判断

void euler_sieve(int n) {
    is_not_prime[0] = is_not_prime[1] = true;//标记0和1
    for (int i = 2; i <= n; i++) {
        if (!is_not_prime[i]) {
            primes.push_back(i);
        }
        for (int p : primes) {
            if ((long long)i * p > n) break;
            is_not_prime[i * p] = true;
            if (i % p == 0) break; //合数 = 最小质数 * 一个数 保证当前是最小质数 才能保证合数的唯一性
        }
    }
}
```

线性筛因数

所属分类: 数论与数学 › 筛素数 复杂度: 筛 $O(n)$ / 分解质因数 $O(log\ n)$

适用问题

- 筛出 1 n 每个数的最小质因子 spf
- O(log n) 分解质因数
- 由质因数分解求因数个数/约数和

完整模板

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
const i64 MAXN = 1e7 + 5; //需要的范围 自行修改
vector<i64> primes; //求得的素数数组
bool is_not_prime[MAXN]; //不是素数的判断
i64 spf[MAXN]; //每个数的最小质因子

void euler_sieve(int n) {
    is_not_prime[0] = is_not_prime[1] = true; //标记0和1
    spf[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!is_not_prime[i]) {
            primes.push_back(i);
            spf[i] = i; //质数的最小质因子是自己
        }
        for (int p : primes) {
            if ((long long)i * p > n) break;
            is_not_prime[i * p] = true;
            spf[i * p] = p;
            if (i % p == 0) break; //合数 = 最小质数 * 一个数 保证当前是最小质数 才能保证合数的唯一性
        }
    }
}

// 分解质因数 O(log n)
vector<pair<i64, i64>> factorize(i64 x) {
    vector<pair<i64, i64>> res;
    while (x > 1) {
        i64 p = spf[x];
        i64 cnt = 0;
        while (x % p == 0) {
            x /= p;
            cnt++;
        }
        res.push_back({p, cnt});
    }
    return res;
}
```

注意

- 若只需素数表, 用 线性筛;需分解质因数/求因数个数时用本板子

等比数列求和

所属分类: 数论与数学 › 等比数列求和 复杂度: $O(1)$(公式)或 $O(\log n)$(结合快速幂)

适用问题

- 等比数列前 n 项和、无穷等比级数(|q|<1)

6.2 等比数列求和公式笔记

1. 基本公式

设等比数列首项为 a_1，公比为 q，项数为 n。

当 $q \neq 1$ 时：

$$S_n = \frac{a_1(1-q^n)}{1-q}$$

当 $q = 1$ 时：

$$S_n = na_1$$

2. 公式推导（错位相减法）

$$S_n = a_1 + a_1q + a_1q^2 + \cdots + a_1q^{n-1}$$
$$qS_n = a_1q + a_1q^2 + \cdots + a_1q^{n-1} + a_1q^n$$

两式相减：

$$S_n - qS_n = a_1 - a_1q^n$$
$$(1-q)S_n = a_1(1-q^n)$$
$$\therefore S_n = \frac{a_1(1-q^n)}{1-q}$$

3. 无穷等比数列（|q|<1）

$$S = \frac{a_1}{1-q}$$

4. 常用变形

已知条件	公式
已知 a_1、a_n	$S_n = \frac{a_1 - a_nq}{1-q}$
已知 a_1、a_n	$S_n = \frac{a_nq - a_1}{q-1}$

其中 $a_n = a_1q^{n-1}$。

整除取整技巧

向上取整

所属分类: 数论与数学 › 整除取整技巧 复杂度: $O(1)$

适用问题

- 求 $\lceil a/b \rceil$, 避免浮点数误差
- 二进制数组求和技巧: 每个数 $/2$ 向下取整的和 $=$ (数组总和 - 奇数的个数) $/$ 2

```
⌈a / b⌉ = (a + b - 1) / b

// 每个数/2向下取整的和 = (数组总和 - 奇数个数) / 2
sum = (total - odd_cnt) / 2
```

组合数学

组合排列

所属分类: 数论与数学 › 组合数学 用途: 排列数 $A(n,m)$ 与组合数 $C(n,m)$ 的定义/公式/区分
适用问题
- 排列数/组合数的概念与公式速记: 定义、公式、区别与联系、记忆方法

在中国数学教材里，排列数和组合数分别用 **A** 和 **C** 表示：
1. 排列数 $A(n,m)$
定义：从 n 个不同元素中，取出 m 个元素，**按照一定顺序排成一列**，有多少种排法。
公式：

$$A(n,m)=\frac{n!}{(n-m)!}$$

也可以写成连乘：

$$A(n,m)=n(n-1)(n-2)\cdots(n-m+1)$$

例子：从 5 个人中选 3 个人排队，有多少种排法?

$$A(5,3)=5\times4\times3=60$$

2. 组合数 $C(n,m)$
定义：从 n 个不同元素中，取出 m 个元素，**不考虑顺序**，有多少种取法。
公式：

$$C(n,m)=\frac{n!}{m!(n-m)!}$$

也可以写成：

$$C(n,m)=\frac{A(n,m)}{m!}$$

例子：从 5 个人中选 3 个人组成小组，有多少种选法?

$$C(5,3)=\frac{5\times4\times3}{3\times2\times1}=10$$

3. 两者的区别与联系

名称	符号	是否考虑顺序	公式
排列数	$A(n,m)$ 或 $P(n,m)$	考虑顺序	$\frac{n!}{(n-m)!}$
组合数	$C(n,m)$ 或 $\binom{n}{m}$	不考虑顺序	$\frac{n!}{m!(n-m)!}$

关系：

$$C(n,m)=\frac{A(n,m)}{m!}$$

因为选出的 m 个元素，如果考虑顺序有 $m!$ 种排法，组合只算一种，所以要除以 $m!$。
4. 记忆方法
- **A**：Arrangement（排列），强调“排”，有顺序。
- **C**：Combination（组合），强调“合”，无顺序。
- 公式中：
  - 排列：分母只有 $(n-m)!$
  - 组合：分母多了个 $m!$

所以组合数就是在排列数的基础上，再除以 $m!$。

七、动态规划

背包

01 背包

所属分类: 动态规划 › 背包 复杂度: $O(n \cdot C)$ 时间, $O(C)$ 空间

适用问题

- 每个物品选或不选(01), 容量不超过 C 的最大价值
- 对应题目: P1048 采药、P1060 开心的金明

下面把你已经做过的 01 背包经典问题，按**不同变式**整理成可直接抄用的板子。每个板子包含：适用问题、状态定义、初始化、转移、循环、答案、代码模板、对应题目。

基础模型(最大价值)

- 有 `n` 个物品，背包容量 `C`
- 第 `i` 个物品：重量 `w[i]`，价值 `v[i]`
- 每个物品最多选 1 次

所有变式都基于这个模型，区别只在**问法**和**初始化**。

模板

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, C;
    cin >> n >> C;
    vector<int> w(n + 1), v(n + 1);
    for (int i = 1; i <= n; i++) cin >> w[i] >> v[i];

    vector<int> dp(C + 1, 0);
    for (int i = 1; i <= n; i++)
        for (int j = C; j >= w[i]; j--)
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);

    cout << dp[C] << endl;
    return 0;
}
```

变式

恰好装满

- 初始化: `dp` 全置为 `-1e9`(负无穷表示不可达), `dp[0] = 0`(容量 0 恰好装满, 价值 0)
- 转移: 同基础模型(内层倒序): `dp[j] = max(dp[j], dp[j - w[i]] + v[i])`
- 答案: `dp[C]`(若仍为 `-1e9` 说明无法恰好装满)
- 对应题目: 牛客模板 01 背包(恰好装满部分)

可行性

- 初始化: `vector<bool> dp` 全为 `0`, `dp[0] = 1`(容量 0 总是可达)
- 转移: 内层倒序, `dp[j] |= dp[j - w[i]]`(即 `if (dp[j - w[i]]) dp[j] = true`)
- 答案: 能否恰好装满 `C` 看 `dp[C]`; 最大可达容量: 从 `C` 向下找第一个 `dp[j] == 1`
- 对应题目: P1049 装箱问题、LeetCode 416 分割等和子集

方案数

- 初始化: `dp[0] = 1`(容量 0 有 1 种方案: 什么都不选), 其余 `0`
- 转移: 内层倒序, `dp[j] += dp[j - w[i]]`
- 答案: `dp[C]`
- 对应题目: P1164 小 A 点菜

最大价值方案数

- 初始化: `dp` 全为 `0`, `cnt` 全为 `1`(什么都不选, 价值 0, 方案数 1)
- 转移: 内层倒序, 更优则继承、相等则相加:

```
int take = dp[j - w[i]] + v[i];
int not_take = dp[j];
```

```
if (take > not_take) { dp[j] = take; cnt[j] = cnt[j - w[i]]; }
else if (take == not_take) { cnt[j] = (cnt[j] + cnt[j - w[i]]) % MOD; }
```

- 答案: `cnt[C]`
- 对应题目: AcWing 11 / 洛谷 U224067

输出方案

- 初始化: 二维 `dp` 保留每层状态: `vector<vector<int>> dp(N + 2, vector<int>(V + 1, 0))`
- 转移: 从后往前 DP: `dp[i][j] = dp[i + 1][j]`, 选时与 `dp[i + 1][j - v[i]] + w[i]` 取 `max`
- 答案: 从前往后倒推, 能选就选(即字典序最小的方案):

```
int bag = V;
for (int i = 1; i <= N; i++) {
    if (bag >= v[i] && dp[i][bag] == dp[i + 1][bag - v[i]] + w[i]) {
        cout << i << " "; bag -= v[i];
    }
}
```

- 对应题目: AcWing 12 / 洛谷 U224072

易错点

1. **一维 01 背包内层必须倒序**：`j` 从 `C` 到 `w[i]`。
2. **二维转移选物品时用 `dp[i-1]`**，不是 `dp[i]`。
3. **恰好装满**：初始化 `dp[0]=0`，其余 `-inf`。
4. **方案数（恰好）**：初始化 `dp[0]=1`，其余 `0`。
5. **最大价值方案数**：`cnt` 初始全为 `1`。
6. **输出具体方案**：必须二维，从后往前 DP，从前往后倒推。
7. **完全背包内层正序**：`for (int j = w[i]; j <= C; j++)`。

背包变式(完全/多重/分组/混合/多维)

所属分类: 动态规划 › 背包 复杂度: 见各小节 备注: *2026-10-18* 初稿, 待你核对

适用问题

- 01 背包之外的五类常见变形(每题按需取用)

完全背包

每种物品可选无限多次; 内层 `j` **正序**, 使同一物品可被重复选取, 转移仍是 `dp[j] = max(dp[j], dp[j - w[i]] + v[i])`。

- 复杂度: $O(n \cdot C)$ 时间, $O(C)$ 空间
- 对应题目: P1616 疯狂的采药

```
vector<int> dp(C + 1, 0);
for (int i = 1; i <= n; i++)
    for (int j = w[i]; j <= C; j++)              // 正序
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
cout << dp[C] << endl;
```

多重背包

每件物品最多选 `cnt[i]` 件; 用二进制拆分转成 01: 拆成 `1, 2, 4, ...` 与余数若干包, 每包按 01 背包倒序处理。

- 复杂度: $O(\Sigma \log \text{cnt}[i] \cdot C)$ 时间, $O(C)$ 空间
- 对应题目: P2347 砝码称重(可行性版)

```
vector<int> dp(C + 1, 0);
for (int i = 1; i <= n; i++) {
    int k = cnt[i];
    for (int p = 1; p <= k; p <<= 1) {           // 二进制拆分: 1, 2, 4, ...
        for (int j = C; j >= p * w[i]; j--)      // 每包按 01 倒序
            dp[j] = max(dp[j], dp[j - p * w[i]] + p * v[i]);
        k -= p;
    }
    if (k > 0)                                   // 剩余部分单独一包
        for (int j = C; j >= k * w[i]; j--)
            dp[j] = max(dp[j], dp[j - k * w[i]] + k * v[i]);
}
cout << dp[C] << endl;
```

分组背包

每组至多选一件; 循环顺序为 组 → 容量 `j` 倒序 → 组内物品, 保证同组不会被重复选。

- 复杂度: $O(n \cdot C)$ 时间, $O(C)$ 空间(组内物品合计 n 件)
- 对应题目: P1757、P1064(依赖即分组思想)

```
vector<int> dp(C + 1, 0);
for (int g = 1; g <= G; g++)                     // 每组
    for (int j = C; j >= 0; j--)                 // 容量倒序
        for (int t : group[g])                   // 组内物品
            if (j >= w[t])
                dp[j] = max(dp[j], dp[j - w[t]] + v[t]);
cout << dp[C] << endl;
```

混合背包

01 / 完全 / 多重混在同一批物品里, 按类型分派: 完全物品内层正序, 其余(01 与多重)统一按二进制拆分后倒序(约定 `type[i] == 1` 为完全, 其余按 `cnt[i]` 件处理)。

- 复杂度: $O(\Sigma \log \text{cnt}[i] \cdot C)$ 级(拆分后的总包数 × 容量)

```
vector<int> dp(C + 1, 0);
for (int i = 1; i <= n; i++)
    if (type[i] == 1)                            // 完全: 内层正序
```

```
        for (int j = w[i]; j <= C; j++)
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    else {                                          // 01 (cnt=1) / 多重: 拆分后倒序
        int k = cnt[i];
        for (int p = 1; k > 0; p <<= 1) {           // 二进制拆分: 1, 2, 4, ...
            int t = min(p, k);                      // 最后一包可能不足 p
            for (int j = C; j >= t * w[i]; j--)
                dp[j] = max(dp[j], dp[j - t * w[i]] + t * v[i]);
            k -= t;
        }
    }
cout << dp[C] << endl;
```

多维背包(二维费用)

除容量外加一维费用(如体积 + 质量), 多加一层费用循环、两维都倒序即可。

- 复杂度: $O(n \cdot C1 \cdot C2)$ 时间, $O(C1 \cdot C2)$ 空间
- 对应题目: P1507 NASA 的食物计划

```
vector<vector<int>> dp(C1 + 1, vector<int>(C2 + 1, 0));
for (int i = 1; i <= n; i++)
    for (int j = C1; j >= a[i]; j--)            // 费用 1 倒序
        for (int k = C2; k >= b[i]; k--)        // 费用 2 倒序
            dp[j][k] = max(dp[j][k], dp[j - a[i]][k - b[i]] + v[i]);
cout << dp[C1][C2] << endl;
```

树形 DP

树上背包

所属分类: 动态规划 › 树形 DP 复杂度: $O(n \cdot m)$(按 siz 合并)

适用问题

- 树形依赖背包: 选儿子前必须先选父亲; P2014 选课(加虚拟根 0, 输出 `dp[0][M + 1]`)

模板

```cpp
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'
i64 N, M;
vector<i64> course;
vector<vector<i64>> Node;
vector<vector<i64>> dp;
vector<i64> sz;

void dfs(i64 u, i64 p) {
  sz[u] += 1;
  dp[u][1] = course[u];
  for(auto &v:Node[u]) {
    if(v == p) continue;
    dfs(v, u);

    i64 i_max = min(sz[u] + sz[v], M + 1);
    for(i64 i = i_max; i >= 1; i--) { //总背包容量
      i64 j_max = min(sz[v], i - 1);
      for(i64 j = 0; j <= j_max; j++) {
        dp[u][i] = max(dp[u][i], dp[u][i - j] + dp[v][j]);
      }
    }

    sz[u] += sz[v];
  }
}

int main() {
  ios::sync_with_stdio(0);
  cin.tie(0);
  cin >> N >> M;
  course.resize(N + 2);
  Node.resize(N + 2);
  dp.assign(N + 2, vector<i64>(M + 2));
  sz.assign(N + 2, 0);

  for(int i = 1; i <= N; i++) {
    i64 k, s;
    cin >> k >> s;
    course[i] = s;
    Node[i].push_back(k);
    Node[k].push_back(i);
  }

  dfs(0, -1);
```

```
  cout << dp[0][M + 1];
}
```

树形 DP 基础

所属分类: 动态规划 › 树形 DP 复杂度: O(n)(基础选/不选型) 备注: 2026-10-18 初稿, 待你核对

适用问题

- 树上「选 / 不选」决策的最优化问题; 例题: P1352 没有上司的舞会、P2016 战略游戏、P1122 最大子树和

基础模型(选/不选型)

- 状态: `dp[u][0]` / `dp[u][1]` 分别表示「不选 / 选」结点 u 时, 以 u 为根的子树内的最优值
- 转移要点: 约束只发生在父子一条边上, 合并儿子时按 u 选不选分两类:
  - 不选 u: 儿子可选可不选 — `dp[u][0] = Σ max(dp[v][0], dp[v][1])`
  - 选 u: 以没有上司的舞会为例, 上司出席则下属不能出席 — `dp[u][1] = happy[u] + Σ dp[v][0]`
- 合并顺序: 后序 `dfs`, 先递归儿子再拿儿子的 `dp` 更新 u; 无父结点的点即根(代码用 `has_p` 找根), 答案取 `max(dp[root][0], dp[root][1])`

下面给出 P1352 没有上司的舞会的题意: 结点是员工、`happy[u]` 是员工 u 的快乐值, 父子边表示直接上下级关系; 目标是选出互不冲突的点集使快乐值之和最大, 输出该最大值。

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
#define endl '\n'
vector<i64> happy;
vector<vector<i64>> Node;
vector<bool> has_p;
vector<vector<i64>> dp;

void dfs(i64 u, i64 p) {
  dp[u][1] = happy[u];
  for(auto v:Node[u]) {
    if(v == p) continue;
    dfs(v, u);
    dp[u][0] += max(dp[v][1], dp[v][0]);
    dp[u][1] += dp[v][0];
  }

}

void solve() {
  int n;
  cin >> n;
  happy.resize(n + 1);
  Node.resize(n + 1);
  has_p.resize(n + 1, 0);
  dp.resize(n + 1, vector<i64>(2, 0)); //1选该结点 0不选该结点

  for(int i = 1; i <= n; i++) {
    cin >> happy[i];
  }

  for(int i = 1; i <= n - 1; i++) {
    i64 l, k;
    cin >> l >> k;
    Node[l].push_back(k);
    Node[k].push_back(l);
    has_p[l] = 1;
  }

  for(int i = 1; i <= n; i++) {
    if(!has_p[i]) {
```

```
      dfs(i, -1);
      cout << max(dp[i][0], dp[i][1]) << endl;
      break;
    }
  }


}

int main() {
  ios::sync_with_stdio(0);
  cin.tie(0);
  solve();
}
```

变式

同一个按子树合并的框架, 换转移就能套到其它树上最优化问题; 下面片段只保留转移本体(省略 `dfs` 框架、父结点判断与初始化)。

最小点覆盖(P2016 战略游戏)

每条边至少要有一个端点被选中, 求最少选中点数: 不选 u 时每个儿子必须选(否则边 (u, v) 没人覆盖), 选 u 时儿子选不选都行。

- `dp[u][0] = Σ dp[v][1]`
- `dp[u][1] = 1 + Σ min(dp[v][0], dp[v][1])`

```
dp[u][1] += 1;                          // 选 u 自身计 1
for (auto &v : Node[u]) {
    dp[u][0] += dp[v][1];               // 不选 u: 儿子必须选
    dp[u][1] += min(dp[v][0], dp[v][1]); // 选 u: 儿子选不选都行
}
```

最大子树和(P1122)

求带权树上连通块的最大权值和: `dp[u]` 表示强制包含 u、只在 u 子树内取点的最大和; 儿子贡献为负就不取, 等价于只加正的 `dp[v]`。

- `dp[u] = w[u] + Σ max(0, dp[v])`(w 是点权, 代码中为 `beauty[u]`)

```
i64 sum = beauty[u];
for (auto v : Node[u])
    if (dp[v] >= 0) sum += dp[v];       // 等价于 sum += max(0, dp[v])
dp[u] = sum;
```

答案对所有 `dp[i]` 取最大值(最大连通块不一定以根为最高点)。

同理, 子树大小、深度分布一类的计数 DP(如 161D)可设 `dp[u][d]` 表示「u 子树内与 u 距离为 d」的信息, 同样按子树逐个合并。

LIS 与 LCS

所属分类: 动态规划 › 线性 *DP* 复杂度: *LIS* $O(n^2)$ / $O(n \log n)$, *LCS* $O(nm)$ 备注: *2026-10-18* 初稿, 待你核对

适用问题

- LIS(最长上升子序列): 单个序列内的最长上升子序列长度
- LCS(最长公共子序列): 两个序列的最长公共子序列长度(相似度/匹配类)
- 两节均只收录求长度的模板

LIS(最长上升子序列)

n 较小时 $O(n^2)$ DP 直观; n 较大时用贪心 + 二分。

$O(n^2)$ DP(以 i 结尾)

- `dp[i]` 表示以 `a[i]` 结尾的最长上升子序列长度; 答案取所有 `dp[i]` 的最大值
- 转移: 枚举前驱 `j < i` 且 `a[j] < a[i]`, `dp[i] = max(dp[i], dp[j] + 1)`
- 复杂度: $O(n^2)$ 时间, $O(n)$ 空间

```
int LIS_n2(const vector<int> &a) {
    int n = (int)a.size(), ans = 0;
    vector<int> dp(n, 1);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++)
            if (a[j] < a[i]) dp[i] = max(dp[i], dp[j] + 1);
        ans = max(ans, dp[i]);
    }
    return ans;
}
```

$O(n \log n)$(贪心 + 二分)

- 维护 `tails[k]`: 所有长度为 `k + 1` 的上升子序列中, 最小的结尾值(它单调不减)
- 每个 `a[i]` 在 `tails` 里二分找第一个可替换的位置: 接在末尾或替换; `tails` 的长度即答案
- 复杂度: $O(n \log n)$ 时间, $O(n)$ 空间

```
int LIS_nlogn(const vector<int> &a) {
    vector<int> tails;                                  // tails[k]: 长度 k+1 的最小结尾
    for (int x : a) {
        auto it = lower_bound(tails.begin(), tails.end(), x); // 严格上升: 第一个 >= x
        if (it == tails.end()) tails.push_back(x);
        else *it = x;
    }
    return tails.size();
}
```

一句话区分: 求**严格上升**用 `lower_bound`(找第一个 `>= x`, 相等会顶掉); 求**非严格上升**(不下降)改用 `upper_bound`(找第一个 `> x`, 相等保留), 两段只差这一处($O(n^2)$ 版对应把 `a[j] < a[i]` 改成 `a[j] <= a[i]`)。

- 对应题目: P1020 导弹拦截(第一问不上升用 `upper_bound`、第二问上升用 `lower_bound`)、LeetCode 300 最长递增子序列
- 注意: 求最长**不升**子序列时 `tails` 是降序, 需要 `upper_bound(tails.begin(), tails.end(), x, greater<int>())`。

LCS(最长公共子序列)

- `dp[i][j]` 表示 `a` 的前 `i` 个与 `b` 的前 `j` 个的最长公共子序列长度
- 转移: `a[i - 1] == b[j - 1]` 时 `dp[i][j] = dp[i - 1][j - 1] + 1`(该字符直接接上); 否则 `dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])`(两个末尾字符配不上, 至少丢掉一个)
- 复杂度: $O(nm)$ 时间, $O(nm)$ 空间

```
int LCS(const string &a, const string &b) {
    int n = (int)a.size(), m = (int)b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
```

```
            if (a[i - 1] == b[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
            else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
    return dp[n][m];
}
```

提点: 字母表小、串长很大时可用 bitset 把 $O(nm)$ 压位优化(约 $O(nm/64)$), 本板子不展开。

- 对应题目: AcWing 897 最长公共子序列、LeetCode 1143 最长公共子序列

八、工具速查

GCC 内置函数

所属分类: 工具速查 用途: 比赛速查(不要求记忆, 用时查表) 备注: *2026-10-18* 初稿, 待你核对

速查表

__builtin_* 是 GCC/Clang 的编译器内建函数, 不用 include 就能用; __lg 来自 libstdc++, 有 <bits/stdc++.h>(配合 using namespace std;)即可。**popcount/clz/ctz/parity/ffs 等返回 int; __lg 的返回类型随参数类型(int/long/long long)。**

函数	参数	返回	说明
__builtin_popcount(x)	unsigned int x	int	x 的二进制中 1 的个数
__builtin_popcountl(x)	unsigned long x	int	同上, unsigned long 版
__builtin_popcountll(x)	unsigned long long x	int	同上, unsigned long long 版 (64 位常用这个)
__builtin_clz(x)	unsigned int x	int	高位起连续 0 的个数; **x = 0 时是 UB**
__builtin_clzl(x) / __builtin_clzll(x)	unsigned long / unsigned long long	int	同上(64 位用 clzll)
__builtin_ctz(x)	unsigned int x	int	低位起连续 0 的个数(即最低位 1 的下标, 0-based); **x = 0 时是 UB**
__builtin_ctzl(x) / __builtin_ctzll(x)	unsigned long / unsigned long long	int	同上(64 位用 ctzll)
__builtin_parity(x)	unsigned int x	int	x 中 1 的个数奇偶: 奇数 → 1, 偶数 → 0
__builtin_parityl(x) / __builtin_parityll(x)	unsigned long / unsigned long long	int	同上(64 位用 parityll)
__builtin_ffs(x)	int x	int	最低位 1 的位置, **1-based**; x = 0 → 返回 0
__builtin_ffsl(x) / __builtin_ffsll(x)	long / long long	int	同上(64 位用 ffsll)
__lg(x)	正整数(int / long long / unsigned 均可)	同参数类型	⌊log2 x⌋ = 最高位 1 的下标; x = 0 时是 UB; libstdc++ 内部函数(非标准)

常用组合

```
int pc  = __builtin_popcountll(x);   // x 中 1 的个数
int hi  = 63 - __builtin_clzll(x);   // x > 0: 最高位 1 的下标 = ⌊log2 x⌋
int lo  = __builtin_ctzll(x);        // x > 0: 最低位 1 的下标(0-based)
int f   = __builtin_ffsll(x);        // x ≠ 0: 最低位 1 的下标 + 1(1-based)
int odd = __builtin_parityll(x);     // x 中 1 的个数为奇数 → 1
int l2  = __lg(x);                   // 同 hi, 写法更短
```

求最高位下标的两种写法: 32 位数 31 - __builtin_clz(x), 64 位数 63 - __builtin_clzll(x)。

坑

- **64 位必须用 ll 后缀**: __builtin_popcount(1LL << 40) 会先把参数截断成 unsigned int(高 32 位丢掉), 结果是 0, 编译器只给个 -Woverflow 警告; 应写 __builtin_popcountll(1LL << 40)。同理 64 位用 clzll / ctzll / ffsll / parityll。

- **Windows(MinGW) 下 `long` 是 32 位**(`__builtin_popcountl(~0UL)` 得 32), Linux 评测机下 `long` 是 64 位; 所以 64 位的数一律用 `ll` 版最保险。
- **`clz` / `ctz` / `__lg` 在 0 上不可用**: 参数为 0 时行为未定义, 用前先判 `x != 0`(负数对这几个函数也无意义)。
- **返回类型是 `int`(不是 `long long`)**: 结果不超过 64, 本身不会溢出; 但和别的大数相乘时注意先转, 如 `1LL * __builtin_popcountll(x) * y`。
- **`ffs` 与 `ctz` 的下标基准不同**: `ffs` 是 1-based(最低位为 1), `ctz` 是 0-based(最低位为 0), 混用时差 1。
- **C++20 起有标准版可平替**: `std::popcount` / `std::countl_zero` / `std::countr_zero` / `std::bit_width` / `std::has_single_bit`(都在 `<bit>` 头); 注意它们要求 C++20 且参数必须是**无符号**类型(传 `int` 编译不过)。本板子按 C++17, 统一用 `__builtin_*`。

字符串操作

所属分类: 工具速查 用途: 比赛速查(不要求记忆, 用时查表) 备注: 2026-10-18 初稿, 待你核对

速查表

函数	签名 / 用法	返回	说明
stoi	`int stoi(const string& s, size_t* pos = nullptr, int base = 10)`	int	字符串 → int; 无法转换抛 `std::invalid_argument`, 超 int 范围抛 `std::out_of_range`
stoll	`long long stoll(const string& s, size_t* pos = nullptr, int base = 10)`	long long	同上; 数字可能超 int 时用
stod	`double stod(const string& s, size_t* pos = nullptr)`	double	字符串 → double(没有 base 参数)
to_string	`string to_string(数值)`	string	数值 → 字符串(int / long long / double 等都有重载)
substr	`string s.substr(size_t pos = 0, size_t len = npos)`	string	从 pos 起取 **len 个字符**(第 2 参是长度, 不是结束下标); pos > size() 抛 `out_of_range`
find	`size_t s.find(子串或字符, size_t pos = 0)`	size_t	从 pos 起第一个出现位置; 找不到返回 `string::npos`
rfind	`size_t s.rfind(子串或字符, size_t pos = npos)`	size_t	从 pos 往前找最后一个出现位置; 找不到 npos
getline	`istream& getline(istream& in, string& s, char delim = '\n')`	istream&	读一行(能含空格), 丢弃行尾的分隔符
isdigit	`int isdigit(int c)`	int	是否 '0'~'9'; 非 0 表示是
toupper	`int toupper(int c)`	int	小写字母 → 大写, 其它字符原样; 转回 char 用 `(char)toupper(c)`
tolower	`int tolower(int c)`	int	大写字母 → 小写, 其它字符原样

补充: `s.size()` / `s.length()` 取字符数, `s.empty()` 判空, `s += t` 拼接, `s.push_back(c)` / `s.pop_back()`, `s.front()` / `s.back()`; 反转与排序用 `reverse(s.begin(), s.end())` / `sort(s.begin(), s.end())`。

常用写法

读一行(getline)与 cin >> 混用

```
int n;
string s;
cin >> n;
cin.ignore();          // 必须先扔掉数字后留下的换行符
getline(cin, s);       // 之后才能读到正确的一行
// 更保险的写法(丢弃到行尾): cin.ignore(numeric_limits<streamsize>::max(), '\n');  // 需 <limits>
```

按行读到 EOF: 直接写 `while (getline(cin, s))` 作循环条件。

stringstream 切词

按空白(空格 / 换行)切:

```
stringstream ss("I love ACM");
string w;
while (ss >> w) cout << w << '\n';   // I / love / ACM 各一行
```

按自定义分隔符切(第 3 参):

```
stringstream ss("1,2,3");
string t;
while (getline(ss, t, ',')) cout << t << '\n';   // 1 / 2 / 3 各一行
```

需 `#include <sstream>`; 同一个 stringstream 复用前先 `ss.clear(); ss.str(新串);`。

stoi 用 pos 检查整串

```
string s = "123abc";
size_t pos;
int x = stoi(s, &pos);   // x = 123, pos = 3(第一个没转换的字符下标)
if (pos == s.size()) { /* 整串都是数字 */ }
```

找所有出现位置

```
string s = "abababab", p = "ab";
for (size_t f = s.find(p); f != string::npos; f = s.find(p, f + 1))
    cout << f << ' ';    // 0 2 4
```

坑

- **`string::npos` 是无符号数**(`size_t` 的 -1): 判"没找到"只能写 `!= string::npos`(写 `>= 0` 恒真); 也别用 `int` 接 find 的返回值。
- **`cin >> n` 后直接 `getline` 会读到空行**: `>>` 把行尾 '\n' 留在了缓冲区, 先 `cin.ignore()`。
- **`stoi` 越界 / 转换失败会抛异常, 不捕获直接 RE**: 超范围抛 `std::out_of_range`(如 11 位以上的数), 完全不是数字抛 `std::invalid_argument`; 数据超 int 一律改用 `stoll`。
- **`stoi("12abc")` 不报错**, 返回 12(pos 停在 2); 要判断"整串是数字"必须用 pos。
- **`substr(i, len)` 第 2 参是长度**: `s.substr(2, 3)` 取 s[2] s[3] s[4]; 长度写超了不报错, 取到串尾为止。
- **`to_string(1.5) == "1.500000"`**: 浮点固定 6 位小数; 输出小数一般直接 `cout << x`。
- **`isdigit` / `toupper` 的参数要转 `unsigned char`**: 参数是负的 char(非 ASCII 字节)时行为未定义, 写 `toupper((unsigned char)c)`; 返回值是 int, 转回 char。
- **`s.size()` 是无符号的**: `for (int i = 0; i < s.size(); i++)` 会触发 signed/unsigned 比较警告, 习惯写 `for (int i = 0; i < (int)s.size(); i++)`。

容器操作

所属分类: 工具速查 用途: 比赛速查(不要求记忆, 用时查表) 备注: 2026-10-18 初稿, 待你核对

priority_queue(堆)

```
priority_queue<int> pq;                                    // 默认大根堆: top() 是最大值
priority_queue<int, vector<int>, greater<int>> pq2;        // 小根堆: top() 是最小值
// pair 按 (first, second) 字典序, top() 最大
priority_queue<pair<int,int>> pq3;
priority_queue<pair<int,int>, vector<pair<int,int>>,
               greater<pair<int,int>>> pq4;                // 小根堆
```

- push(x) 入堆, top() 堆顶, pop() 弹堆顶, empty() / size(); 每次 O(log n)。
- 记法: **less 大根堆, greater 小根堆**(和 sort 的升序 / 降序直觉相反, 别记反)。
- priority_queue **不能遍历**(没有迭代器), 要取全部元素只能一个个 pop; 需要遍历就换 set / multiset。

自定义比较器(逐字收录 板子/优先队列结构体重载 cmp.md):

```
struct cmp {
    bool operator()(const task& a, const task& b) {
        return a.priority > b.priority;
    }
};
```

用法与语义:

```
struct task { int priority; };
priority_queue<task, vector<task>, cmp> pq;   // top() 是 priority 最小的 task
```

- cmp(a, b) == true 表示 a 输出得比 b 晚(优先级更低); 堆顶是**最先弹出**(pop)的元素。
- 片段里 a.priority > b.priority ⇒ priority 大的排后面 ⇒ **top() 是 priority 最小的**(小根堆)。
- 想改成大根堆: 把 > 换成 <。
- lambda 版(不能直接当模板实参, 要 decltype 且构造时传进去):

```
auto cmp2 = [](int a, int b) { return a > b; };
priority_queue<int, vector<int>, decltype(cmp2)> pq5(cmp2);   // 小根堆
```

set / map(有序)

- set<int> 有序去重; multiset<int> 有序可重复; map<K, V> 按 key 排序; 查找 / 插入 / 删除均 O(log n)。
- 成员二分(是 O(log n), 比泛型 std::lower_bound 快):
  - s.lower_bound(x) → 第一个 ≥ x 的迭代器; 没有则 s.end()。
  - s.upper_bound(x) → 第一个 > x 的迭代器。
  - 前驱(≤ x 的最大值): auto it = s.upper_bound(x); if (it != s.begin()) --it; 之后 *it 就是前驱。
  - 存在性: s.count(x)(set / map 是 0/1)或 s.find(x) != s.end()。
- **map 判存在别用 mp[x]**: 会把 x 插进去(默认值); 用 count / find。
- *s.begin() 是最小值, *s.rbegin() 是最大值。
- erase 的三种用法:
  - s.erase(x) 按值删, 返回删掉的个数; **multiset 会把等于 x 的全部删掉** — 只删一个用 ms.erase(ms.find(x))(用前先确认 x 存在)。
  - s.erase(it) 按迭代器删, **返回下一个迭代器**(C++11 起)。
  - s.erase(first, last) 删区间。
- 边遍历边删的安全写法(以删掉所有偶数为例):

```
set<int> s{1, 2, 3, 4, 5};
for (auto it = s.begin(); it != s.end(); )
    if (*it % 2 == 0) it = s.erase(it);
    else ++it;
```

- 遍历 map(C++17):

```
map<string, int> mp;
for (auto& [k, v] : mp)
    cout << k << ' ' << v << '\n';    // 按 key 升序输出
```

- ⚠️ 不要对 set / map 写 `lower_bound(s.begin(), s.end(), x)`: 泛型版在非随机访问迭代器上只能逐个前进, 是 **O(n)**; 必须用成员函数。

unordered_map(防 hack)

- `unordered_map` 平均 O(1), 但默认 hash 固定, 出题人可以构造同 hash 的数据卡到 O(n)(CF 上常有); 想稳就加自定义 hash。
- 自定义 hash(64 位 splitmix64 + 运行期随机种子):

```
struct chash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};

unordered_map<long long, int, chash> mp;    // 键用 long long(或 int)
```

- `pair` 键没有默认 hash, 必须自己写:

```
struct phash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(const pair<int, int>& p) const {
        static const uint64_t FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        uint64_t h = ((uint64_t)(uint32_t)p.first << 32) ^ (uint32_t)p.second;
        return splitmix64(h + FIXED_RANDOM);
    }
};

unordered_map<pair<int, int>, int, phash> mp2;
```

- 大致知道元素个数时先 `mp.reserve(n)`, 减少 rehash。
- 不想折腾就直接用 `map`(稳定 O(log n))。

vector(排序 / 去重 / 二分)

```
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());   // 去重(先排序, 再删尾)
```

- `unique` 只把相邻重复项挪到后面(不真删元素), 返回新的逻辑结尾; 所以**必须先排序**再配 `erase` 真删, 整体 O(n log n)。
- 有序 vector 二分: `lower_bound(v.begin(), v.end(), x)`(第一个 ≥ x)/ `upper_bound`(第一个 > x), O(log n); 判存在: `auto it = lower_bound(...); it != v.end() && *it == x`。
- 删单个: `v.erase(v.begin() + i)`, O(n)(后面元素整体前移)。

迭代器失效

- `vector`: `push_back` / `insert` 若扩容, 所有迭代器、指针、引用失效(不扩容时插入点之后失效); `erase(it)` 使 it 及之后全部失效 → 用返回值接着走: `it = v.erase(it);`。
- `map` / `set`: 红黑树, `insert` 不影响已有迭代器; `erase(it)` 只失效被删的那个并返回下一个 → 上面"边遍历边删"的写法安全。

- `unordered_map`: rehash(扩容)时所有迭代器失效(指向元素的指针、引用不失效); `erase(it)` 同样返回下一个。
- 通用习惯: 遍历途中要删元素, 用 `it = c.erase(it)` 的 while 循环推进; 别在 range-for 里增删。

pair / tuple

```
pair<int, int> p{1, 2};    // p.first, p.second; 比较按字典序(先 first 再 second)
auto q = make_pair(3, 4);

int a, b;
tie(a, b) = p;             // 解包: a = 1, b = 2
tie(a, ignore) = p;        // 只取第一个(ignore 在 std 里)

tuple<int, int, int> t{1, 2, 3};
get<0>(t);                 // 取第 0 个(get<1> / get<2>; 下标必须是编译期常量)
auto [x, y, z] = t;        // C++17 结构化绑定
```

- `pair` 排序 = 先按 first 再按 second(map 的 key 也是这个顺序)。
- 几个值一起比较: `tie(a, b) < tie(c, d)`(按字典序), 免手写多段 if。

bitset

所属分类: 工具速查 用途: 比赛速查(不要求记忆, 用时查表) 备注: 2026-10-18 初稿, 待你核对

声明

```
bitset<100005> b;              // N 必须是编译期常量(模板参数); 内存约 N/8 字节
const int N = 100003;
bitset<N> f;                   // 可以: const int 由常量表达式初始化
// int n; cin >> n;  bitset<n> g;   // 不行: n 不是编译期常量
bitset<8> h(string("1010"));   // 从字符串构造: 右端是低位(与 to_string 方向一致), 字符串短了高位补 0
```

≤ 64 位的需求直接 `unsigned long long` 当位掩码就行(还能配 `__builtin_popcountll`), bitset 用在 N 更大或需要大量整段位运算的场合。

位运算

```
bitset<8> a(5);              // 00000101
bitset<8> b(string("1010")); // 00001010
a |= b;                      // 00001111
cout << (a << 1) << '\n';    // 00011110(移出界的位直接丢)
cout << ~a << '\n';          // 11110000(对全部 N 位取反)
a[0] = 1;                    // 第 0 位(最低位)可读可写
```

- 支持 `&` `|` `^` `~` `<<` `>>` 与 `==` / `!=`; 其中 `&` `|` `^` 两边必须同类型 / 同 N, `<<` / `>>` 的右操作数是整型移位数(如 `a << 1`), 不是 bitset。
- 输出方向: `cout << b` 与 `b.to_string()` 一致, **最左是最高位**(下标 N-1); 从字符串构造也是同一方向, 方向不用特别记。

成员函数速查表

成员	例子	返回 / 说明
`test(i)`	`b.test(3)`	第 i 位是否为 1; **i ≥ N 抛 `out_of_range`**
`operator[]`	`b[i] = 1`	访问第 i 位(可读可写); **不做越界检查**(越界是 UB)
`set(i)` / `set()`	`b.set(2)` / `b.set()`	第 i 位置 1 / 全 1; 越界抛 `out_of_range`
`set(i, v)`	`b.set(2, false)`	第 i 位设为 v
`reset(i)` / `reset()`	`b.reset(2)` / `b.reset()`	第 i 位置 0 / 全 0
`flip(i)` / `flip()`	`b.flip(2)` / `b.flip()`	第 i 位取反 / 全取反
`count()`	`b.count()`	1 的个数(约 O(N/64))
`size()`	`b.size()`	N(编译期常量)
`any()` / `none()`	`b.any()` / `b.none()`	是否存在 1 / 是否全 0
`all()`	`b.all()`	是否全 1(C++11 起)
`to_string()`	`b.to_string()`	转 `string`: N 个 '0'/'1', 最左是最高位
`to_ulong()` / `to_ullong()`	`b.to_ullong()`	转 `unsigned long` / `unsigned long long`(C++11); **值超范围抛 `overflow_error`**

坑

- **N 必须是编译期常量**: `int n; cin >> n; bitset<n> b;` 编译不过; 规模不定时用 `vector<bool>` 或按题目上界开固定 N。
- **`to_ullong()` 溢出会抛异常**: `bitset<64>` 全 1 没问题(= 2^64-1), 但 N > 64 时只要有下标 ≥ 64 的位是 1(值 ≥ 2^64)就抛 `std::overflow_error`; 不要拿它直接转大 bitset。
- **`operator[]` 不查越界**; `test(i)` / `set(i)` / `reset(i)` / `flip(i)` 越界会抛 `out_of_range`。
- **`to_string()` 是 O(N)**(大 bitset 别在循环里反复调用)。
- 字符串构造时含非 '0'/'1' 字符会抛 `std::invalid_argument`。

典型用途

可达性 / 传递闭包

用 `bitset<N>` 数组当邻接矩阵, 一次 `|=` 处理一整行, 传递闭包 $O(n^3/64)$:

```
const int N = 2005;
bitset<N> g[N];              // g[i] 的第 j 位 = i 能一步到 j
// 加边: g[u].set(v);
for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
        if (g[i].test(k)) g[i] |= g[k];
// 之后 g[i].test(j) 即 i 能否到达 j
```

集合表示 / 状态压缩

一个 bitset 就是一个集合: `|=` 并, `&` 交, `& ~` 差, `count()` 大小, 判包含 `(A & B) == B`; 26 个字母之类的"出现过哪些"用它很顺手:

```
bitset<26> has;
for (char c : s) has.set(c - 'a');
int kinds = has.count();     // 出现了多少种字母
```

背包可行性 / 子集和(0/1)

`f` 的第 s 位表示"和为 s 可达", 每个数用一次:

```
bitset<MAXS> f;
f[0] = 1;
for (int i = 1; i <= n; i++) f |= f << a[i];
// 询问: f.test(s)
```

单次转移 O(MAXS/64), 比朴素 DP 快约 64 倍; MAXS 按总和上界开。

随机数

所属分类: 工具速查 用途: 比赛速查(不要求记忆, 用时查表) 备注: 2026-10-18 初稿, 待你核对

基本用法

逐字收录 板子/获取随机数.md:

```
#include <random>
#include <chrono>

// 用高精度时钟做种子, 避免被 Hack
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

// 生成 [1, n] 的随机整数
int rand_int(int l, int r) {
    uniform_int_distribution<int> dist(l, r);
    return dist(rng);
}

// 使用
int idx = rand_int(1, n);
```

- mt19937 是 32 位梅森旋转, 周期 2^19937-1, 比赛够用; 头文件 <random>。
- 种子用 steady_clock 的时钟计数(纳秒级), 每次运行都不同, 没法被提前预判 ⇒ 不容易被 Hack。
- uniform_int_distribution<int> dist(l, r) 均匀生成 [l, r] 的整数(两端都取得到), 构造开销很小, 写在函数里每次现建也没问题。
- 想省事就直接两行:

```
uniform_int_distribution<int> dist(1, n);
int x = dist(rng);
```

64 位: mt19937_64

值域要到 long long 时换 mt19937_64, 用法一模一样:

```
mt19937_64 rng64(chrono::steady_clock::now().time_since_epoch().count());

long long rand_ll(long long l, long long r) {
    uniform_int_distribution<long long> dist(l, r);
    return dist(rng64);
}
```

- 一步到 64 位随机数; 拿两个 32 位结果拼也行, 但没必要。
- 别拿 uniform_int_distribution<int> 装 64 位范围: 它的参数是 int, 传 long long 会被窄化(范围悄悄变小), 64 位就配 long long 模板参数。

洗牌: shuffle

```
shuffle(v.begin(), v.end(), rng);    // 需要 <algorithm> 和 <random>
```

- 随机排列一个 vector / 数组, 一行搞定。
- 别写 random_shuffle: C++14 起弃用, C++17 起被移除/弃用, 部分标准库版本仍可编译但只给警告, 不要依赖; 用 shuffle。
- 别给 sort 传随机比较器: 违反严格弱序, 行为未定义; 要打乱就 shuffle。

坑

- **别用 rand() / srand()**:
  - RAND_MAX 两系统不一样: Windows 上(MSVC 以及 MinGW 用的 msvcrt, 本机 MinGW 15.2 实测)**只有 32767**, 想生成 [1, 1e9] 这种范围根本不够; Linux(glibc)是 **2147483647**, 够 1e9, 但实现仍不可移植、质量也差 —— 结论一样: 别用。
  - 质量和周期都远不如 mt19937, 而且各平台实现不同, 同一份代码换评测机行为就变。
  - rand() % n 只在 n 整除 RAND_MAX+1 时才均匀, 否则有偏。
  - 结论: 模板开头固定写上 mt19937 那三行, 别省这点事。

- **固定种子会被 Hack**: `mt19937 rng(114514)` / `srand(1)` 写死, 出题人按同样种子造卡数据(卡哈希 / 卡快排)就完了; 用时钟种子。
- **反过来, 对拍 / 调试要可复现**: 那种场合改成从命令行拿种子(`gen.exe <seed>` / `./gen <seed>`), 别用时钟, 见 `对拍与打表.md`。
- `rng()` 直接 `% n` 也有偏(mt19937 值域 2^32 不整除 n 时), 用 `uniform_int_distribution` 就干净了。
- 别用 `default_random_engine`: 标准没规定它是哪个发生器(常见实现是 `minstd_rand0`, 质量差), 直接写 `mt19937` / `mt19937_64`。

常见用途

- 对拍造数据: `gen.cpp` 的随机数据; 见 `对拍与打表.md`。
- 防 Hack: `unordered_map` 自定义哈希里混一个运行期随机数(见 `容器操作.md` 的 unordered_map 一节)。
- 随机化算法: 随机采样(交互题, 如 CF843B)、随机化构造、爬山 / 模拟退火。
- 随机排列 / 随机树 / 随机图: `shuffle` 配随机连边。

常用代码片段

所属分类: 工具速查 用途: 比赛速查(不要求记忆, 用时查表) 备注: *2026-10-18* 初稿, 待你核对
取前两大(扫描维护最大的两个值)
逐字收录 板子/取前两大.md:

```
if (best[v] > mx1) {
            mx2 = mx1;
            mx1 = best[v];
        } else if (best[v] > mx2) {
            mx2 = best[v];
        }
```

用途:

- 扫描一串数, 维护"最大的两个值"(mx1 ≥ mx2), 每个值 $O(1)$ 更新, 不用排序、不用堆。
- 树上合并子树: 每个子节点 v 带一个值 best[v] 上来, 在父节点处取它们中最大的两个。
- 典型题: **经过某个点的两条最长链** —— 把子树最深链长当 best, 两条最长链拼起来就是经过该点的最长路径(树直径的 dp 写法)。
- 变体: 求第二大 / 次大值、维护 top-2 分数、需要"次优解"的贪心。

放进上下文(树直径)长这样:

```
// best[u] = u 向下的最深链长(以边数计); 片段里叫 best[v], 这里 v 是父节点, 换名而已
void dfs(int v, int p) {
    int mx1 = 0, mx2 = 0;            // 空臂按 0 算: 叶子/单臂时 mx2 保持 0
    for (int u : adj[v]) {
        if (u == p) continue;
        dfs(u, v);
        best[u] += 1;                // u 的链接上 v-u 这条边
        if (best[u] > mx1) {         // 片段原样用在这里
            mx2 = mx1;
            mx1 = best[u];
        } else if (best[u] > mx2) {
            mx2 = best[u];
        }
    }
    ans = max(ans, mx1 + mx2);       // 经过 v 的两条最长链拼起来; 单臂时 mx2 = 0, 正好是这一条链
    best[v] = mx1;                   // v 自己的向下最深链(叶子为 0)
}
```

注意:

- **必须是 else if**: 换成两个独立的 if, 第一个分支刚做完(mx2 = mx1, mx1 = best[v]), 第二个 if 会拿同一个值再比一次 —— 此时 best[v] > mx2 必然成立, mx2 被刷成同一个值, 变成 mx1 == mx2, 同一个元素被当两个用。
- 初值按题意设: 树直径这种"允许空臂"的写法用 mx1 = mx2 = 0(叶子/单臂时 mx2 保持 0, mx1 + mx2 正好是这一条链); 严格要求两个值都真有来源(如取最大两个数)才用 -INF。
- 判断用严格 >: 碰到与 mx1 相等的元素会掉进 else if 记进 mx2 ⟹ 两个不同的元素(哪怕相等)都能正确维护, 不用特判。

对拍与打表

所属分类: 工具速查 用途: 比赛速查(不要求记忆, 用时查表) 备注: 2026-10-18 初稿, 待你核对

对拍

拿"正解(std)"和"暴力(brute)"在随机小数据上对答案。三件套: 一个 gen 造数据, 三个程序都编译好放在同一目录(Windows 下是 .exe, Linux / macOS 下没有后缀)。

Windows(cmd / PowerShell):

```
g++ -O2 -std=c++17 gen.cpp -o gen.exe
g++ -O2 -std=c++17 std.cpp -o std.exe
g++ -O2 -std=c++17 brute.cpp -o brute.exe
```

Linux / macOS(bash / zsh):

```
g++ -O2 -std=c++17 gen.cpp -o gen
g++ -O2 -std=c++17 std.cpp -o std
g++ -O2 -std=c++17 brute.cpp -o brute
```

- gen.cpp: 造一组随机数据输出到 stdout。**种子从命令行拿**(argv[1]), 这样出 WA 后能精确复现那一组; 没给种子再用 time(0)。
- std.cpp: 你的正解 / 模板。
- brute.cpp: 暴力做法, 慢但显然对。**别和 std 共用同一个函数**: 两边一起错的部分, 对拍永远看不出来。
- brute 也要开 -O2: 数据规模先开小, 保证暴力跑得动。

gen.cpp 的关键就两点: 种子从命令行拿、数据写到 stdout。例(区间和查询的造数据):

```
// gen: 造数据, 种子从 argv[1] 取(便于复现); 题目: 区间和查询
#include <bits/stdc++.h>
using namespace std;
int main(int argc, char** argv) {
    unsigned seed = argc > 1 ? (unsigned)stoul(argv[1]) : (unsigned)time(0);
    mt19937 rng(seed);
    int n = uniform_int_distribution<int>(1, 200)(rng);
    int q = uniform_int_distribution<int>(1, 200)(rng);
    printf("%d %d\n", n, q);
    for (int i = 1; i <= n; i++)
        printf("%d%c", uniform_int_distribution<int>(0, 1000000000)(rng), " \n"[i == n]);
    for (int i = 1; i <= q; i++) {
        int l = uniform_int_distribution<int>(1, n)(rng);
        int r = uniform_int_distribution<int>(1, n)(rng);
        if (l > r) swap(l, r);
        printf("%d %d\n", l, r);
    }
    return 0;
}
```

std.cpp / brute.cpp 就是普通程序: 从 stdin 读、往 stdout 写, 数据由脚本用 < in.txt 重定向喂进去。

循环脚本(Windows bat)

存成 judge.bat, 和三个 exe 放同一个目录, 双击或 cmd 里跑:

```
@echo off
rem ===== stress test: gen + std + brute, compare with fc =====
setlocal
rem ---- config ----
set GEN=gen.exe
set STD=std.exe
set BRUTE=brute.exe
set ROUNDS=100
rem ----------------
for /L %%i in (1,1,%ROUNDS%) do (
```

```
    %GEN% %%i > in.txt
    if errorlevel 1 (
        set "BAD=%%i"
        goto :genfail
    )
    %STD% < in.txt > out1.txt
    %BRUTE% < in.txt > out2.txt
    fc out1.txt out2.txt > nul
    if errorlevel 1 (
        set "BAD=%%i"
        goto :wa
    )
)
echo All %ROUNDS% tests passed
exit /b 0

:genfail
echo gen failed on test %BAD%
exit /b 2

:wa
echo WA on test %BAD%
echo ---- data in in.txt, outputs in out1.txt / out2.txt ----
fc out1.txt out2.txt
exit /b 1
```

- 次数就是 ROUNDS(默认 100): 第一次先跑 100 组确认脚本本身没毛病, 再改大到几千。
- gen 缺失 / 崩溃时立即 goto :genfail, 输出 gen failed on test <轮次> 并返回 2, 避免空输入假通过。
- 出 WA 时现场都留着: in.txt(那组数据)、out1.txt(std 输出)、out2.txt(brute 输出); 想单独复现同一组就 gen.exe 7(Windows)/ ./gen 7(Linux / macOS)。
- 换被测程序只改一行 set STD=...(bash 里改脚本中的 ./std)。
- **bat 里别写中文**: cmd 按系统 ANSI 码页读 bat, 中文注释 / echo 会乱码, 所以上面脚本和输出全用英文。
- 脚本全过返回 0、WA 返回 1、gen 缺失 / 崩溃返回 2, 方便被别的工具调用; 注意 **exit /b 1 直接写在 for / if 括号块里返回码不生效**(cmd 的坑, 实测变 0), 所以这里是 goto :wa / goto :genfail 跳到末尾再退。
- Linux / macOS 用下面的 judge.sh, 和三个可执行文件放同一目录。

循环脚本(Linux bash)

存成 judge.sh, 和三个可执行文件放同一个目录, 先 chmod +x judge.sh, 再 ./judge.sh(或 bash judge.sh): 脚本必须以 LF 保存: Windows 编辑器另存成 CRLF 会让 shebang 变成 bash\r 而报错; 已中招可 sed -i 's/\r$//' judge.sh。

```
#!/usr/bin/env bash
# ===== stress test: gen + std + brute, compare with diff =====
# ---- config ----
ROUNDS=100
# ----------------
for i in $(seq 1 "$ROUNDS"); do
    ./gen "$i" > in.txt || { echo "gen failed on test $i"; exit 2; }
    ./std < in.txt > out1.txt
    ./brute < in.txt > out2.txt
    diff -q out1.txt out2.txt > /dev/null || {
        echo "WA on test $i"
        echo "---- data in in.txt, outputs in out1.txt / out2.txt ----"
        diff out1.txt out2.txt
        exit 1
    }
done
```

```
echo "All $ROUNDS tests passed"
exit 0
```

- gen 缺失 / 崩溃时立即输出 gen failed on test <轮次> 并 exit 2, 不会出现 std / brute 都读空输入、输出同为空文件而假通过。
- diff -q 的退出码: 0 = 相同, 1 = 有差异, 2 = 出错(比如文件打不开)。上面判"非 0"就同时接住了"答案不同"和"std 崩了没输出"两种情况, 和 bat 的 if errorlevel 1 等价。
- 忽略空白用 diff -w(对应 fc /W); 要逐字节完全相同用 cmp -s out1.txt out2.txt(静默, 退出码 0 = 相同 / 1 = 不同 / 2 = 出错)。
- Linux 可以用 timeout 2 ./std < in.txt > out1.txt 限制单轮时间(超时返回 124)。注意 timeout 只负责杀进程: 如果 std 已经输出完整正确结果后才挂住, diff 仍可能判相同 ⇒ 想严谨就显式判 $? -eq 124(或者只把 timeout 当手工排障手段); Windows 没有内建等价命令, 只能小数据 + Ctrl+C。

要点

- **限制次数**: bat 的 for /L %%i in (1,1,%ROUNDS%) / bash 的 seq 1 "$ROUNDS" 上界就是轮数, 从小往大加; std 死循环时 bat 会干等(Ctrl+C 停), Linux 可以用 timeout 2 ./std < in.txt > out1.txt 切断(超时返回 124), 但若程序已经输出完整正确结果后才挂住, diff 仍可能判相同, 要严谨就显式判 $? -eq 124; 先用小数据把这类问题清掉。
- **随机种子**: 每轮把轮次号传给 gen 当种子 ⇒ 每轮数据不同, 又能精确复现; 只靠 time(0) 做种会丢掉复现能力。
- **文本对比**: Windows 的 fc 逐字符比文本, 行尾 / 行中多一个空格的差别都算"不同"(实测: 行尾多一个空格返回 1, 加 /W 压缩空白后返回 0)。返回码 0 = 相同, 1 = 有差异, 2 = 文件打不开 ⇒ 脚本判 if errorlevel 1(大于等于 1)能同时接住"答案不同"和"std 崩了没输出"两种情况。Linux 的 diff -q 同样: 0 = 相同, 1 = 有差异, 2 = 出错, 判非 0 两种都接; 忽略空白用 diff -w, 逐字节用 cmp -s。
- 想忽略空白的 token 级比较: 自己写个小 judge 程序, Windows 用 fc /W、Linux 用 diff -w。
- gen 里掺边界: n 最小 / 最大、全部相同、已排序 / 逆序、询问 l = r。
- 数据规模要能卡出 std 的错: 小数据拍不到的 bug(溢出 / 越界 / 卡常), 把范围开到 1e5 ~ 1e6 再拍几轮。

打表

把"在线算不出来 / 算得慢"的答案在本地离线算好, 直接以常数数组的形式写进提交代码。

用途

- **规律题**: 答案随 n 变化、看不出公式 —— 先写暴力把小 n 的表打出来, 对着数列找规律(差分 / 比值 / 查 OEIS), 找到通式再实现。
- **预处理答案**: 能离线算的答案直接抄成 const 数组, 在线 O(1) 查表(某些计数题的真值、要跑几秒的预处理、记忆化搜索的转移表)。

做法: 离线程序输出常数数组

打表程序不用管运行时间(离线跑几分钟都行), 把答案按 C++ 数组源码的格式打印出来, 再粘进提交:

```cpp
// 打表示例: 长为 n 的 01 串中, 没有两个相邻的 1 的方案数(离线暴力枚举 2^n 种)
#include <bits/stdc++.h>
using namespace std;
int main() {
    for (int n = 1; n <= 20; n++) {
        int cnt = 0;
        for (int mask = 0; mask < (1 << n); mask++)
            if (!(mask & (mask << 1))) cnt++;
        printf("%d, ", cnt);
    }
    printf("\n");
    return 0;
}
```

离线跑出来:

```
2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711,
```

粘进提交:

```
// 下标 = n - 1(n = 1..20); 这串其实是斐波那契
const int f[] = {2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765,
                 10946, 17711};
```

- 打表程序**别提交**(在线跑不动), 提交里只留常数表 + 查表 / 特判逻辑。

暴力枚举边界

- 规律常常在 n 很小的时候不成立(n = 1, 2, 3): 表从 n 的最小值开始逐个枚举, 看清楚规律从哪一项起才对得上。
- 边界几项单独特判, 或者干脆把表的前几项写进提交(比推通式省事)。
- 通式写好后**逐项**和表核对, 别只看中间的大项; 也可以把表数当 brute 塞进上面那套对拍脚本自动对。
- 打表区间两端(极小 n / 极大 n)是规律最容易翻车的地方, 留出余量多打两项。

表过大时分段 / 差分

- **分段打表**: 只打稀疏的锚点(比如每 K 项存一个), 其余在程序里从锚点递推 / 暴力补出来。
- **差分**: 表是前缀和性质(单调、相邻差小)时改存差分数组, 用的时候前缀和还原, 常数更小。
- **对着表做差分找规律**: 一阶 / 二阶差分后变成常数 ⇒ 低阶多项式规律; 出现周期 ⇒ 循环节规律; 比值稳定 ⇒ 指数 / 等比规律。看出规律就不用打全表了。
- 表太大还可能撞上 OJ 的源代码长度限制, 也会让编译变慢, 按需压缩(分段 + 递推复原)。

交互题技巧

所属分类: 工具速查 用途: 比赛速查(不要求记忆, 用时查表) 备注: 2026-10-18 初稿, 待你核对

输出与刷新

交互题里每条输出都得**立刻**送到交互器手上, 否则交互器在等输入、你的程序也在等输入, 两边一起干等 —— 评测机上就判 Idleness limit exceeded(看起来像 TLE)。

```
cout << "? " << l << ' ' << r << endl;   // endl = 换行 + flush, 最省心
cin >> x;                                // 紧接着读回答

cout << "! " << ans << endl;             // 最后的答案也要换行
```

其他刷新写法:

```
cout << "? " << x << '\n' << flush;      // 先只换行, 再手动 flush
printf("? %d\n", x);
fflush(stdout);                          // C 写法

setvbuf(stdout, NULL, _IONBF, 0);        // 也可以在 main 开头直接关缓冲
```

关同步(常见快读模板)时注意:

- `ios::sync_with_stdio(false); cin.tie(nullptr);` 是快读标配, **交互题里要小心**: 默认 `cin` 与 `cout` 绑定, 读入前会自动 flush `cout`; 一旦 `cin.tie(nullptr)`, 这个自动 flush 就没了 ⇒ 每个询问后必须自己 `endl` / `flush` / `fflush(stdout)`。
- 反过来说: 只要没写 `cin.tie(nullptr)`, 即使把 `endl` 定义成 `'\n'`(本仓 `CF1999G2.cpp` 就这么干的)也常常能过 —— 靠的是读入时的自动 flush。**这是运气, 不是设计**: 交互题建议别 `#define endl '\n'`, 或者每个询问后显式 flush。
- 关同步后 `cout` 和 `printf` 各用各的缓冲, 混用顺序会乱, 而且 `fflush(stdout)` 刷不到 `cout` 的缓冲: 一个程序里只用一种输出方式。

常见题型

- **二分猜数型**: 每次询问拿到"某段范围的统计量", 用二分把答案范围折半。
  - 例 **CF1520F1**(Guess the K-th Zero (Easy version)): 隐藏一个 01 数组, `? l r` 回答区间里 1 的个数; 求第 k 个 0 的位置。做法: 每轮 `? 1 mid` 数出前 mid 个里 0 的个数, 与 k 比大小决定二分方向, O(log n) 次询问(本仓 `CF1520F1.cpp`)。
- **区间 / 点对查询型**: 每次询问一段区间或一对位置的相对信息, 在限次内定出答案。
  - 例 求数组最小值 / 最大值的位置: 每次 `? i j` 回答两个位置元素的大小关系(< / > / =), 拼的是"比逐个扫描的 2n-2 次更省": 两两配对, 每对 1 次询问分出 small / large 两堆, 再各自内部比出最值, 共约 3n/2 次 —— 卡的就是次数上界(本仓 `CF1999G2.cpp` 就是这个套路: 读 T 组, 每组读 n, 问完输出 `! MIN MAX`)。
    - 注: 本仓这个文件名对不上 CF1999G2 原题(那是 Ruler, 问 `? a b` 拿矩形面积猜 x), 协议更像 CF730B Minimum and Maximum, 待你核对。
- **随机化型**: 询问次数不够用 / 数据分布不明朗时, 随机采样把问题降到能处理的规模。
  - 例 **CF843B**(Interactive LowerBound): 沿 next 方向值单调递增的链表, `? i` 回答节点的值和 next; 先随机问 ~1000 个点, 取"值 < x 且最大"的采样点当起点, 再顺着 next 扫到第一个 ≥ x 的值(本仓 `CF843B.cpp`)。

注意事项

- 每条询问输出后**立即刷新**(`endl` / `flush` / `fflush(stdout)`): 忘记刷新的典型表现是本地手动喂数据"看着正常", 提交后卡住 → Idleness limit exceeded / TLE。
- **询问次数有上限**: 多问一次(边角特判最容易多问)直接 WA; 写之前先估一下最坏询问次数。
- 输出格式严格: 前缀(`?` / `!`)再参数, 每行以换行结尾, 别漏 `!`。
- **调试信息别走 stdout**: stdout 上多打一个字符都会被交互器当询问解析 → WA; 调试用 `cerr`(stderr 通常被忽略)。
- 输入格式以题面为准: 有的题只有一组数据, 有的开头给 `n` / `T`(如本仓 `CF1999G2.cpp` 读 `T` 组); 别套"stdin 多测"模板乱读, 读完一条回答就跑逻辑, 输出答案(`! ...`)后 `return 0`。
- 本地自写交互器(mock)测试, 见下。

本地 mock(自己写交互器)

评测机的交互器你看不到, 本地自己写一个: 它读你程序的每行询问、按隐藏数据回答, 顺便帮你数询问次数。C++ 搞双管道麻烦, Python 十几行就够(Windows 先 `g++ sol.cpp -o sol.exe`, 运行 `sol.exe`; Linux / macOS 先 `g++ sol.cpp -o sol`, 运行 `./sol`):

```python
# mock.py: 猜 1..100 里的隐藏数, 回答 '<' / '>' / '=', 最多 20 次询问
# 和编译产物放在同目录, 直接 python mock.py
# 两系统的编译产物名不同(Windows 是 sol.exe, Linux / macOS 是 sol), 下面自动选
import os
import subprocess

n, hidden, LIMIT = 100, 42, 20
sol = "sol.exe" if os.name == "nt" else "./sol"
p = subprocess.Popen([sol], stdin=subprocess.PIPE, stdout=subprocess.PIPE,
                     text=True, bufsize=1)

used = 0
for line in p.stdout:                    # 逐行读选手输出; 选手忘 flush 这里就会卡住
    line = line.strip()
    print(">", line)
    if line.startswith("?"):
        used += 1
        if used > LIMIT:
            print("too many queries")
            p.kill()
            break
        x = int(line.split()[1])
        answer = '<' if x < hidden else ('>' if x > hidden else '=')
        p.stdin.write(answer + "\n")
        p.stdin.flush()
    elif line.startswith("!"):
        got = int(line.split()[1])
        print("answer:", got, "correct" if got == hidden else "WRONG")
        break

p.wait()
```

- 选手**不 flush** 时, `for line in p.stdout` 会一直等着 —— 这正是评测机上卡住的原因, mock 顺手就能验证刷新写对没有。
- 改 `LIMIT` / `hidden` 就能测询问次数超限、边界值等情况。
- 询问次数只有个位数的题, 也可以人肉当交互器手动喂, 不非得写 mock。